UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

SISTEMA WEB PARA LA GESTIÓN DE CITAS MÉDICAS DE LA CLÍNICA VIDA SALUD

Materia : Sistemas de Información I

Docente : MSc. Ing. Angélica Gazón Cuéllar

Grupo: 17

Integrantes :

Durán Pérez Melissa	224052349
Vásquez Cordero Angélica Yandira	224029460

Santa Cruz – Bolivia 2-2026

Índice general

Capítulo preliminar. Perfil del proyecto 7
Introducción 7
Objetivo general 8
Objetivos específicos 8
Descripción del problema 9
Alcance y delimitación del primer ciclo 10
Base documental y método de inspección 12
Marco teórico 13
Sistemas de información y aplicaciones web 13
Ingeniería de requisitos, requisitos funcionales y de calidad 13
Proceso Unificado y articulación de los modelos 14
UML, actores y casos de uso 15
Modelos estructurales y de interacción 16
Persistencia relacional, PostgreSQL y concurrencia 17
Seguridad y control de acceso 18
Modelo de negocio 19
Contexto administrativo y participantes 19
Proceso de solicitud, reserva y seguimiento 19
Reglas del negocio recuperadas y límites de interpretación 22
Capítulo 1. Flujo de trabajo: captura de requisitos 23
1.1. Encontrar actores y casos de uso 23
1.1.1. Identificación de actores 23
1.1.2. Identificación de casos de uso 24
1.1.3. Diagramas de casos de uso 27
1.2. Priorizar casos de uso 30
1.3. Detallar un caso de uso 32
CU-REGISTRAR-CITA — Registrar cita médica 32
Especificaciones adicionales de casos de uso 35
1.4. Prototipar interfaz de usuario 38
1.5. Estructurar el modelo de casos de uso 44
Capítulo 2. Flujo de trabajo: análisis 46
2.1. Análisis de arquitectura 46
2.2. Analizar casos de uso 47
Realización de análisis de Registrar cita 50
2.3. Análisis de clases 51
2.4. Analizar paquetes 55
Capítulo 3. Flujo de trabajo: diseño 58

3.1. Diseño de arquitectura 58
3.1.1. Diseño físico de la arquitectura: diagrama de despliegue 59
3.1.2. Diseño lógico de la arquitectura: diagrama de paquetes 61
3.2. Diseñar casos de uso: diagramas de secuencia 62
Interpretación transversal de las secuencias 73
Capítulo complementario. Diseño de datos 74
Modelo lógico de datos 74
Modelo físico y diccionario de datos 76
Integridad referencial, unicidad e índices 82
Consistencia de intervalos y límites de concurrencia 83
Implementación física de la base de datos: script SQL 85
Capítulo complementario. Implementación del Ciclo 1 88
Herramientas de desarrollo y evidencia disponible 88
Arquitectura implementada y organización del repositorio 88
Integración de los módulos RF01–RF13 90
Capítulo complementario. Pruebas del Ciclo 1 92
Plan de pruebas 92
Matriz de pruebas funcionales 93
Resultados conocidos y verificaciones efectivamente realizadas 108
Pruebas pendientes y evaluación de riesgos 109
Capítulo complementario. Trazabilidad de modelos 111
Propósito y criterios de vinculación 111
Control interno de consistencia 114
Conclusiones 116
Recomendaciones 118
Referencias bibliográficas 120
Anexos 122
Anexo A. Inventario de evidencias y fuentes consultadas 122
Anexo B. Registro técnico de revisión 122
Anexo C. Modelo físico completo y fuentes editables 123
Relación entre la vista completa y las vistas impresas 123
Anexo D. Repositorio y código QR 124

Índice de tablas

Tabla 1 *Objetivos específicos y productos verificables* 8
Tabla 2 *Relación entre problemas del caso y respuesta del sistema* 10
Tabla 3 *Alcance solicitado, implementación y pendientes* 11
Tabla 4 *Flujos, responsabilidades metodológicas y artefactos del ciclo* 15
Tabla 5 *Reglas operativas del núcleo de citas* 22
Tabla 6 *Actores del sistema y disponibilidad del Ciclo 1* 23
Tabla 7 *Relación de requisitos y casos de uso* 24
Tabla 8 *Priorización razonada de casos de uso* 31
Tabla 9 *Identificación y condiciones del caso principal* 32
Tabla 10 *Flujo principal de Registrar cita médica* 33
Tabla 11 *Flujos alternativos y excepciones del caso principal* 33
Tabla 12 *Criterios de aceptación del caso principal* 35
Tabla 13 *CU-AUT — Iniciar sesión* 35
Tabla 14 *CU-PAC-ALTA — Registrar paciente* 36
Tabla 15 *CU-CAN — Cancelar cita* 36
Tabla 16 *CU-REP — Reprogramar cita* 36
Tabla 17 *CU-ASI — Registrar asistencia* 37
Tabla 18 *CU-HIS — Consultar historial* 37
Tabla 19 *CU-INF — Generar reportes* 38
Tabla 20 *Interfaces reales identificadas en el código* 39
Tabla 21 *Aspectos arquitectónicos críticos y asignación de responsabilidad* 47
Tabla 22 *Análisis del caso Iniciar sesión* 48
Tabla 23 *Análisis del caso Registrar paciente* 48
Tabla 24 *Análisis del caso Registrar cita médica* 48
Tabla 25 *Análisis del caso Reprogramar cita* 49
Tabla 26 *Análisis del caso Cancelar cita* 49
Tabla 27 *Responsabilidades de las clases conceptuales* 54
Tabla 28 *Cohesión y dependencias de los paquetes de análisis* 57
Tabla 29 *Correspondencia entre paquetes de diseño y archivos reales* 61
Tabla 30 *Diccionario físico de roles* 79
Tabla 31 *Diccionario físico de usuarios* 79
Tabla 32 *Diccionario físico de pacientes* 79
Tabla 33 *Diccionario físico de medicos* 80
Tabla 34 *Diccionario físico de especialidades* 81
Tabla 35 *Diccionario físico de medico_especialidad* 81
Tabla 36 *Diccionario físico de horarios* 81
Tabla 37 *Diccionario físico de citas* 82
Tabla 38 *Restricciones e índices relevantes* 82
Tabla 39 *Mapa de implementación por subsistema* 89

Tabla 40 *Componentes operativos del plan* 92
Tabla 41 *PF01 — Registrar paciente* 93
Tabla 42 *PF02 — Rechazar CI duplicado* 94
Tabla 43 *PF03 — Registrar cita en turno libre* 94
Tabla 44 *PF04 — Rechazar cita superpuesta* 95
Tabla 45 *PF05 — Rechazar reserva fuera del horario* 95
Tabla 46 *PF06 — Cancelar cita* 96
Tabla 47 *PF07 — Consultar historial de paciente* 96
Tabla 48 *PF08 — Consultar citas de médico por fecha* 97
Tabla 49 *PF09 — Denegar acceso no autorizado* 97
Tabla 50 *PF10 — Rechazar datos inválidos* 98
Tabla 51 *PF11 — Iniciar y cerrar sesión* 98
Tabla 52 *PF12 — Controlar permisos por rol* 99
Tabla 53 *PF13 — Registrar resultado atendida* 103
Tabla 54 *PF14 — Registrar inasistencia* 103
Tabla 55 *PF15 — Reprogramar integralmente* 104
Tabla 56 *PF16 — Validar estadísticas y filtros completos* 104
Tabla 57 *PF17 — Navegación adaptable* 105
Tabla 58 *PF18 — Verificar despliegue público* 105
Tabla 59 *PF19 — Validar médico y asignaciones* 106
Tabla 60 *PF20 — Validar unicidad de especialidad* 106
Tabla 61 *PF21 — Rechazar horario semanal solapado* 107
Tabla 62 *PF22 — Concurrencia de reservas y reprogramación* 107
Tabla 63 *Registro de evidencia y nivel de afirmación* 109
Tabla 64 *Trazabilidad de requisitos a modelos y pruebas — RF01 a RF07* 113
Tabla 65 *Trazabilidad de requisitos a modelos y pruebas — RF08 a RF13* 113
Tabla 66 *Materiales de referencia solicitados y disponibilidad* 122

Índice de figuras

Figura 1 *D04 — Actividad del proceso administrativo de reserva* 21
Figura 2 *D01 — Modelo general de casos de uso* 27
Figura 3 *D02 — Casos de uso del subsistema Gestión de Citas* 28
Figura 4 *D03 — Casos de uso del subsistema Administrativo* 29
Figura 5 *D20 — Organización del modelo de casos de uso por paquetes* 45
Figura 6 *D21 — Realización de análisis de Registrar cita* 50
Figura 7 *D05 — Clases de análisis: interacción, coordinación y dominio* 52
Figura 8 *D05B — Clases de análisis del contexto de seguridad* 53
Figura 9 *D06 — Paquetes del modelo de análisis* 56
Figura 10 *D07 — Despliegue físico del entorno local* 59
Figura 11 *D08 — Paquetes lógicos del diseño y correspondencia procedimental* 61
Figura 12 *D09 — Secuencia de inicio de sesión* 63
Figura 13 *D10A — Registrar cita: selección y consulta de disponibilidad* 64
Figura 14 *D10B — Registrar cita: validación del envío y del bloque* 65
Figura 15 *D10C — Registrar cita: resolución de la persistencia y errores* 66
Figura 16 *D16 — Secuencia de registro de paciente* 67
Figura 17 *D11 — Secuencia de reprogramación de cita* 68
Figura 18 *D12 — Secuencia de cancelación de cita* 69
Figura 19 *D13 — Secuencia de registro de asistencia e inasistencia* 70
Figura 20 *D14 — Secuencia de consulta del historial de citas* 71
Figura 21 *D15 — Secuencia de generación de reportes administrativos* 72
Figura 22 *D17 — Modelo lógico de datos* 75
Figura 23 *D18A — Modelo físico: seguridad y perfiles* 77
Figura 24 *D18B — Modelo físico: oferta y reservas* 78
Figura 25 *D19 — Trazabilidad entre los modelos del ciclo* 112
Figura 26 *Código QR del repositorio oficial de Clínica Vida Salud* 124

Capítulo preliminar. Perfil del proyecto

Introducción

La organización de las citas constituye un punto de articulación entre la demanda de atención y la capacidad disponible de una institución médica. Un registro aislado en papel o en archivos dispersos puede conservar datos de una solicitud, pero dificulta conocer de manera consistente qué profesional atiende, en qué especialidad y durante qué intervalo. La digitalización adquiere valor cuando transforma esa información en decisiones administrativas verificables: identificar al paciente, elegir una oferta de atención válida y conservar una reserva sin conflictos de horario.

En un escenario de gestión manual, dos personas pueden interpretar como libre un mismo turno; una cancelación puede quedar anotada en un lugar y no actualizarse en otro; y localizar el historial de solicitudes puede exigir revisar documentos separados. Estas situaciones se plantean como problemas del caso académico y no como resultados de una investigación de campo. No se dispone de entrevistas, encuestas ni estadísticas institucionales que permitan atribuir frecuencias o efectos cuantificados a una clínica real.

El Sistema Web para la Gestión de Citas Médicas de la Clínica Vida Salud centraliza la administración de pacientes, médicos, especialidades, horarios y citas. El primer ciclo incluye procedimientos de reserva, reprogramación, cancelación, registro del resultado de asistencia y consulta administrativa. Su implementación en PHP y PostgreSQL permite estudiar cómo los requisitos se convierten en reglas de validación, restricciones relacionales e interacciones de una aplicación web. La integridad de las reservas merece atención especial porque una pantalla de disponibilidad es una lectura temporal y no una garantía suficiente frente a solicitudes simultáneas.

La primera presentación del Ciclo 1 organiza los artefactos mediante el enfoque PUDS-UML establecido en la asignatura. Se mantiene el orden de los flujos de captura de requisitos, análisis y diseño mostrado en la pizarra, complementado con el diseño de datos, la implementación, las pruebas y la trazabilidad. Esta organización permite relacionar las necesidades del sistema con los modelos UML, los componentes de código y los criterios de verificación.

Objetivo general

Desarrollar y documentar un incremento del sistema web de gestión de citas médicas de la Clínica Vida Salud, mediante la captura de requisitos, el análisis y diseño con PUDS-UML y la implementación en PHP y PostgreSQL, que permita organizar pacientes y oferta médica, gestionar reservas y su seguimiento con control de acceso e integridad de intervalos, y establecer una base verificable para las pruebas y los ciclos posteriores.

Objetivos específicos

Tabla 1
Objetivos específicos y productos verificables

Código	Objetivo específico	Producto o criterio
OE01	Identificar requisitos y distinguir su fuente y grado de confirmación.	Catálogo RF01–RF13 y registro de discrepancias.
OE02	Identificar los cuatro actores y delimitar sus autorizaciones actuales y previstas.	Tabla de actores y diagramas D01–D03.
OE03	Modelar y priorizar los casos de uso según valor, dependencia y riesgo.	Matriz de prioridad y CU-REGISTRAR-CITA especificado.
OE04	Analizar las responsabilidades y procesos del núcleo de reservas.	Actividad D04, clases D05/D05B y realización D21.
OE05	Diseñar la arquitectura física y lógica coherente con el entorno local.	Despliegue D07 y paquetes D08.

Código	Objetivo específico	Producto o criterio
OE06	Describir el diseño de datos con campos, claves y restricciones reales.	Modelos D17–D18B y diccionario del esquema.
OE07	Relacionar la implementación procedimental con los requisitos y secuencias.	Mapa de archivos y matriz de trazabilidad.
OE08	Examinar los mecanismos de autenticación, autorización y protección de formularios.	Análisis de RF01 y pruebas PF09, PF11 y PF12 planificadas.
OE09	Explicar el registro de citas y la prevención de superposiciones.	Secuencias D10A/D10B y restricciones contrastadas con catálogo.
OE10	Definir pruebas funcionales y separar resultados reportados, observados y pendientes.	Plan, matriz PF01–PF22 y evidencia técnica conservada.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Descripción del problema

El problema central del caso es la falta de una fuente integrada para coordinar la información administrativa de las citas. Cuando los datos del paciente, la agenda del profesional y los cambios de programación se administran separadamente, la decisión de reservar depende de consultas manuales y de la actualización oportuna de cada registro. La causa no se reduce a la ausencia de una página web: implica ausencia de reglas compartidas, identificación consistente y control de los cambios de estado.

Una consecuencia relevante es el riesgo de reservar intervalos incompatibles para un mismo médico. Comparar únicamente la hora inicial no detecta todas las superposiciones; una cita de mayor duración puede invadir otra aun cuando sus horas iniciales sean diferentes. Otra consecuencia es la dificultad para localizar a los pacientes o distinguir registros activos de información que debe conservarse. Eliminar una ficha o una cita para reflejar una baja resolvería superficialmente la consulta actual, pero perjudicaría la continuidad del historial administrativo.

Las cancelaciones, inasistencias y atenciones realizadas modifican el significado de la reserva. Sin un estado normalizado, los reportes pueden mezclar demanda prevista con atención efectivamente registrada. La falta de consultas por paciente, médico y fecha limita la revisión administrativa. Estas necesidades justifican el núcleo funcional del sistema; no permiten concluir que se hayan reducido tiempos o errores en una institución real, pues no se cuenta con una medición anterior y posterior.

Tabla 2

Relación entre problemas del caso y respuesta del sistema

Problema planteado	**Causa administrativa**	**Respuesta y límite del Ciclo 1**
Datos dispersos y búsqueda lenta	Registros sin consulta centralizada	Pacientes con búsqueda por CI, nombres y apellidos; sin medición de tiempos reales.
Citas duplicadas o superpuestas	Disponibilidad no compartida y verificación incompleta	Bloques disponibles y EXCLUDE por médico; falta ensayo concurrente formal.
Cambios de horario difíciles de seguir	Actualizaciones manuales no coordinadas	Reprogramación conserva identidad; no existe auditoría de cada horario anterior.
Pérdida de antecedentes administrativos	Borrado para representar bajas	Desactivación y estados finales conservan citas; no constituye historia clínica.
Escasa información de gestión	Falta de agregación consistente	Reportes administrativos solo para administrador; batería completa pendiente.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Alcance y delimitación del primer ciclo

El enunciado oficial del segundo examen parcial establece cuatro roles y trece requisitos funcionales, RF01–RF13. Su contraste con el README, las páginas PHP y el esquema SQL permite distinguir obligaciones del curso, funcionalidades presentes y validaciones pendientes. Se identificó una diferencia en RF01: el enunciado admite inicio de sesión por nombre de usuario o correo electrónico, mientras que la consulta actual de login.php busca exclusivamente

nombre de usuario. Asimismo, la administración web de usuarios y las interfaces personales de médico y paciente requieren completarse.

En el código se encuentra un núcleo administrativo integrado: autenticación, pacientes, médicos, especialidades, horarios, registro y prevención de superposición de citas, reprogramación, cancelación, resultados de asistencia, agenda administrativa, historial y reportes. La presencia de esos módulos no se utiliza como sinónimo de aprobación funcional integral. Médico y paciente pueden autenticarse, pero index.php informa que sus funciones personales todavía no están integradas; no se les habilitan los módulos administrativos.

El entorno acreditado es local, con Apache mediante XAMPP, PHP 8.2.12 y PostgreSQL 18.6 observado mediante consulta de versión. La existencia del repositorio en GitHub no equivale a publicar la aplicación en la nube. La selección de infraestructura pública, instalación remota, HTTPS y pruebas de despliegue permanecen pendientes de confirmación.

La entrega se prepara para el hito académico que solicita un mínimo de 50 % de avance. Se presenta evidencia del núcleo funcional y de los artefactos de los tres flujos prioritarios; no se asigna un porcentaje global certificado, porque no existe en las fuentes disponibles una ponderación oficial de requisitos, pruebas y documentación. El cumplimiento de ese umbral deberá evaluarse con la rúbrica del curso, evitando calcularlo por simple conteo de archivos.

Tabla 3

Alcance solicitado, implementación y pendientes

Dimensión	**Disponible o inspeccionado**	**Pendiente o por confirmar**
Gestión administrativa	RF02–RF10 y RF12 con lógica procedimental y SQL.	Recorridos formales, valores de frontera y regresión integral.
Seguridad	Cuatro roles, sesión, CSRF, autorización en servidor.	Administración completa de usuarios web y matriz funcional exhaustiva.
Agenda y consultas	Agenda por médico/fecha para administrador y recepción.	Agenda propia del médico y consulta propia del paciente.

Dimensión	Disponible o inspeccionado	Pendiente o por confirmar
Reportes	Seis secciones, filtros, KPIs, gráficos CSS e impresión.	Validación completa de filtros; exportación como función opcional no confirmada.
Despliegue	Ejecución local y repositorio público accesible.	Instalación pública y evidencia de operación en la nube.
Fuentes académicas	Enunciado oficial, pizarra, material PUDS-UML y portada de referencia consultados en esta revisión.	Seguimiento de requisitos, evidencias de pruebas y actualizaciones de documentación.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Base documental y método de inspección

La elaboración combina inspección estática de los archivos versionados, contraste del esquema SQL con el catálogo de PostgreSQL y revisión de documentación técnica pública. Se registró el commit de referencia, posteriormente integrado a la rama principal main, se verificó la sintaxis de los 25 archivos PHP y se conservaron huellas SHA-256 de los archivos versionados. La consulta a PostgreSQL se ejecutó con BEGIN TRANSACTION READ ONLY y finalización mediante rollback; solo se obtuvieron metadatos y agregados anónimos de demostración.

La documentación se fundamenta en el enunciado oficial del segundo parcial, la fotografía de la pizarra y el material de 54 páginas de Gustavo Torossi sobre el Proceso Unificado (s. f.). La pizarra establece las actividades 1.1–1.5, 2.1–2.4 y 3.1–3.2; Torossi (s. f., pp. 26–45) desarrolla los flujos y artefactos de captura, análisis y diseño. La descripción técnica se apoya asimismo en los archivos PHP y SQL del repositorio, en el inventario de código y en los resultados de las consultas de solo lectura, diferenciando implementación observada de pruebas funcionales aún pendientes.

Marco teórico

Sistemas de información y aplicaciones web

Un sistema de información integra componentes tecnológicos, personas y procesos para recoger, organizar y utilizar información. Esta perspectiva evita reducir el proyecto a sus formularios: el registro del paciente, la decisión de recepción y la disponibilidad del médico participan conjuntamente en el resultado administrativo. Bourgeois (2019) distingue hardware, software, datos, personas y procesos; en Clínica Vida Salud, PostgreSQL conserva los datos y las personas autorizadas ejecutan procedimientos cuya validez depende de reglas compartidas.

Una aplicación web distribuye responsabilidades entre un cliente que presenta e intercambia información y un servidor que procesa solicitudes. MDN contributors (s. f.) explica el intercambio cliente-servidor mediante solicitudes y respuestas HTTP. En el proyecto, el navegador recibe HTML y CSS, ejecuta JavaScript y envía formularios; Apache ejecuta PHP, que consulta PostgreSQL mediante PDO. Las respuestas JSON de disponibilidad complementan la renderización de páginas. Esta arquitectura no requiere que la aplicación sea pública: el mismo modelo funciona en localhost.

Ingeniería de requisitos, requisitos funcionales y de calidad

La ingeniería de requisitos comprende identificación, análisis, especificación, validación y gestión de las necesidades y restricciones de un producto. SWEBOK (s. f.-a) destaca que un requisito debe ser verificable y que sus actividades se refinan durante el ciclo de vida. Para esta entrega, la inspección del código permite recuperar reglas implementadas, pero no convierte esa recuperación en un levantamiento con usuarios reales. Distinguir procedencia evita atribuir al negocio decisiones que podrían ser solo elecciones técnicas del prototipo.

Los requisitos funcionales expresan capacidades, como registrar una cita o consultar su historial. Los no funcionales restringen la solución o sus cualidades: autorización en servidor, integridad, mantenibilidad y adaptabilidad de la interfaz. RF07, aunque aparece en el catálogo funcional, actúa además como una regla transversal de consistencia. Su verificación requiere observar tanto la interfaz como la aceptación o rechazo de operaciones concurrentes en el motor, no únicamente comprobar que exista una consulta de disponibilidad.

Los requisitos de calidad se formulan aquí como criterios observables y no como promesas de rendimiento sin medida. Por ejemplo, una solicitud HTML sin sesión debe redirigirse al login y una identidad con rol no permitido debe recibir 403. No se establece un tiempo máximo de respuesta ni una disponibilidad porcentual porque no hay un acuerdo ni resultados que los sustenten. La definición de pruebas y evidencias permite transformar afirmaciones vagas en verificaciones futuras.

Proceso Unificado y articulación de los modelos

El enfoque PUDS-UML adoptado organiza el trabajo alrededor de los casos de uso, las decisiones arquitectónicas y la construcción incremental. En una lectura iterativa, requisitos, análisis, diseño, implementación y pruebas se relacionan en cada incremento, en lugar de constituir compartimentos que se completan definitivamente una sola vez. SWEBOK (s. f.-c) sustenta la necesidad de adaptar los procesos al contexto, identificar entradas y salidas, y reconocer la superposición de actividades técnicas.

En Clínica Vida Salud, Registrar cita es un hilo conductor arquitectónicamente significativo: necesita identidad autorizada, un paciente activo, una oferta médica consistente, reglas temporales y una reserva persistida sin superposición. Documentar ese caso permite verificar si cada modelo aporta una decisión distinta. El modelo de casos de uso expresa el

objetivo; el análisis asigna responsabilidades conceptuales; el diseño especifica interacciones y mecanismos; el código materializa los procedimientos; y las pruebas comprueban escenarios seleccionados.

Inicio, elaboración, construcción y transición son las fases del Proceso Unificado descritas por Torossi (s. f., pp. 5–9). Estas fases no deben confundirse con el Ciclo 1 académico de la asignatura. Esta presentación organiza un incremento ya desarrollado, conserva la trazabilidad de sus modelos y señala las pruebas y funciones pendientes. La documentación se contrastó con los flujos de trabajo de requisitos, análisis y diseño descritos en el material docente (Torossi, s. f., pp. 26–45).

Tabla 4

Flujos, responsabilidades metodológicas y artefactos del ciclo

Flujo	Responsabilidad metodológica	Artefacto de esta entrega	Relación posterior
Requisitos	Analista: delimitar necesidades y actores.	Catálogo, prioridades y especificaciones CU.	Orienta análisis y criterios de prueba.
Análisis	Analista: asignar responsabilidades conceptuales.	Clases boundary/control/entity y paquetes.	Explica qué debe resolver el diseño.
Diseño	Diseñador/arquitecto: decidir colaboración y mecanismos.	Despliegue, paquetes, secuencias y datos.	Permite contrastar implementación.
Implementación	Desarrollador: materializar procedimientos y persistencia.	Archivos PHP y SQL existentes inspeccionados.	Proporciona objetos verificables.
Pruebas	Responsable de verificación: planificar, ejecutar y evaluar.	PF01–PF22 y registros técnicos de lectura.	Retroalimenta requisitos y decisiones.

Nota. Los roles metodológicos describen responsabilidades; no asignan actividades históricas individuales no acreditadas a las integrantes.

UML, actores y casos de uso

UML es un lenguaje de modelado que permite expresar distintas vistas del sistema. La especificación 2.5.1 de Object Management Group (2017) proporciona el marco de notación; no

impone una tecnología de implementación. Por esa razón, emplear clases conceptuales en un proyecto procedimental es válido si se aclara su naturaleza y no se afirma que existen clases PHP equivalentes.

Un actor representa un rol externo que intercambia acciones con el sistema; no es idéntico a una tabla de usuarios ni a una persona concreta. Administrador, recepcionista, médico y paciente identifican expectativas distintas. Un caso de uso describe un objetivo observable de ese actor y una secuencia que conduce a un resultado; una validación aislada, como comprobar un entero, no necesita convertirse en un caso autónomo. include se reserva para comportamiento requerido reutilizado; extend se usaría para un comportamiento opcional en un punto de extensión definido. No se utiliza extend para representar cualquier cambio de estado de una cita.

Modelos estructurales y de interacción

El diagrama de clases de análisis organiza conceptos y responsabilidades con asociaciones y multiplicidades. Los estereotipos boundary, control y entity se emplean como convención de análisis: interacción con el usuario, coordinación de reglas y conceptos persistentes del dominio. El modelo no es un inventario de tablas ni una declaración de programación orientada a objetos. Sus relaciones ayudan a explicar por qué una reserva necesita un paciente y una asignación médico-especialidad válidos.

Los diagramas de paquetes agrupan elementos relacionados y muestran dependencias dirigidas. En el análisis se agrupan responsabilidades del problema; en el diseño se relacionan esas responsabilidades con páginas y funciones concretas. Los diagramas de secuencia ordenan mensajes entre líneas de vida y utilizan fragmentos alt u opt para condiciones y alternativas. Las etiquetas de los mensajes de diseño reproducen operaciones o responsabilidades verificadas en el código, sin inventar servicios ni llamadas asíncronas inexistentes.

El diagrama de despliegue muestra nodos físicos, entornos de ejecución, artefactos y conexiones. Esta vista distingue la estructura lógica de una instalación: tener presentación, control y persistencia no significa tener tres computadoras contratadas. En el entorno actual, esos roles de ejecución residen localmente. El diagrama de actividad complementa la vista técnica al expresar decisiones del proceso de reserva y responsabilidades de recepción y del sistema.

Persistencia relacional, PostgreSQL y concurrencia

Una base de datos relacional organiza datos mediante tablas, claves y restricciones. Las claves primarias identifican registros; las foráneas exigen referencias válidas; UNIQUE controla valores repetidos y CHECK limita condiciones de una fila. PostgreSQL Global Development Group (s. f.-a) distingue estas restricciones. En el sistema, la FK compuesta de citas a medico_especialidad expresa que no basta con que existan el médico y la especialidad por separado: debe existir su asignación.

La prevención de intervalos superpuestos exige otra clase de condición. PostgreSQL Global Development Group (s. f.-b) documenta los tipos de rango y las restricciones de exclusión. El esquema combina igualdad de id_medico con intersección de tsrange y un predicado de estados vigentes. El rango [inicio, fin) incluye el comienzo y excluye el final, de modo que dos turnos consecutivos pueden compartir el límite sin solaparse. La extensión btree_gist permite combinar la comparación del identificador escalar con el rango temporal.

La transacción delimita una operación que debe confirmarse o revertirse. Un SELECT previo puede mostrar un turno libre a dos sesiones; la exclusión del motor resuelve la incompatibilidad al intentar escribir. La integridad referencial y la exclusión no prueban automáticamente que un turno esté dentro del horario semanal: esa pertenencia se valida en PHP.

Esta distribución de garantías es central para el análisis de riesgo y para seleccionar pruebas de integración.

Seguridad y control de acceso

Autenticar significa determinar la identidad; autorizar significa decidir si esa identidad puede realizar una acción. The PHP Documentation Group (s. f.-b) describe password_verify como verificación de una contraseña contra un hash. En el proyecto, la sesión guarda identificadores y rol, pero no la contraseña ni el hash. La autorización se vuelve a contrastar con usuarios y roles en cada solicitud mediante cvs_usuario_actual.

Las consultas preparadas separan la estructura SQL de los valores enviados; su finalidad y uso se documentan en The PHP Documentation Group (s. f.-a). El token CSRF vincula una operación de formulario con la sesión y el escape HTML limita la interpretación de texto como marcado. Esos mecanismos atienden amenazas diferentes: usar PDO no reemplaza comprobar permisos, y ocultar enlaces no protege una URL. La evaluación del sistema debe conservar esa distinción y comprobar los controles en el servidor.

Modelo de negocio

Contexto administrativo y participantes

El negocio modelado es la coordinación administrativa de citas médicas del caso de estudio. El paciente origina una solicitud de atención y recibe información sobre la reserva; recepción identifica a la persona, consulta la oferta y ejecuta la gestión; el médico representa la capacidad profesional programada y el destinatario de la agenda; y el administrador mantiene catálogos, horarios y consultas de gestión. Esta descripción es conceptual y no acredita la existencia de trabajadores entrevistados ni procesos observados en una clínica real.

Es necesario distinguir al actor del negocio del usuario implementado. El paciente participa en el proceso aun cuando recepción opere la pantalla de reservas. Del mismo modo, el profesional participa en la atención, pero el módulo actual de resultados es administrativo y no una interfaz personal del médico. Esta distinción evita ampliar los permisos de la aplicación al dibujar el proceso de negocio.

Proceso de solicitud, reserva y seguimiento

La solicitud inicia con la identificación del paciente. Si no existe una ficha utilizable, recepción debe registrarla o revisar su estado antes de reservar. Luego elige especialidad, médico y fecha. La oferta depende de médicos activos y de sus asignaciones; la disponibilidad depende además del día de la semana, los horarios activos, su duración de turno y las reservas vigentes. La pantalla muestra opciones, pero la decisión definitiva se confirma únicamente al guardar con éxito.

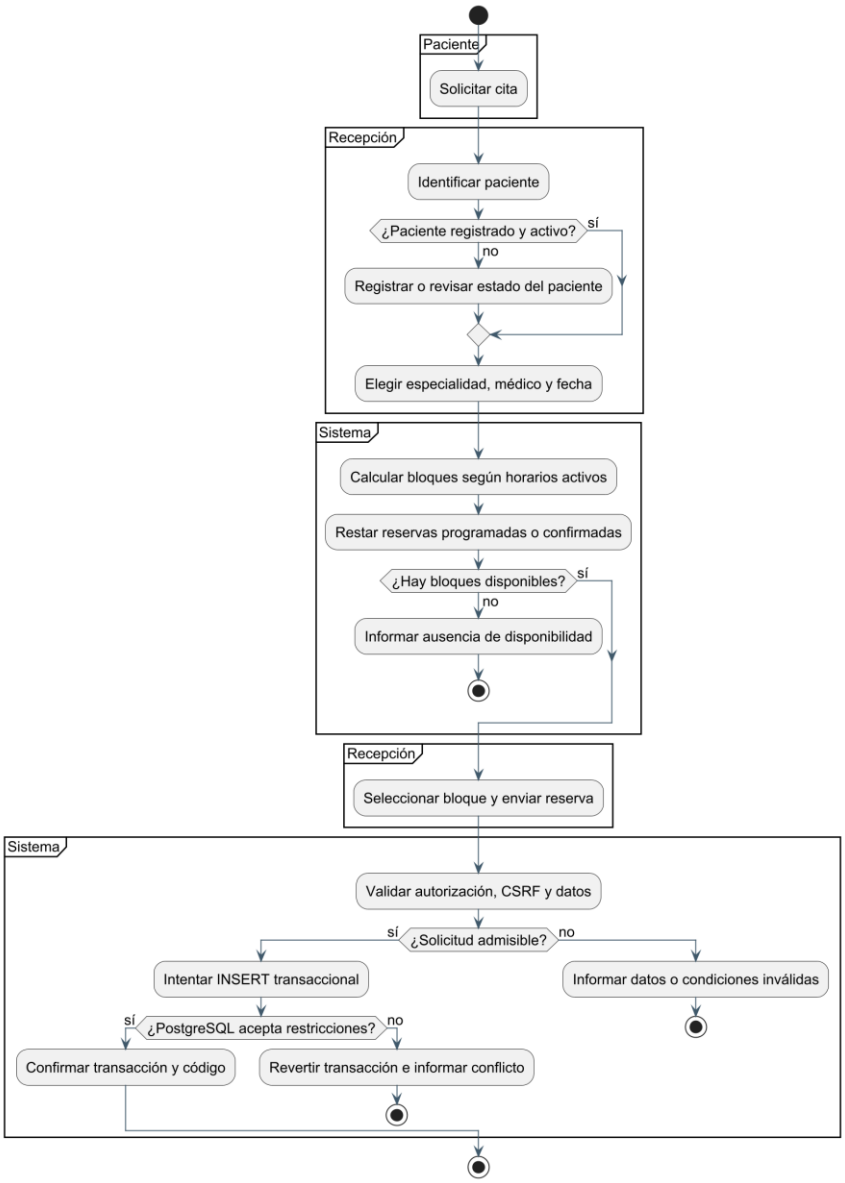
El registro válido produce una cita programada y un código. Una modificación posterior se realiza como reprogramación de fecha y horario del mismo registro, no como cambio de

médico o especialidad. Aunque confirmada es un estado admitido por el esquema y reconocido por las consultas, no se identificó una operación específica de confirmación en las páginas inspeccionadas. No se modela, por ello, una notificación automática ni una transición confirmada disponible que no estén implementadas.

Antes del inicio puede cancelarse una cita vigente mediante confirmación expresa. Después de su fin programado puede registrarse atendida o inasistencia. Los estados finales se conservan y no se rectifican desde el control de asistencia. El historial consulta las citas administrativas del paciente; los reportes agregan información de demanda y resultados. El sistema no documenta diagnósticos, prescripciones, facturación, pagos ni tratamientos clínicos como procesos funcionales del alcance.

Figura 1

D04 — Actividad del proceso administrativo de reserva



Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D04.puml.

Las particiones separan la solicitud del paciente, el trabajo de recepción y las verificaciones automáticas. Es un proceso conceptual respaldado por los módulos: registrar un paciente implica ir a pacientes.php y retornar, no un alta automática dentro de citas.php. La decisión final reconoce que la disponibilidad mostrada puede cambiar por concurrencia.

Reglas del negocio recuperadas y límites de interpretación

Tabla 5

Reglas operativas del núcleo de citas

Código	Regla observada	Ubicación y alcance
RN01	Una reserva exige paciente activo y médico activo.	Validación transaccional en citas.php.
RN02	El médico debe estar asignado a la especialidad elegida.	PHP y FK compuesta en citas.
RN03	El bloque debe alinearse a la duración desde el inicio del horario y caber completamente.	PHP en registro y reprogramación.
RN04	No se reservan horas pasadas ni fechas anteriores a hoy.	America/La_Paz en los módulos de reserva.
RN05	Reservas programadas/confirmadas de un mismo médico no se superponen.	EXCLUDE parcial en PostgreSQL; no es exclusión por paciente.
RN06	Cancelar conserva datos y exige cita vigente no iniciada.	RF09 con bloqueo y comprobación de reloj.
RN07	Atendida/inasistencia se registran solo después del fin programado.	RF10; estados finales no rectificables en ese módulo.
RN08	Reprogramar cambia fecha y horas y conserva la identidad de la cita.	RF08; sin bitácora de los horarios anteriores.
RN09	Desactivar pacientes o especialidades conserva relaciones históricas.	Cambios de activo, sin eliminación de citas.
RN10	Los reportes administrativos son exclusivos del administrador.	Autorización de reportes.php, no solo del menú.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Estas reglas se denominan recuperadas porque su procedencia comprobable es el código y el esquema. Su aceptación como política definitiva de la clínica requiere contraste con el enunciado y con las decisiones docentes. Por ejemplo, el sistema protege la agenda del médico contra superposición, pero no incluye una exclusión para impedir que un paciente tenga dos citas simultáneas con médicos distintos. Esa ausencia no debe ocultarse mediante una regla conceptual más amplia que la implementación.

Capítulo 1. Flujo de trabajo: captura de requisitos

1.1. Encontrar actores y casos de uso

El propósito de esta actividad es delimitar quién necesita interactuar con el producto y qué resultados espera obtener. La identificación se realizó mediante lectura del catálogo del README, las listas de roles admitidos, las guardas de acceso de cada página y los recorridos de formularios y consultas. Este procedimiento documental es adecuado para reconstruir el avance; no se lo presenta como entrevista, observación participante o validación con usuarios reales.

Cada actor se relaciona con un objetivo de uso, y luego se distingue entre capacidad administrativa actual y servicio personal previsto. La tabla de actores integra ambas perspectivas. Para el catálogo se conserva un identificador estable por caso, aun cuando un requisito abarque varios objetivos o una regla transversal contribuya a más de un caso.

1.1.1. Identificación de actores

Tabla 6
Actores del sistema y disponibilidad del Ciclo 1

Actor	Descripción y responsabilidades	Funciones previstas	Actualmente disponible
Administrador	Mantiene oferta médica, supervisa operación y consulta gestión.	Catálogos, pacientes, citas, usuarios y reportes.	Módulos administrativos; administración completa de usuarios web pendiente.
Recepcionista	Opera identificación de pacientes y coordinación de turnos.	Pacientes, registro, cambios y seguimiento de citas, consultas administrativas.	Pacientes, citas, reprogramación, cancelación, asistencia, agenda e historial.
Médico	Profesional asociado a una oferta y una agenda personal.	Consultar su agenda y funciones personales autorizadas.	Autenticación y aviso de integración pendiente; sin acceso a agendas administrativas.

Actor	Descripción y responsabilidades	Funciones previstas	Actualmente disponible
Paciente	Persona solicitante y titular conceptual de sus citas.	Consultar perfil y citas propias.	Autenticación y aviso de pantalla pendiente; sin consulta personal integrada.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Los cuatro valores técnicos son administrador, recepcionista, medico y paciente. No se añade un quinto rol visitante por la existencia de una portada anónima: el acceso público al inicio es una condición de navegación, no una cuenta registrada ni un perfil de autorización del esquema. Tampoco se modela PostgreSQL como actor humano; en las secuencias de diseño participa como infraestructura interna.

1.1.2. Identificación de casos de uso

Tabla 7

Relación de requisitos y casos de uso

RF	Caso de uso	Actor principal	Objetivo	Prioridad	Estado
RF01	CU-AUT; CU-SAL	Los cuatro roles	Identificar usuario y proteger recursos	Alta	Parcial: acceso y roles implementados; administración web de usuarios pendiente
RF02	CU-PAC; CU-PAC-ALTA	Administrador/recepcionista	Mantener fichas sin perder historial	Alta	Implementación inspeccionada; alta reportada; duplicado pendiente
RF03	CU-MED	Administrador	Mantener profesionales y asignaciones	Alta	Implementación inspeccionada; validación funcional integral pendiente
RF04	CU-ESP	Administrador	Organizar catálogo de atención	Alta	Implementación inspeccionada; validación funcional integral pendiente

RF	Caso de uso	Actor principal	Objetivo	Prioridad	Estado
RF05	CU-HOR; CU-DIS	Administrador	Definir oferta temporal y bloques	Alta	Implementación inspeccionada; disponibilidad reportada; fronteras pendientes
RF06	CU-REGISTRAR-CITA; CU-DIS	Administrador/recepcionista	Conservar una reserva válida	Alta	Implementación inspeccionada; registro válido reportado
RF07	CU-REGISTRAR-CITA; CU-REP; CU-DIS	Administrador/recepcionista	Mantener intervalos vigentes incompatibles excluidos	Alta	Restricción confirmada en catálogo; prueba concurrente pendiente
RF08	CU-REP; CU-DIS	Administrador/recepcionista	Cambiar turno sin cambiar identidad	Alta	Implementación inspeccionada; recorrido integral pendiente
RF09	CU-CAN	Administrador/recepcionista	Cerrar reserva antes de su inicio	Alta	Implementación inspeccionada; cancelación reportada; dato persistido confirmado
RF10	CU-ASI	Administrador/recepcionista	Registrar resultado administrativo final	Alta	Implementación inspeccionada; resultados reportados y agregados confirmados
RF11	CU-AGE; CU-MIA	Administrador/recepcionista; médico previsto	Consultar agenda por profesional/fecha	Media	Parcial: agenda administrativa; agenda personal médica pendiente
RF12	CU-HIS; CU-MIP	Administrador/recepcionista; paciente previsto	Consultar citas del paciente	Media	Historial administrativo implementado y reportado; consulta personal pendiente
RF13	CU-INF	Administrador	Obtener información administrativa agregada	Media	Implementación inspeccionada; consulta reportada; batería completa pendiente

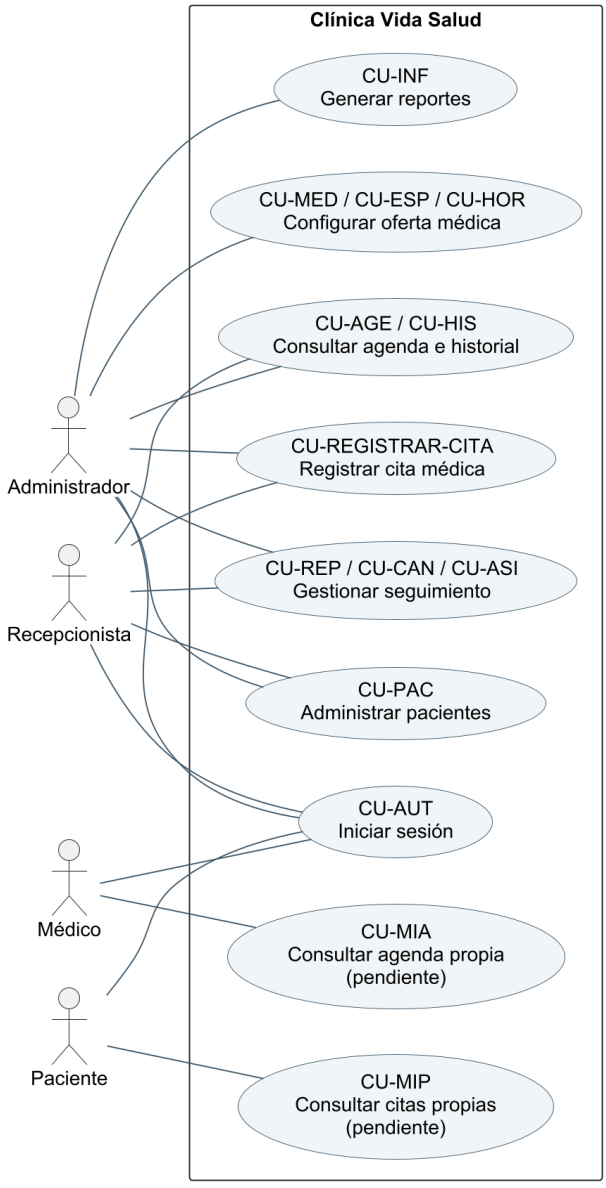
Nota. Nombres RF derivados del README y del código. Correspondencia oficial pendiente de disponer del enunciado; prioridad razonada documentalmente.

RF07 no origina obligatoriamente un caso visible independiente: la prevención de superposición es una garantía de Registrar cita y Reprogramar cita. RF02, en cambio, incluye alta, edición, búsqueda y cambio de actividad, por lo que CU-PAC es un resumen y CU-PAC-ALTA detalla el alta. RF01 conecta inicio/cierre de sesión con la autorización transversal. Esta relación muchos a muchos evita una trazabilidad artificial de un requisito por una sola pantalla.

CU-DIS identifica Consultar disponibilidad; CU-SAL, Cerrar sesión; CU-MIA, Consultar agenda propia; CU-MIP, Consultar citas propias; y CU-USU, Administrar usuarios web. Los tres últimos se conservan como alcance pendiente. Los identificadores CU-PAC, CU-MED, CU-ESP, CU-HOR, CU-REP, CU-CAN, CU-ASI, CU-AGE, CU-HIS y CU-INF se utilizan sin variaciones en matrices y figuras.

1.1.3. Diagramas de casos de uso

Figura 2
D01 — Modelo general de casos de uso



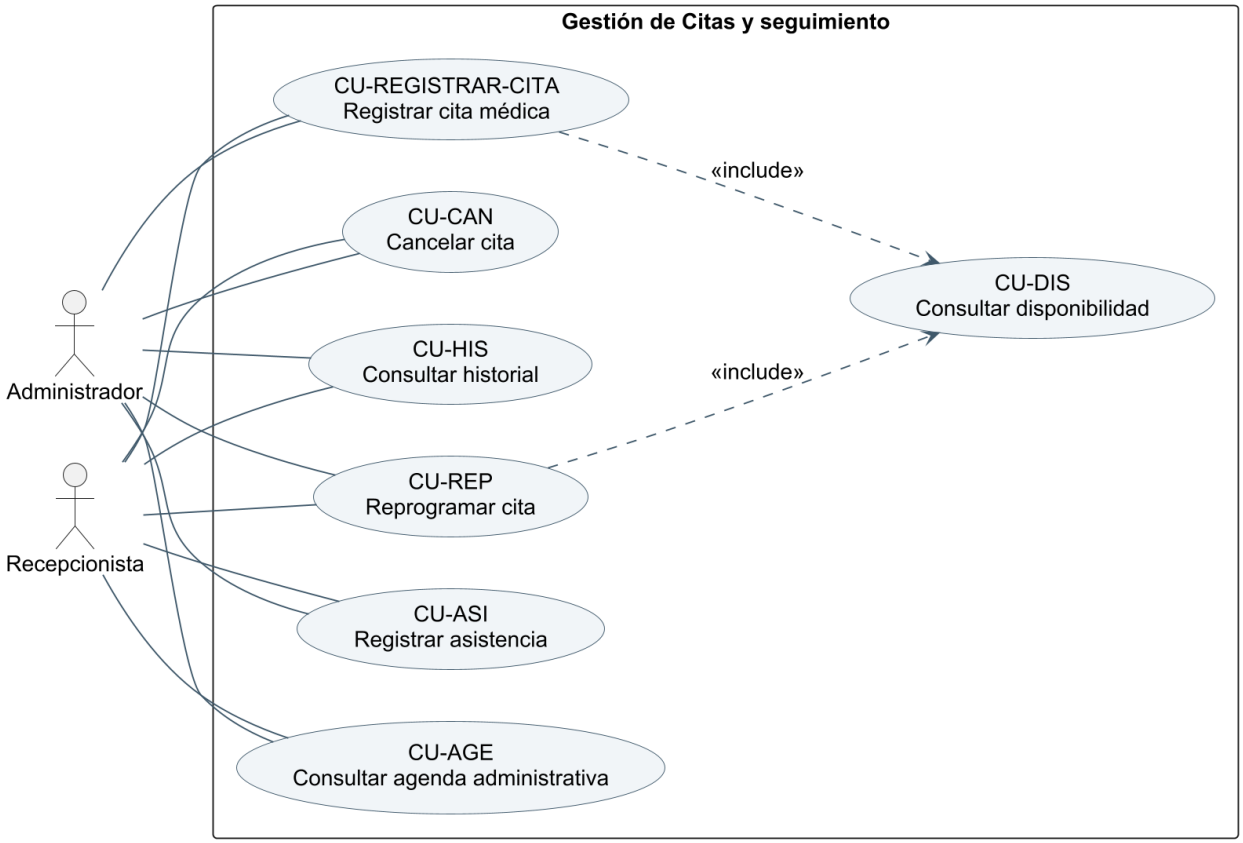
Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D01.puml.

La vista de contexto reúne objetivos de usuario y conserva los cuatro actores del alcance. Las agrupaciones CU-MED/CU-ESP/CU-HOR y CU-REP/CU-CAN/CU-ASI se descomponen en los diagramas D02 y D03. Las asociaciones de médico y paciente con sus consultas propias

representan alcance pendiente, no autorización disponible. Iniciar sesión no se modela como inclusión de cada operación: es una precondición de acceso.

Figura 3

D02 — Casos de uso del subsistema Gestión de Citas

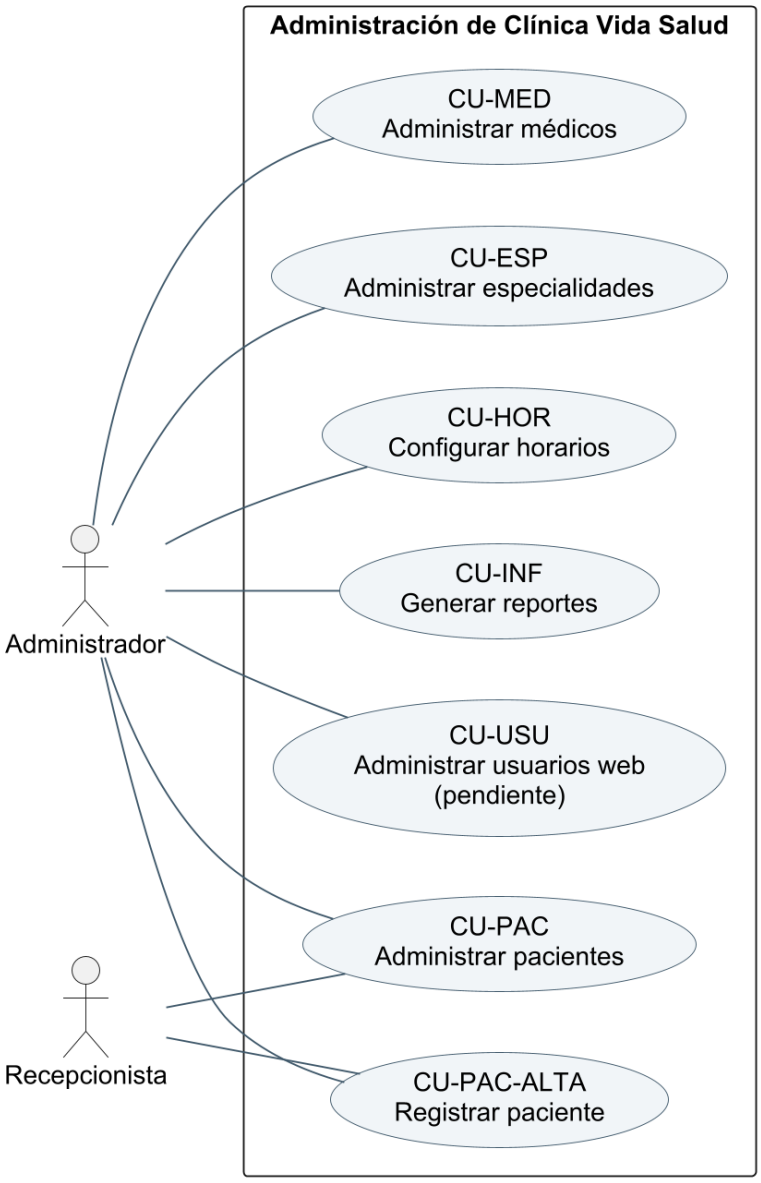


Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D02.puml.

La consulta de disponibilidad es un comportamiento requerido tanto para reservar como para reprogramar mediante la interfaz implementada. La flecha de inclusión parte del caso que necesita el comportamiento y apunta al caso incluido. Cancelar y registrar asistencia son objetivos independientes, con distintas guardas temporales; no son extensiones automáticas del registro.

Figura 4

D03 — Casos de uso del subsistema Administrativo



Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D03.puml.

La recepcionista administra pacientes y participa en su alta; el catálogo médico, las especialidades, los horarios y los reportes se restringen al administrador. CU-PAC resume el mantenimiento y CU-PAC-ALTA explicita el escenario de alta detallado. CU-USU conserva una necesidad pendiente: los scripts de consola no equivalen a una pantalla web de administración de cuentas.

La legibilidad se obtiene mediante una vista de contexto y dos descomposiciones funcionales. El límite del sistema separa objetivos externos de mecanismos internos. La autenticación se representa como objetivo propio y precondición de los casos protegidos; se evita dibujar include desde todas las funciones hacia Iniciar sesión, porque una sesión ya vigente no exige repetir el acceso en cada operación. Las figuras no autorizan funciones futuras: el estado pendiente está explícito y se complementa con la tabla de actores.

1.2. Priorizar casos de uso

La priorización busca construir y validar primero una base que reduzca riesgos de integración. Se valora la importancia para el objetivo de reservar, la complejidad de reglas y dependencias, y el impacto de una falla. Alta, media y baja son categorías cualitativas de esta revisión, no resultados de entrevistas ni estimaciones de horas verificadas. La disponibilidad y el control de superposición reciben prioridad alta porque un error puede producir resultados inconsistentes aun cuando la interfaz parezca correcta.

La secuencia razonada del primer ciclo comprende acceso seguro y catálogos mínimos, identificación del paciente, definición de horarios, reserva consistente y seguimiento. Los reportes aprovechan datos ya integrados. Aunque hay implementación de todos esos grupos, la planificación de pruebas debe volver al núcleo crítico antes de ampliar pantallas personales. El enfoque incremental admite refinar requisitos y ejecutar pruebas en cada recorrido; no exige finalizar toda la captura antes de examinar un riesgo arquitectónico.

Tabla 8

Priorización razonada de casos de uso

CU	Importancia	Complejidad	Riesgo	Justificación	Iteración propuesta
CU-AUT / CU-SAL	Alta	Media	Alto	Condiciona identidad, acceso y uso del resto.	C1-A: base segura
CU-PAC-ALTA / CU-PAC	Alta	Media	Medio	La reserva depende de una ficha válida y localizable.	C1-A: datos básicos
CU-ESP / CU-MED	Alta	Media	Medio	Permiten una asignación médico-especialidad válida.	C1-A: oferta
CU-HOR	Alta	Alta	Alto	Define bloques, límites y solapamientos semanales.	C1-B: oferta temporal
CU-REGISTRAR-CITA / CU-DIS	Alta	Alta	Alto	Integra permisos, fechas, oferta, persistencia y concurrencia.	C1-B: núcleo
CU-REP	Alta	Alta	Alto	Modifica un intervalo y requiere preservar consistencia.	C1-C: seguimiento
CU-CAN / CU-ASI	Alta	Alta	Alto	Transiciones temporales y conservación histórica.	C1-C: seguimiento
CU-HIS / CU-AGE	Media	Media	Medio	Reutilizan datos; deben preservar ámbito y filtros.	C1-D: consultas
CU-INF	Media	Alta	Medio	Agregados y filtros requieren contraste con datos.	C1-D: gestión
CU-MIA / CU-MIP / CU-USU	Alta para completar alcance	Alta	Alto	Necesitan propiedad de datos y administración de cuentas.	Ciclo posterior; pendiente

Nota. C1-A a C1-D son agrupaciones propuestas para explicar dependencias del incremento, no fechas ni iteraciones históricas acreditadas.

1.3. Detallar un caso de uso

CU-REGISTRAR-CITA — Registrar cita médica

El caso principal especifica el contrato entre una persona autorizada y el sistema para producir una reserva consistente. Su descripción reúne comportamiento de interfaz y validación de servidor sin confundirlos: la interfaz orienta la selección, mientras el POST y PostgreSQL deciden si la cita puede conservarse. Los requisitos principales son RF06 y RF07; RF01, RF02, RF03, RF04 y RF05 aportan capacidades previas necesarias.

Tabla 9
Identificación y condiciones del caso principal

Elemento	**Especificación**
Identificador y nombre	CU-REGISTRAR-CITA — Registrar cita médica.
Objetivo	Crear una cita programada para paciente y oferta válidos, sin superposición vigente del mismo médico.
Actor principal	Recepcionista; el administrador puede ejecutar el mismo objetivo.
Actores secundarios	Paciente como solicitante del negocio, sin operación directa de la pantalla actual. No se identifica un sistema externo de notificación.
Descripción	Seleccionar paciente, especialidad, médico, fecha y bloque; validar y persistir una reserva con código.
Precondiciones	Sesión válida; rol administrador/recepcionista; datos de oferta disponibles; paciente activo registrado para completar la reserva.
Disparador	El actor abre Registrar citas para atender una solicitud de reserva.
Entradas	id_paciente, id_especialidad, id_medico, fecha, hora_inicio, motivo opcional y token csrf_citas.
Postcondición de éxito	Una fila en citas con código único, estado programada y hora_fin calculada; transacción confirmada.
Garantía de rechazo	La reserva intentada no se confirma; si la transacción comenzó, se revierte y se muestra el error aplicable.
Restricciones	Entorno local; operación administrativa; sin cambio de médico mediante reprogramación; sin notificación externa.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 10

Flujo principal de Registrar cita médica

Paso	Acción del actor o del sistema
1	El actor accede a citas.php; el servidor exige identidad y rol permitido.
2	El sistema carga pacientes activos y especialidades activas para los selectores.
3	El actor identifica al paciente y selecciona una especialidad.
4	JavaScript solicita por fetch el endpoint de médicos; el servidor devuelve médicos activos de esa asignación.
5	El actor selecciona médico y fecha actual o futura.
6	El endpoint de bloques consulta horarios activos del día y reservas programadas/confirmadas; genera bloques completos y excluye pasados u ocupados.
7	El actor elige un bloque, introduce motivo opcional y envía el formulario POST.
8	El servidor verifica CSRF, identificadores, fecha estricta, formato de hora, temporalidad y longitud del motivo.
9	El servidor inicia transacción y verifica paciente activo, médico activo y asignación médico-especialidad.
10	El servidor recupera horarios del día; valida pertenencia y alineación del bloque y calcula su hora final.
11	Genera código e intenta INSERT preparado en citas con estado programada; PostgreSQL aplica claves, CHECK y EXCLUDE.
12	Si el motor acepta, confirma transacción, conserva un aviso y redirige al listado con el código de la cita.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 11

Flujos alternativos y excepciones del caso principal

Código y punto	Condición	Respuesta real y continuación
A01; paso 3	El paciente no existe o no está activo.	No aparece entre pacientes activos; debe registrarse o revisarse en pacientes.php. Un ID inexistente/inactivo enviado al POST se rechaza; no hay alta automática.
A02; paso 6	No hay horarios o bloques disponibles.	El endpoint devuelve lista vacía y la interfaz informa ausencia de bloques. Cambiar fecha o médico; no se registra cita.

Código y punto	Condición	Respuesta real y continuación
A03; paso 8	Fecha inexistente, anterior a hoy o hora pasada de hoy.	Validación estricta y mensaje de fecha/hora no válida; no comienza la inserción.
A04; paso 9	El médico está inactivo o no existe.	La consulta de médico activo falla y se revierte con mensaje Médico inválido o inactivo.
A05; paso 9	El médico no atiende la especialidad elegida.	Se rechaza por falta de asignación; la FK compuesta también protege la referencia al guardar.
A06; paso 10	No tiene horario activo, bloque fuera de rango o desalineado.	Se revierte y se informa la regla temporal incumplida.
E01; paso 11	El bloque mostrado ya fue reservado por otra sesión.	PostgreSQL puede emitir 23P01; rollback e indicación de elegir otro bloque disponible.
E02; paso 11	El motor rechaza una clave única o referencia.	23505 informa conflicto de código; 23503 informa restricción de integridad; no hay confirmación.
E03; pasos de consulta/guardado	Error de base de datos no tipificado.	Se muestra error genérico correspondiente; rollback si procede. No se declara éxito.
E04; acceso o envío	Sesión ausente, rol no permitido o token inválido.	HTML sin sesión redirige; JSON anónimo 401; rol no permitido 403; CSRF inválido genera rechazo del formulario.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

El sistema lista especialidades activas en la interfaz, pero la validación transaccional del POST comprueba la existencia de la asignación y no consulta expresamente activo de especialidades. Por tanto, no se afirma que una especialidad desactivada entre la carga de la página y el envío sea rechazada por una regla específica de actividad. Esta condición debe contrastarse con la política esperada y ensayarse. En cambio, paciente y médico sí reciben comprobación explícita de activo en el procedimiento de registro.

El cálculo de hora_fin se realiza en servidor y no se acepta como una duración arbitraria enviada por el navegador. La comprobación de alineación usa la diferencia de minutos respecto del inicio del horario y la duración de turno. La regla de exclusión considera únicamente las citas

programadas o confirmadas del mismo médico; una cita cancelada, atendida o con inasistencia no bloquea esa restricción. La garantía no se extiende a impedir superposición por paciente ni a mantener una auditoría de cada intento.

Tabla 12
Criterios de aceptación del caso principal

Código	Condición observable	Prueba vinculada
CA01	Con datos válidos y bloque libre se conserva una cita programada y se comunica su código.	PF03
CA02	Un paciente inexistente/inactivo, un médico inactivo o una asignación inválida no producen reserva confirmada.	PF10
CA03	Fecha inexistente, hora pasada, bloque fuera del horario o desalineado se rechazan.	PF05; PF10
CA04	Dos intentos de reservar intervalos superpuestos del mismo médico no dejan dos reservas vigentes.	PF04; PF22
CA05	La operación rechazada por PostgreSQL revierte y muestra un resultado de error.	PF04; PF10
CA06	Acceso sin permiso o token inválido no permite la operación protegida.	PF09; PF12

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Especificaciones adicionales de casos de uso

Tabla 13
CU-AUT — Iniciar sesión

Elemento	Especificación
Actor	Cualquiera de los cuatro roles
Precondiciones	Cuenta existente activa y rol reconocido; formulario con token.
Flujo principal	Introducir nombre y contraseña; validar entrada y límite; consultar cuenta/rol; verificar hash; regenerar sesión y redirigir.
Alternativas y excepciones	CSRF inválido, vacío, bloqueo o credenciales incorrectas producen aviso sin identidad autenticada. Cinco fallos bloquean quince minutos por sesión.

Elemento	Especificación
Postcondición	Identidad en sesión sin contraseña; no habilita por sí sola módulos personales pendientes.
Trazas	RF01; D09; PF11

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 14

CU-PAC-ALTA — Registrar paciente

Elemento	Especificación
Actor	Administrador o recepcionista
Precondiciones	Sesión autorizada y datos de ensayo válidos.
Flujo principal	Completar nombres, apellidos, nacimiento y teléfono; validar campos opcionales; normalizar vacíos; INSERT preparado.
Alternativas y excepciones	Fecha futura, correo inválido, longitudes o campos vacíos generan errores. UNIQUE de CI provoca mensaje de duplicado.
Postcondición	Ficha activa nueva; CI opcional. No se crea automáticamente cuenta de usuario.
Trazas	RF02; D16; PF01–PF02

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 15

CU-CAN — Cancelar cita

Elemento	Especificación
Actor	Administrador o recepcionista
Precondiciones	Cita programada/confirmada no iniciada; confirmación expresa.
Flujo principal	Consultar candidata; confirmar con motivo opcional; validar token; bloquear cita; revalidar usuario y tiempo; actualizar estado y confirmar.
Alternativas y excepciones	Estado final, inicio alcanzado, datos inválidos o falta de permiso impiden cancelar. Motivo limitado a 500 caracteres por PHP.
Postcondición	Cita conservada cancelada, motivo y fecha de actualización; intervalo deja de ocupar exclusión.
Trazas	RF09; D12; PF06

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 16

CU-REP — Reprogramar cita

Elemento	Especificación
Actor	Administrador o recepcionista

Elemento	Especificación
Precondiciones	Cita vigente futura y médico activo; mismo médico/especialidad.
Flujo principal	Seleccionar candidata; consultar bloques de nueva fecha excluyendo la propia cita; validar POST; bloquear fila; actualizar fecha y horas.
Alternativas y excepciones	Nueva fecha inválida, bloque no admisible, estado cambiado o 23P01 producen rollback.
Postcondición	Mismo ID y código; nueva fecha y horas; no hay tabla de auditoría de cambios.
Trazas	RF08; RF07; D11; PF15

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 17

CU-ASI — Registrar asistencia

Elemento	Especificación
Actor	Administrador o recepcionista
Precondiciones	Cita vigente; fin programado alcanzado; confirmación expresa.
Flujo principal	Seleccionar atendida o inasistencia; validar token y resultado; bloquear cita; revalidar usuario y reloj; actualizar estado.
Alternativas y excepciones	No se acepta una cita aún no finalizada, cancelada o con resultado final; no hay rectificación desde el módulo.
Postcondición	Resultado final conservado sin modificar motivos ni horarios de la cita.
Trazas	RF10; D13; PF13–PF14

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 18

CU-HIS — Consultar historial

Elemento	Especificación
Actor	Administrador o recepcionista
Precondiciones	Paciente identificable; parámetros GET válidos.
Flujo principal	Buscar o recibir id_paciente desde pacientes; aplicar filtros; contar total/filtrado; acotar página; consultar detalle de cincuenta filas.
Alternativas y excepciones	Parámetros desconocidos, fechas inválidas o estado ajeno al dominio impiden consultar citas; paciente inexistente genera aviso.
Postcondición	Vista administrativa de citas del paciente, con registros futuros y finales; sin modificación de datos.
Trazas	RF12; D14; PF07

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 19
CU-INF — Generar reportes

Elemento	Especificación
Actor	Administrador
Precondiciones	Sesión administrativa; sección y filtros válidos.
Flujo principal	Elegir tipo y filtros GET; validar; consultar agregados y detalle; mostrar KPIs y gráficos; imprimir si se solicita.
Alternativas y excepciones	Rango invertido o fechas/estado inválidos se informan. Un rol distinto recibe 403. Fallos de consulta no equivalen a datos vacíos válidos.
Postcondición	Resultados de lectura; pacientes sin período de registro; agenda con médico/fecha propios.
Trazas	RF13; D15; PF16

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

1.4. Prototipar interfaz de usuario

El propósito del prototipado en el ciclo es hacer observable la interpretación de los requisitos y facilitar la revisión de navegación, formularios y retroalimentación. SWEBOK (s. f.-a) reconoce su utilidad para validar modelos y expectativas. En esta entrega se dispone del código de interfaces implementadas, pero no de capturas auténticas previamente incorporadas al proyecto. Las imágenes de assets/img son recursos visuales de la aplicación y no se presentan como capturas de ejecución.

La interfaz inspeccionada adopta Dark Accent: el marco lateral utiliza azul petróleo, con acentos turquesa y fondos claros en las áreas de trabajo. La hoja CSS define #04222B y #0A3D4A para tonos oscuros, #1D8B94 y #4FB8B4 para acentos, y una tipografía de sistema que prioriza Segoe UI y familias sans serif. Esta tipografía de la aplicación se distingue de Times New Roman, utilizada en el informe académico.

includes/ui.php produce la marca, barra lateral, topbar, identidad visible y formulario de cierre de sesión. Los enlaces se calculan por rol y el módulo activo recibe señal visual. La

navegación consistente reduce cambios innecesarios de contexto, pero el control real permanece en auth.php y en cada entrada protegida. El menú móvil utiliza una casilla y un velo asociados a una etiqueta; las media queries ajustan disposición y tamaño. Su existencia es evidencia de diseño adaptable, no una certificación de compatibilidad móvil integral.

Tabla 20

Interfaces reales identificadas en el código

Interfaz	**Organización y controles**	**Validación y retroalimentación**	**Evidencia y límite**
Inicio de sesión	Área de marca y tarjeta con usuario/contraseña.	CSRF, obligatorios, mensaje genérico y bloqueo por sesión.	login.php; sin captura de ejecución autenticada conservada.
Dashboard	Banner, KPIs, calendario con intensidad, disponibilidad y actividad.	Aviso ante fallo de consulta; accesos por rol.	index.php e includes/dashboard.php; conteos de citas incluyen estados finales.
Pacientes	Formulario, búsqueda, estado y tabla con acciones.	Errores por campo, mensajes de alta, activación/desactivación y enlace al historial.	pacientes.php; listado hasta cien filas.
Registro de citas	Selectores dependientes y botones de bloques; motivo opcional.	fetch con JSON; errores cliente y servidor; resultado con código.	citas.php; listado hasta doscientas citas.
Reprogramación	Selección paginada; detalle actual y nueva fecha/bloque.	Bloques excluyen la propia reserva; servidor conserva identidad.	seleccionar_reprogramacion.php y reprogramar_cita.php.
Historial	Buscador de pacientes, filtros y tabla paginada.	Validación estricta GET, avisos, total y filtrado.	historial_citas.php; no es historia clínica.
Reportes	Secciones, filtros, KPIs, gráficos CSS, tablas e impresión.	Errores y avisos de filtros; paginación y estado de consulta.	reportes.php; solo administrador.

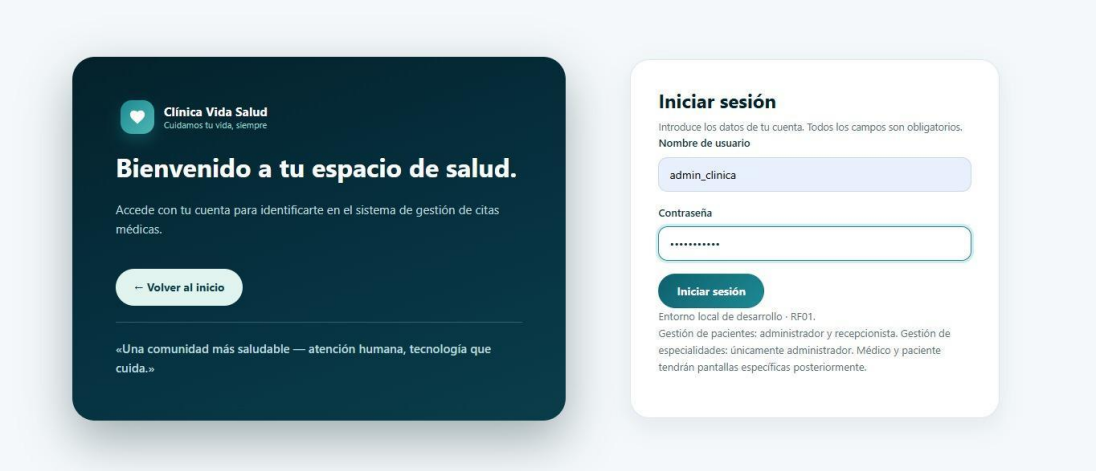
Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Los formularios administrativos utilizan controles nativos y protección CSRF para las operaciones de cambio. La cancelación y el resultado de asistencia exigen una confirmación expresa; los listados y filtros de consulta utilizan GET. El escape de salida se observa en funciones como cvs_escapar y e. Hay diferencias de implementación entre módulos —tokens

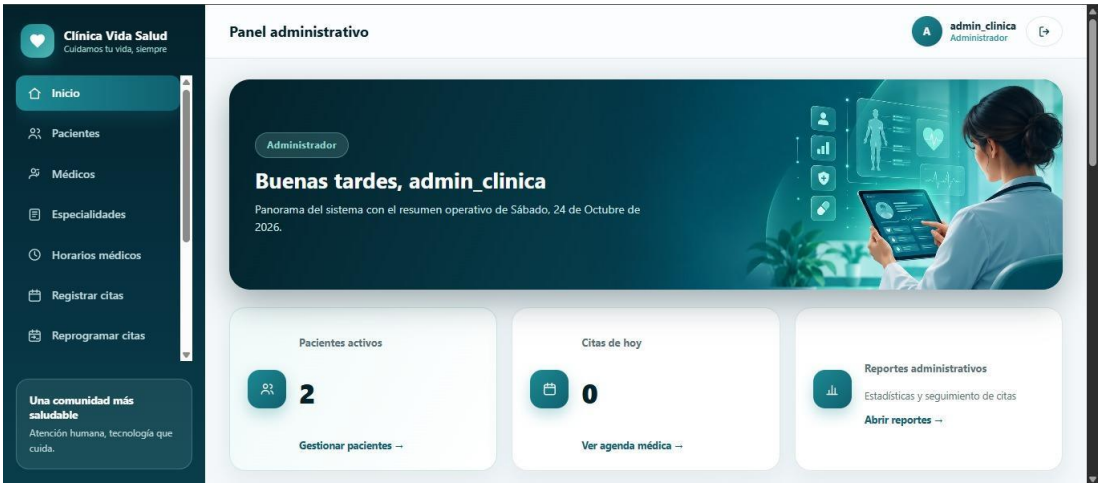
específicos frente al token común— que se documentan como variaciones técnicas, no como una protección uniformemente idéntica.

La revisión estática también identifica cadenas visuales deterioradas en el listado de citas, por ejemplo etiquetas de código y médico con caracteres alterados. Se conserva el hallazgo sin modificar la interfaz en esta tarea. La evaluación de contraste, uso por teclado, lector de pantalla, zoom y dispositivos reales queda pendiente de pruebas. No se fabrican pantallas ni se usa un wireframe como evidencia de ejecución; el inventario de interfaz permite revisar el diseño sin atribuir capturas inexistentes.

Pantalla de inicio de sesión



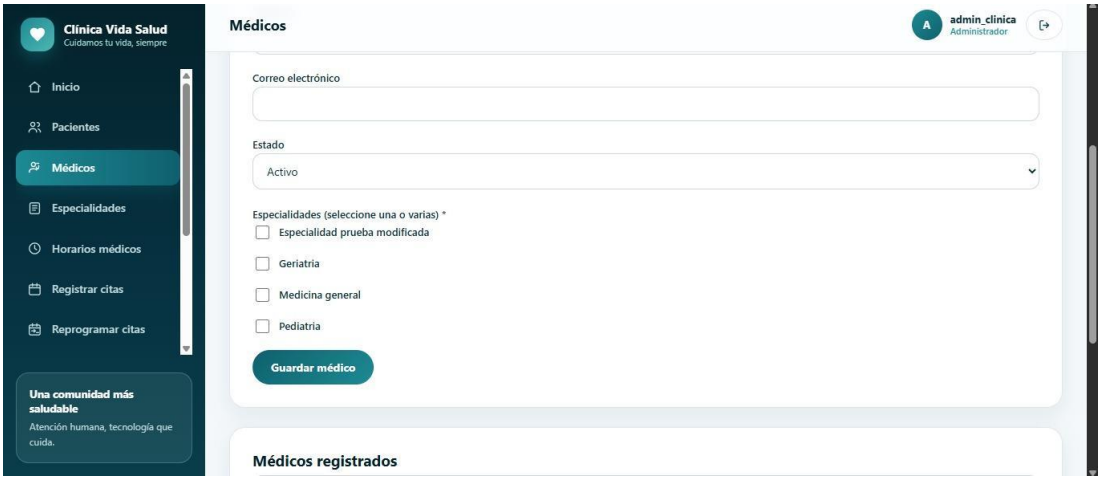
Panel principal o dashboard



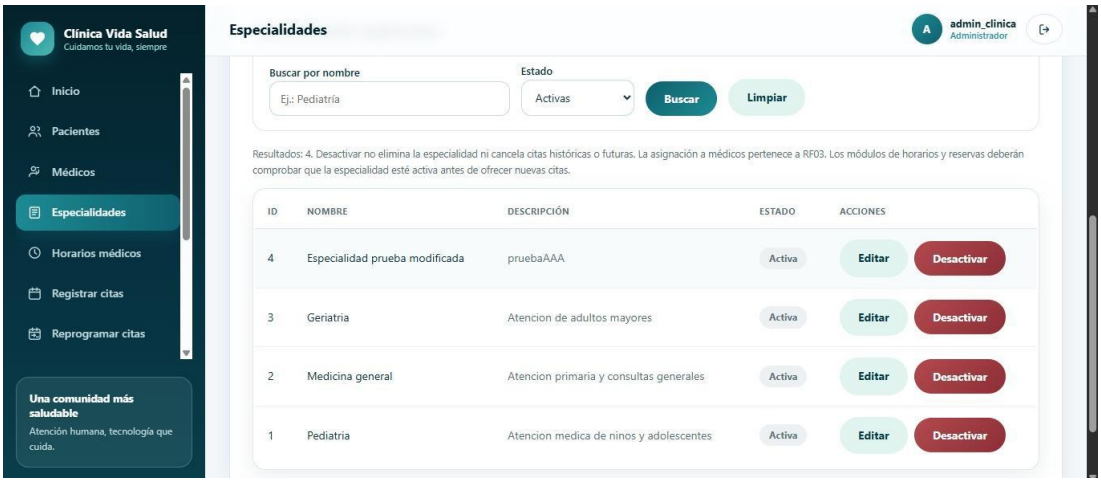
Registro de pacientes



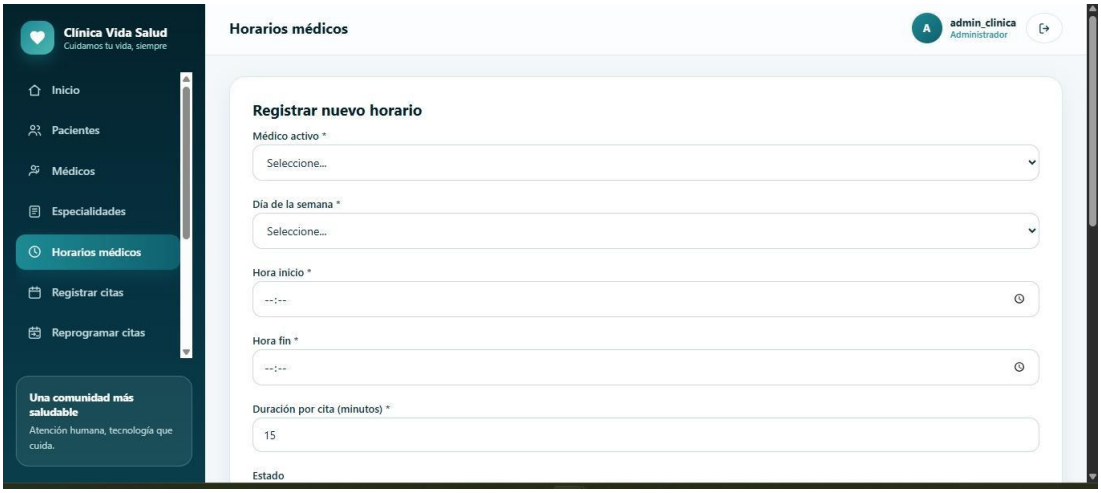
Gestión de médicos



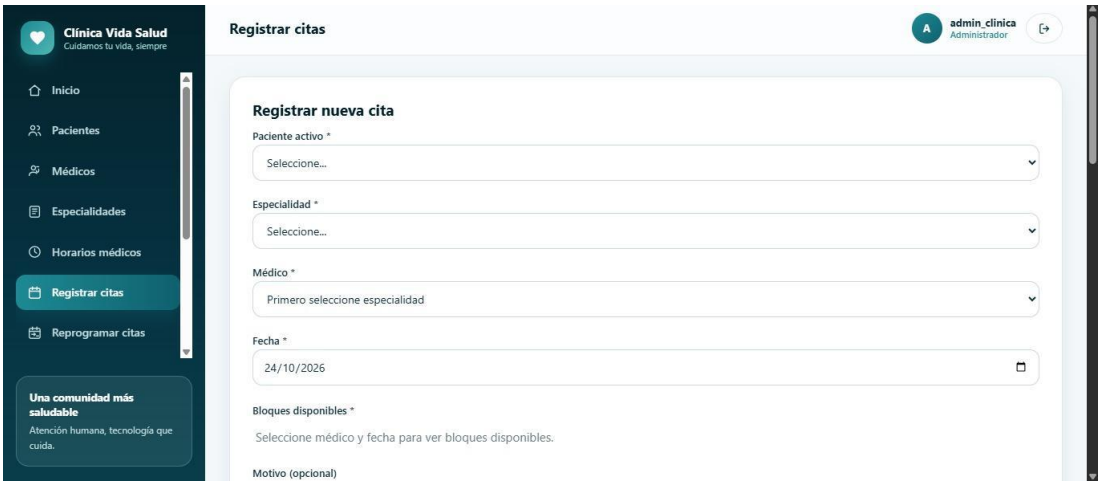
Catálogo de especialidades



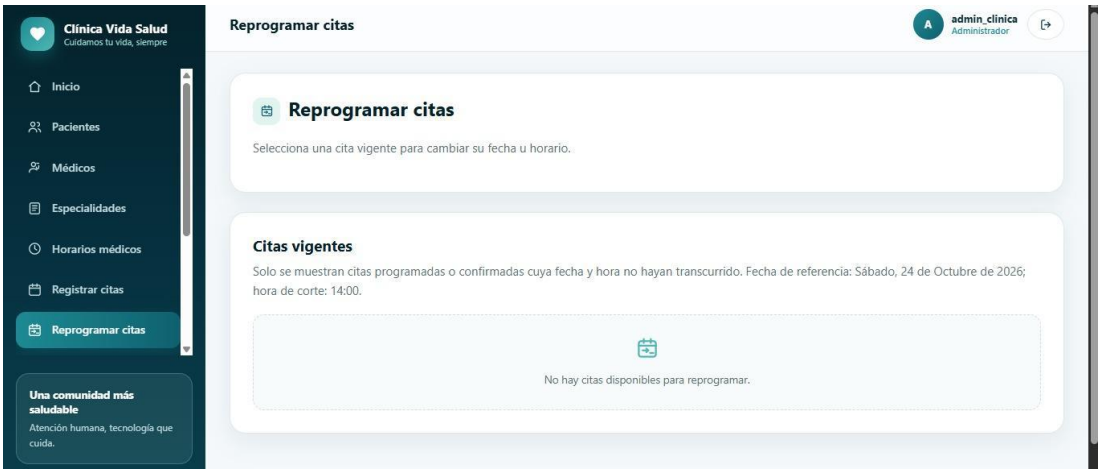
Configuración de horarios



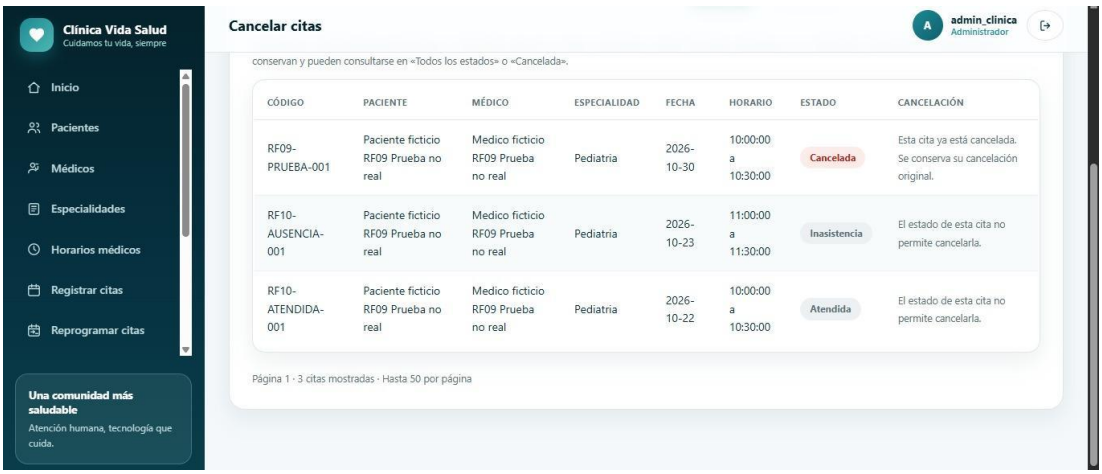
Formulario de registro de citas



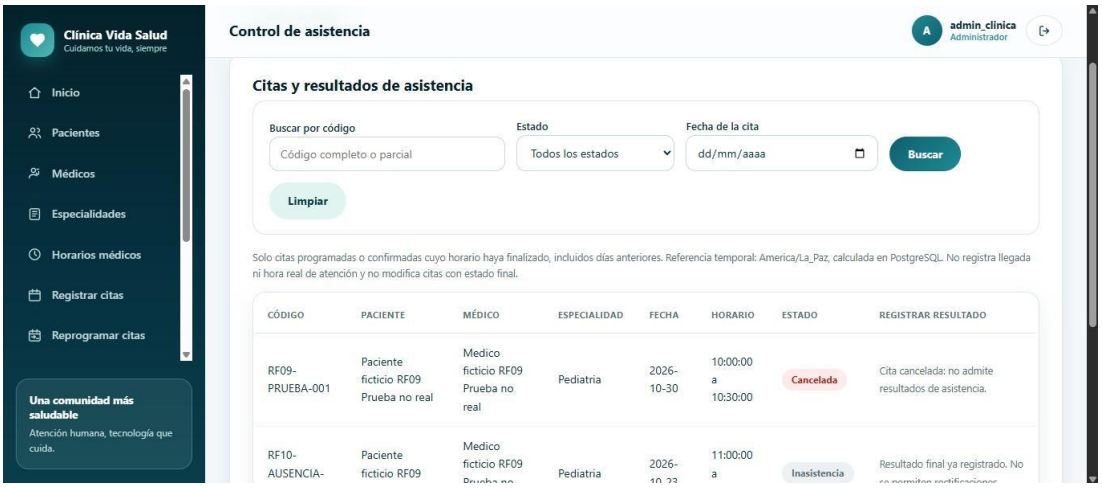
Selección de cita para reprogramar



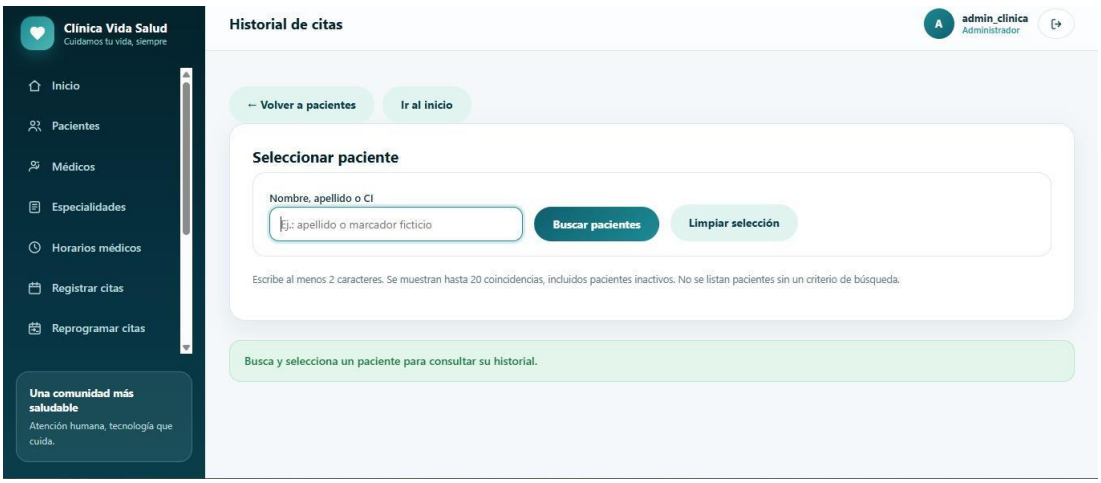
Cancelación de citas



Control de asistencia



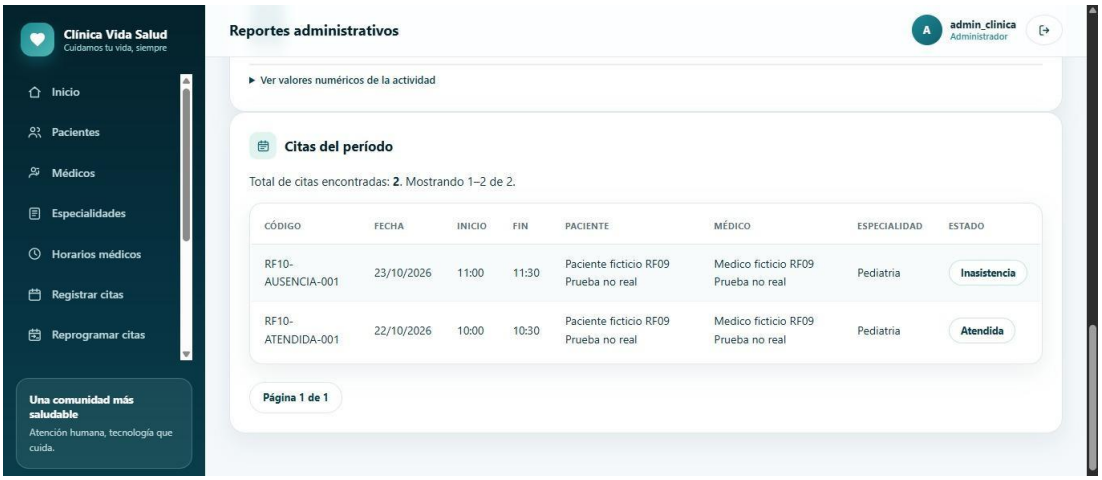
Historial por paciente



Consulta de agenda médica



Reportes administrativos



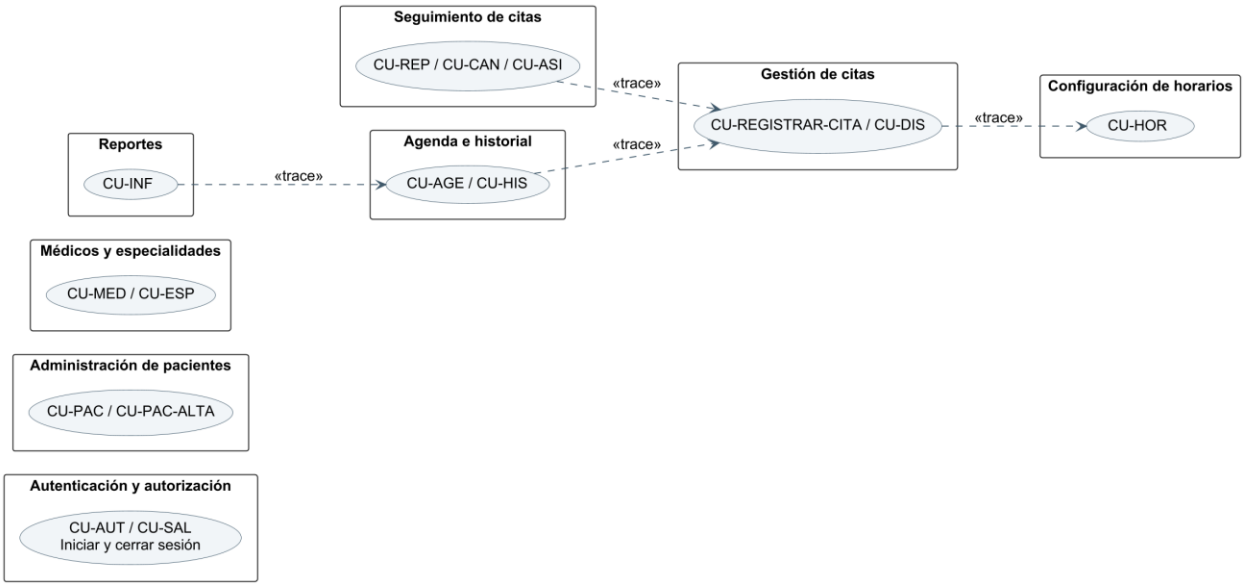
1.5. Estructurar el modelo de casos de uso

La estructuración organiza los objetivos en subsistemas que comparten datos y reglas. Autenticación y autorización es transversal; pacientes proporciona identificación; médicos y especialidades mantiene oferta profesional; horarios configura la capacidad temporal; gestión de citas reserva; seguimiento modifica programación o resultado; agenda e historial recupera información; y reportes la agrega. La división no establece ocho aplicaciones distintas, sino una organización del modelo de requisitos.

El comportamiento de disponibilidad se reutiliza conceptualmente en registro y reprogramación, aunque el código repite parte del algoritmo en archivos diferentes. La autorización atraviesa todos los paquetes protegidos. Agenda, historial y reportes dependen de datos de citas, pero registrar una reserva no necesita generar un reporte. Esta asimetría orienta una estructuración con dependencias comprensibles.

Figura 5

D20 — Organización del modelo de casos de uso por paquetes



Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D20.puml.

Esta vista estructura el catálogo de casos de uso dentro de paquetes UML. Sus dependencias trace documentan afinidad y apoyo entre objetivos, no relaciones include. Las asociaciones con actores se mantienen en D01–D03; su omisión aquí evita repetir una red de líneas y no elimina el actor de ningún caso del catálogo.

D20 es una vista de paquetes que contienen casos de uso. D01–D03 son las vistas que muestran asociaciones con actores y comportamiento de inclusión. No se confunden sus propósitos: el primer grupo responde quién obtiene qué resultado, mientras la organización por paquetes responde dónde se agrupan esos objetivos. Esa distinción se conserva en el análisis y el diseño, cuyas dependencias expresan responsabilidades y artefactos diferentes.

Capítulo 2. Flujo de trabajo: análisis

2.1. Análisis de arquitectura

El análisis arquitectónico busca identificar responsabilidades estables y riesgos que trascienden una página. Se reconocen presentación, control y reglas de negocio, y persistencia. Son capas lógicas: en la implementación procedimental una misma página reúne recepción de parámetros, validación, SQL y generación de HTML. No se afirma una separación física completa ni la existencia de controladores y repositorios orientados a objetos.

La presentación recoge decisiones del actor y comunica resultados; el control determina autorización, validez de entradas y admisibilidad de la operación; la persistencia conserva entidades y aplica garantías del motor. La reserva cruza esas tres responsabilidades. La interfaz muestra un bloque disponible, el procedimiento PHP verifica que encaje en la oferta y PostgreSQL impide conservar dos reservas vigentes incompatibles del mismo médico. La consistencia es una propiedad conjunta, no un efecto de ocultar botones.

Las responsabilidades comunes incluyen sesión, autorización, escape HTML, obtención de PDO y consultas de lectura reutilizables. La autenticación revalida usuario y rol; el acceso a datos usa parámetros preparados; los cambios de cita emplean transacciones. Los errores deben distinguir rechazo por regla del dominio, conflicto concurrente y fallo técnico. En RF09 y RF10 los registros de error son acotados; otros módulos conservan llamadas a error_log con el mensaje completo de la excepción, por lo que no se declara una política de registro homogénea en todo el sistema.

Tabla 21

Aspectos arquitectónicos críticos y asignación de responsabilidad

Aspecto	Responsabilidad analítica	Mecanismo observado	Verificación pendiente
Seguridad	ControlAcceso determina quién actúa.	Sesión revalidada, roles y CSRF.	Barrido por URL y por rol.
Validación	Controles de cada objetivo verifican entradas.	Fechas, IDs, longitudes, actividad y bloques.	Entradas malformadas y fronteras.
Persistencia	Entidades conservan identidad y asociaciones.	PK, FK, UNIQUE y CHECK.	Escenarios de rechazo funcional.
Concurrencia	ControlReservas preserva exclusividad.	EXCLUDE GiST y bloqueo de cita al modificar.	Dos sesiones simultáneas y espera por bloqueo.
Tiempo	ControlSeguimiento respeta inicio y fin.	America/La_Paz y clock_timestamp en RF09/RF10.	Condiciones exactas de inicio/fin y coherencia de módulos.
Errores	Resultado comprensible sin éxito falso.	Rollback y mensajes según SQLSTATE.	Casos inducidos en ambiente de pruebas.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

El navegador envía una solicitud HTTP a Apache. PHP carga las funciones comunes, determina la identidad cuando corresponde y ejecuta las consultas necesarias mediante PDO_PGSQL. PostgreSQL devuelve filas o una excepción; PHP transforma esos resultados en HTML o JSON. La navegación no requiere un servidor de API independiente ni un framework no identificado. Esta descripción anticipa el despliegue local y las secuencias de diseño sin mezclar la topología de ejecución con las clases conceptuales.

2.2. Analizar casos de uso

Analizar un caso de uso consiste en asignar sus pasos a responsabilidades que expliquen por qué el sistema produce un resultado. Antes de escoger llamadas concretas, se distinguen objetos frontera, coordinadores y entidades. Las entradas y validaciones siguientes se recuperan del comportamiento implementado, pero se expresan a nivel de responsabilidad para mantener una diferencia respecto de las secuencias de diseño.

Tabla 22

Análisis del caso Iniciar sesión

Dimensión	Aplicación al sistema
Participantes	PantallaAcceso; ControlAcceso; usuarios; roles
Entradas	Nombre, contraseña y token.
Validaciones	Formato, límite de fallos, coincidencia de hash, actividad y rol.
Interacciones y entidades afectadas	El control obtiene identidad válida y establece el contexto de sesión. No se modifica una tabla de intentos porque el límite se conserva en sesión.
Resultado esperado	Resultado: identidad autenticada o aviso de rechazo; base para autorizar los restantes casos.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 23

Análisis del caso Registrar paciente

Dimensión	Aplicación al sistema
Participantes	FormularioPaciente; ControlPacientes; pacientes
Entradas	Datos identificativos, contacto y opcionales.
Validaciones	Obligatorios, longitudes, nacimiento no futuro, correo y opciones de catálogo.
Interacciones y entidades afectadas	El control normaliza vacíos y solicita persistencia; la entidad impide duplicar un CI no nulo mediante la restricción.
Resultado esperado	Resultado: ficha nueva activa o explicación de rechazo; no crea una cuenta de usuario.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 24

Análisis del caso Registrar cita médica

Dimensión	Aplicación al sistema
Participantes	FormularioCita; ControlReservas; pacientes; medicos; medico_especialidad; horarios; citas
Entradas	Paciente, oferta, fecha, hora y motivo opcional.
Validaciones	Identidad autorizada, actividad, asignación, temporalidad y bloque completo alineado.
Interacciones y entidades afectadas	El control coordina consulta de disponibilidad y reserva consistente; la persistencia decide exclusión concurrente.
Resultado esperado	Resultado: código de cita programada confirmada transaccionalmente o rechazo sin reserva conservada.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 25

Análisis del caso Reprogramar cita

Dimensión	Aplicación al sistema
Participantes	SeleccionCita; FormularioCita; ControlReservas; citas; horarios; medicos
Entradas	ID existente, nueva fecha y bloque.
Validaciones	Estado vigente, inicio original futuro, médico activo y nuevo bloque admisible.
Interacciones y entidades afectadas	El control conserva identidad y modifica programación; no cambia paciente, médico o especialidad.
Resultado esperado	Resultado: mismo registro con nuevas horas o conservación de la programación ante rechazo.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 26

Análisis del caso Cancelar cita

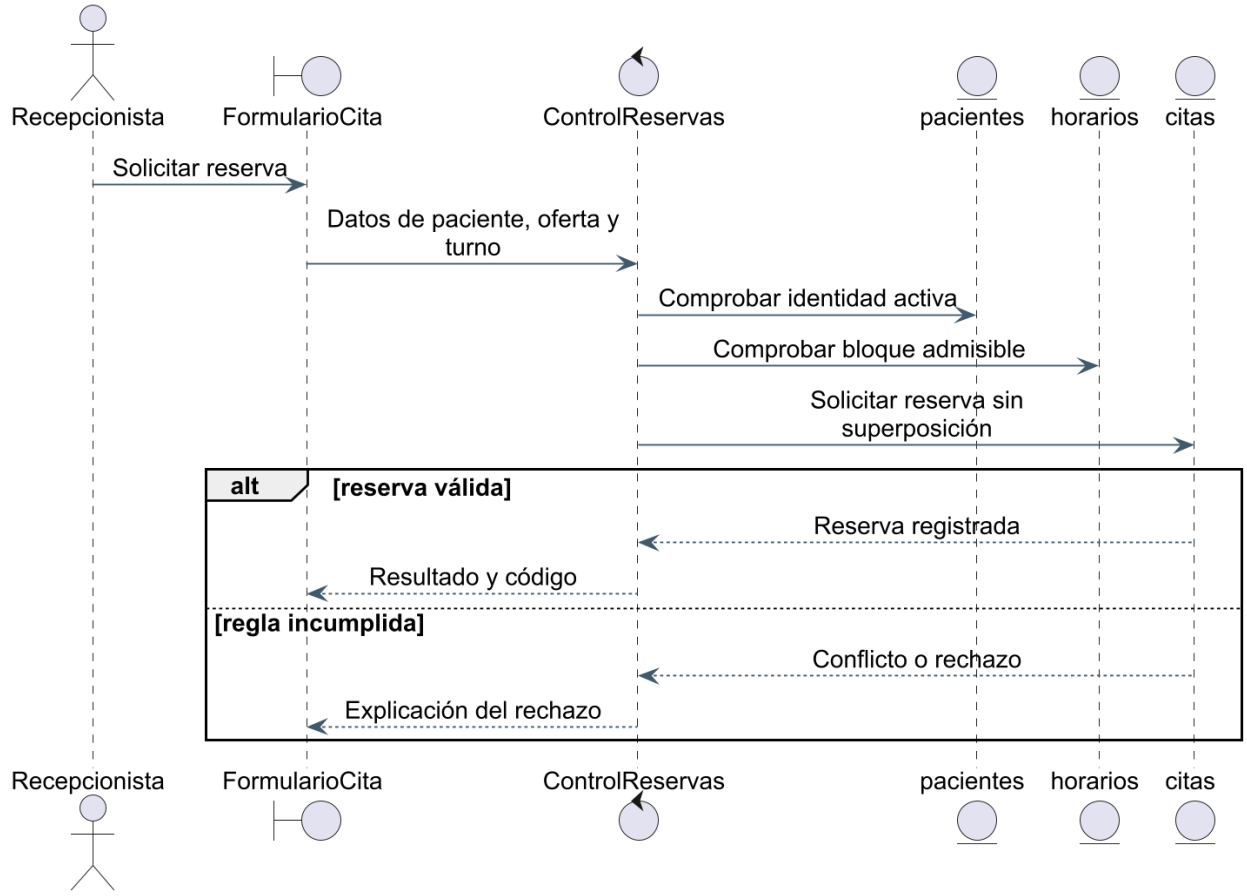
Dimensión	Aplicación al sistema
Participantes	ConfirmacionSeguimiento; ControlSeguimiento; citas; usuarios; roles
Entradas	ID, confirmación, motivo opcional y token.
Validaciones	Permiso revalidado, cita vigente, inicio no alcanzado y motivo válido.
Interacciones y entidades afectadas	El control bloquea y verifica condición temporal antes del cambio; la entidad preserva el registro cancelado.
Resultado esperado	Resultado: cancelada con historial conservado o advertencia que no altera el estado.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Realización de análisis de Registrar cita

Figura 6

D21 — Realización de análisis de Registrar cita



Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D21.puml.

Esta realización asigna responsabilidades sin comprometer nombres de métodos implementados. La entidad citas reúne la necesidad de conservar la reserva consistente; el mecanismo concreto se decide en D10B con PDO y la exclusión PostgreSQL. FormularioCita y ControlReservas no existen como clases PHP. La diferencia entre ambos niveles evita convertir el análisis en una copia del código.

La realización vincula el objetivo con una cadena de responsabilidades: la frontera comunica la solicitud, el control verifica condiciones y las entidades aportan identidad, oferta y consistencia. El control conceptual no representa una nueva clase propuesta para cambiar la

aplicación; permite explicar por qué los bloques procedimentales de citas.php colaboran con datos de varias tablas. El diseño D10A/D10B sustituye esas solicitudes abstractas por GET, POST, consultas, transacción y tratamiento de SQLSTATE.

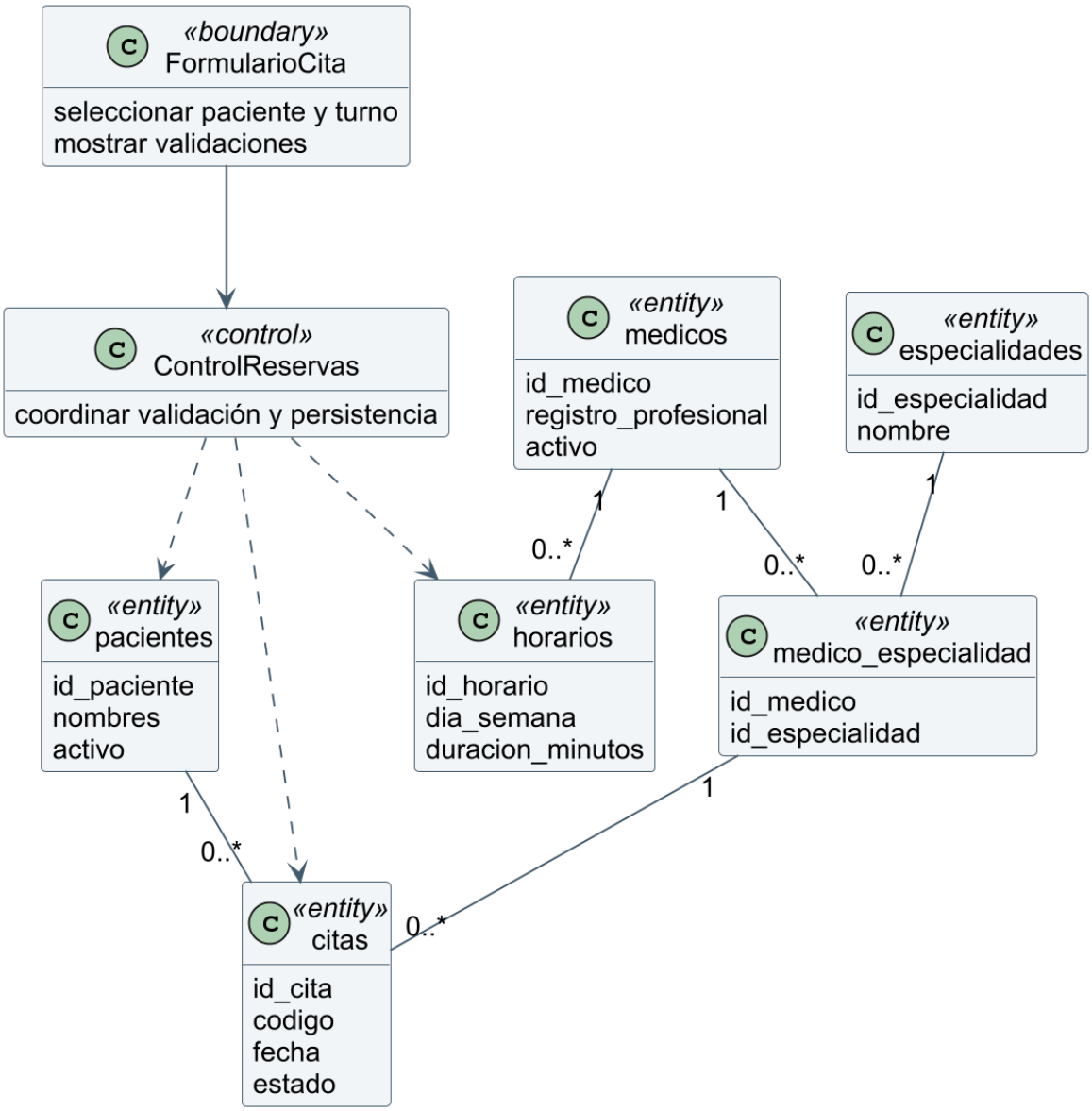
2.3. Análisis de clases

Las clases de análisis se identifican por sustantivos del dominio y responsabilidades de los casos. Paciente, médico, especialidad, horario, cita, usuario y rol son conceptos estables; sus nombres físicos se conservan como pacientes, medicos, especialidades, horarios, citas, usuarios y roles. medico_especialidad expresa la asignación que hace válida la oferta. Las fronteras y controles no se deducen de una tabla: representan interacción y coordinación y se identifican explícitamente como abstracciones.

Una clase conceptual y una tabla no son equivalentes. La primera ayuda a asignar responsabilidades y asociaciones; la segunda define persistencia, tipos y restricciones. En el diagrama analítico se seleccionan atributos relevantes y se omiten detalles que no aportan a comprender la reserva. El capítulo de datos documenta el esquema completo. Las multiplicidades del análisis sí deben respetar lo posible en la persistencia para no introducir supuestos incompatibles.

Figura 7

D05 — Clases de análisis: interacción, coordinación y dominio

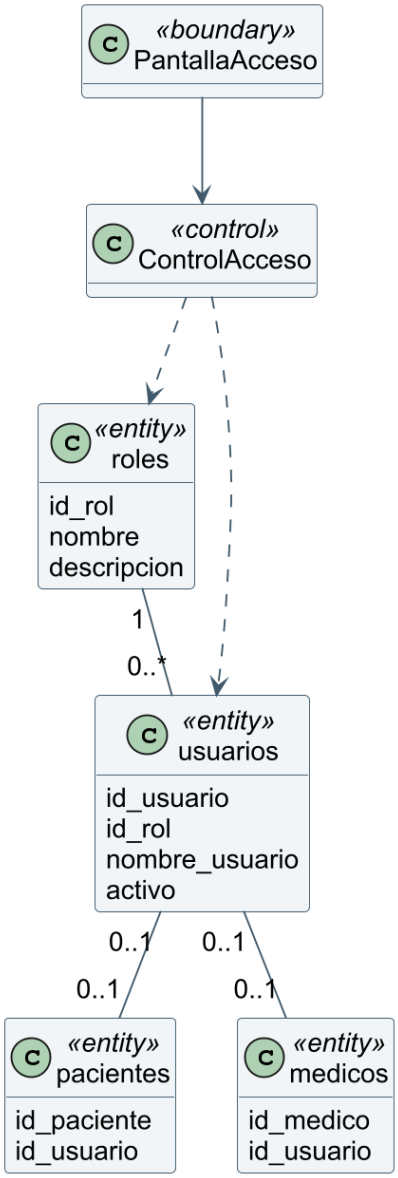


Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D05.puml.

FormularioCita y ControlReservas son abstracciones de análisis y no clases PHP físicas. Las entidades usan los identificadores del SQL para permitir su trazabilidad. La asociación entre horarios y citas se coordina mediante médico, fecha y reglas de bloque; no existe id_horario en citas, por lo que no se dibuja una clave foránea ficticia. El detalle de seguridad se separa en D05B para preservar legibilidad.

Figura 8

D05B — Clases de análisis del contexto de seguridad



Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D05B.puml.

Usuario y Rol se representan como usuarios y roles, respetando el esquema. Las asociaciones con pacientes y medicos son opcionales en ambos extremos por la combinación de FK anulable y UNIQUE. El esquema no impide que una misma cuenta se vincule a ambas tablas; la elección de un rol único no equivale a una restricción relacional entre esos perfiles. No se expone ningún valor de contrasena_hash.

Tabla 27

Responsabilidades de las clases conceptuales

Elemento de análisis	Estereotipo	Responsabilidad y correspondencia
PantallaAcceso / FormularioCita / FormularioPaciente / ConfirmacionSeguimiento	boundary	Representar interacción; corresponden a interfaces de login, citas, pacientes y seguimiento, sin clases PHP propias.
ControlAcceso	control	Coordinar identidad y permisos; trazable a funciones de auth.php y sesion.php.
ControlPacientes / ControlOferta / ControlHorarios	control	Validar y coordinar mantenimiento administrativo; distribuidos en páginas procedimentales.
ControlReservas	control	Coordinar disponibilidad, temporalidad y persistencia de reserva o cambio.
ControlSeguimiento / ControlConsultas / ControlReportes	control	Coordinar transiciones finales, consultas y agregados; no namespaces ni clases implementadas.
usuarios / roles	entity	Conservar identidad técnica y clasificación de permisos.
pacientes / medicos	entity	Conservar identidad administrativa y actividad; vínculo opcional con usuarios.
especialidades / medico_especialidad	entity	Definir catálogo y asignaciones válidas del profesional.
horarios	entity	Definir oferta semanal y duración; no es una cita fechada.
citas	entity	Conservar reserva identificada, programación, motivo y estado actual.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Un paciente puede no tener citas o tener varias; cada cita tiene un paciente obligatorio. Un médico puede tener varios horarios y varias asignaciones; cada horario pertenece a un médico. Cada asignación puede ser utilizada por cero o muchas citas. La posibilidad de un médico sin asignaciones en el esquema no contradice que el formulario de alta exija al menos

una especialidad: la cardinalidad estructural y la regla de una operación actúan en niveles distintos.

La asociación entre usuarios y pacientes o medicos es opcional uno a uno en cada tabla de perfil. Las FK son anulables y únicas; no todo paciente o médico debe tener una cuenta. El esquema no contiene una entidad Recepcionista separada: ese rol es un valor de roles relacionado con usuarios. Tampoco existe una tabla de auditoría de reprogramaciones ni una relación directa cita-horario. Estas observaciones impiden que el modelo de análisis sugiera persistencia que la aplicación no conserva.

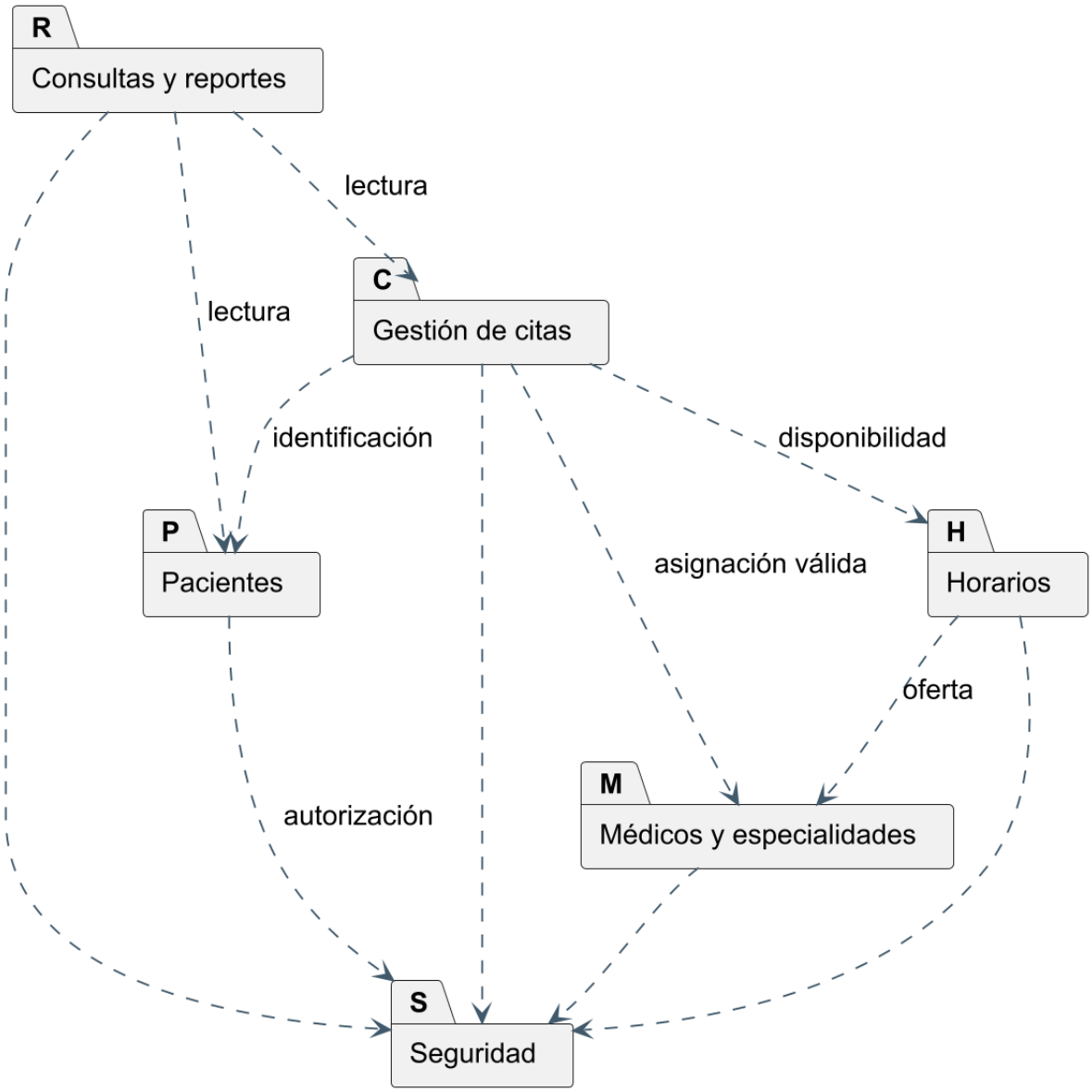
2.4. Analizar paquetes

El análisis de paquetes agrupa responsabilidades que cambian por razones similares. Seguridad concentra identidad; pacientes mantiene fichas; médicos y especialidades administra oferta profesional; horarios define capacidad temporal; gestión de citas coordina la reserva y su seguimiento; y consultas y reportes recupera información. La agrupación ayuda a discutir cohesión y dependencias antes de revisar archivos concretos.

La separación se justifica por la dirección de uso: una cita necesita paciente, oferta y horario; un reporte necesita leer citas, pero la reserva no depende de obtener estadísticas. Seguridad es común a los paquetes protegidos. Las dependencias no se interpretan como despliegue independiente ni como una obligación de reorganizar ahora el código. Permiten localizar el impacto de un cambio, por ejemplo una nueva política de cancelación dentro de gestión de citas, sin convertirla en una modificación del catálogo de especialidades.

Figura 9

D06 — Paquetes del modelo de análisis



Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D06.puml.

Las dependencias expresan conocimiento conceptual, no rutas de include PHP. Seguridad concentra identidad y permisos; pacientes y oferta médica proporcionan información a la gestión de citas; consultas y reportes consumen resultados. La dirección evita que la reserva dependa de la presentación estadística y mantiene una separación cohesiva del análisis.

Tabla 28

Cohesión y dependencias de los paquetes de análisis

Paquete	**Elementos principales**	**Dependencias y CU**
Seguridad	PantallaAcceso, ControlAcceso, usuarios y roles.	CU-AUT/CU-SAL; soporte transversal.
Pacientes	FormularioPaciente, ControlPacientes y pacientes.	Seguridad; CU-PAC y CU-PAC-ALTA.
Médicos y especialidades	ControlOferta, medicos, especialidades y asignación.	Seguridad; CU-MED/CU-ESP.
Horarios	ControlHorarios y horarios.	Oferta y seguridad; CU-HOR/CU-DIS.
Gestión de citas	FormularioCita, ControlReservas, ControlSeguimiento y citas.	Pacientes, oferta, horarios y seguridad; registro, cambios y resultados.
Consultas y reportes	ControlConsultas y ControlReportes.	Lectura de citas y pacientes; seguridad; CU-AGE/CU-HIS/CU-INF.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Capítulo 3. Flujo de trabajo: diseño

3.1. Diseño de arquitectura

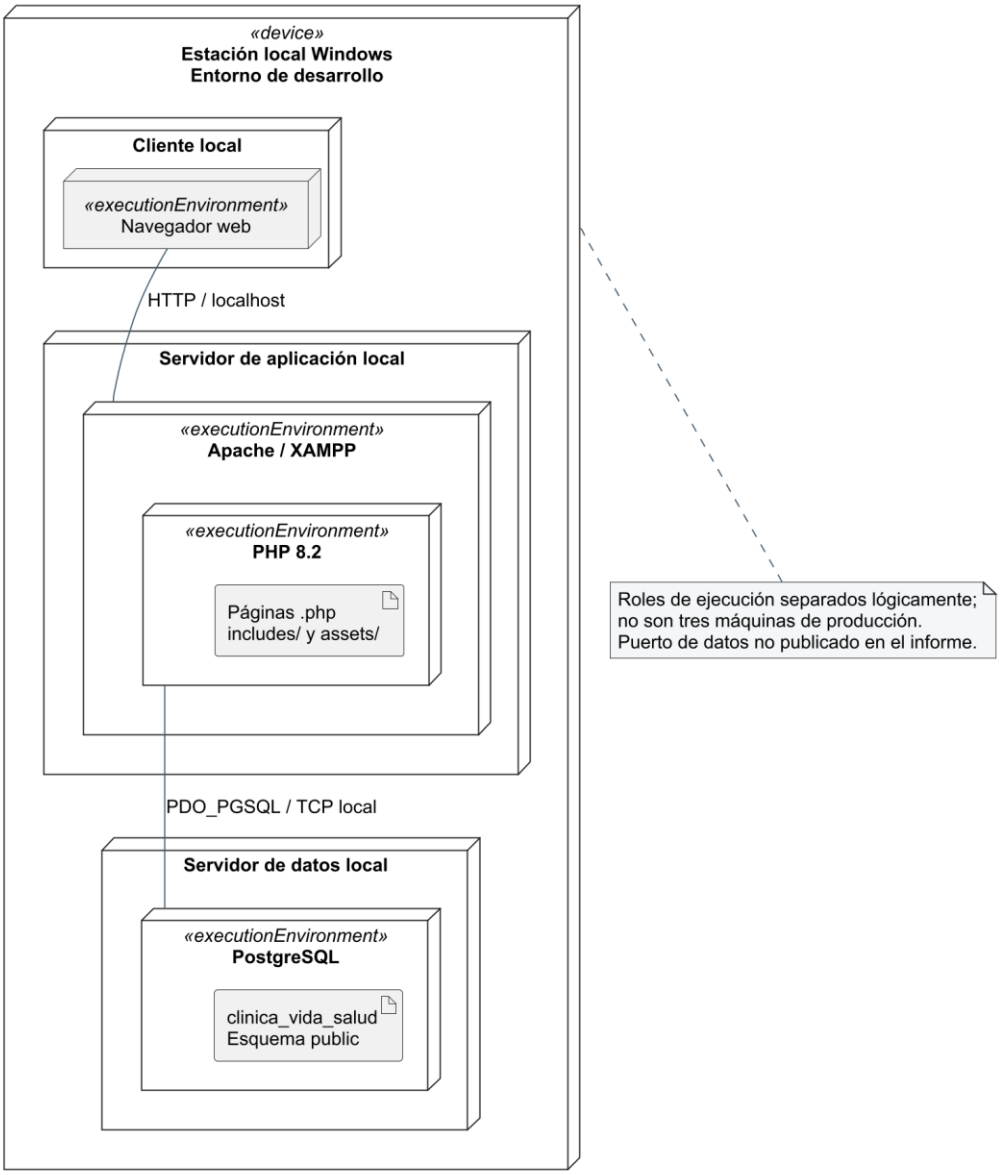
El diseño transforma las responsabilidades del análisis en una organización ejecutable y verificable. En esta entrega se documenta la arquitectura implementada y se explican sus límites: PHP procedimental servido por Apache, funciones compartidas para acceso y presentación, y PostgreSQL como motor de persistencia. Los paquetes de diseño son lógicos y no afirman una modularidad física más estricta que la observada.

La arquitectura favorece un entorno académico reproducible, con puntos de entrada directos y consultas preparadas. A la vez, la distribución de reglas entre páginas exige cuidado para evitar divergencias. La generación de disponibilidad aparece en citas.php, reprogramar_cita.php e includes/dashboard.php. Su correspondencia actual se explica, pero una política futura de duración o temporalidad requerirá revisar los tres lugares y sus pruebas.

3.1.1. Diseño físico de la arquitectura: diagrama de despliegue

Figura 10

D07 — Despliegue físico del entorno local



Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D07.puml.

El diagrama utiliza nodos, entornos de ejecución y artefactos desplegados. Los roles cliente, aplicación y datos están anidados en la estación local: no se afirma que existan servidores públicos. Un teléfono es un cliente posible de una futura instalación accesible por red, pero

localhost desde un teléfono no apunta a esta computadora. La comunicación navegador-servidor es HTTP local; no se acredita HTTPS productivo.

La instalación local contiene un navegador cliente, Apache con PHP y un servicio PostgreSQL. PDO_PGSQL transporta las consultas desde PHP; el navegador no se conecta directamente a la base. Los archivos del sistema se sirven desde el proyecto dentro de htdocs. La configuración privada se carga desde una ubicación externa al repositorio; no se incluyen su contenido, usuario, contraseña ni detalles de acceso.

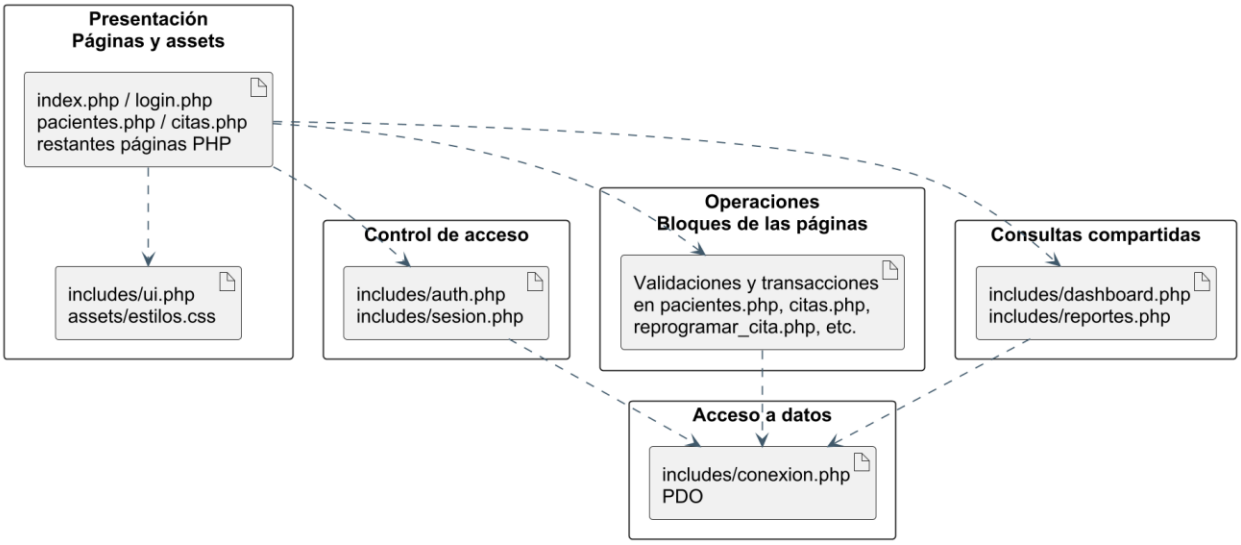
El modelo muestra estos roles anidados en la estación local, con HTTP sobre localhost y comunicación local de datos. Apache pertenece a XAMPP; PostgreSQL se representa como servicio de datos independiente y no como si XAMPP lo incluyera necesariamente. La versión del motor observada es 18.6. La identidad exacta de equipos, capacidad, proveedor o costos de servidores públicos no se inventa.

Un paso posterior a un servidor público requerirá trasladar artefactos y configuración, instalar PHP y el controlador PostgreSQL, preparar el esquema con sus restricciones y verificar conectividad y permisos. Cambiará la ubicación del nodo de aplicación y su exposición de red; el acceso público deberá verificarse mediante URL y configuración efectiva. La migración no se presenta como realizada ni se dibuja una infraestructura contratada sin evidencia. PF18 conserva la comprobación del despliegue como pendiente.

3.1.2. Diseño lógico de la arquitectura: diagrama de paquetes

Figura 11

D08 — Paquetes lógicos del diseño y correspondencia procedimental



Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D08.puml.

Los paquetes son agrupaciones lógicas. No se presentan como namespaces ni como un MVC implementado. Una página puede contener presentación, control y SQL, por lo que su participación en varias responsabilidades es intencional. Las consultas del dashboard y reportes sí se concentran en funciones compartidas; conexion.php entrega la instancia PDO mediante carga diferida.

Tabla 29

Correspondencia entre paquetes de diseño y archivos reales

Responsabilidad lógica	**Archivos reales**	**Criterio de separación**
Interfaces y páginas	index.php, login.php y páginas funcionales; assets/estilos.css; includes/ui.php.	Entrada/salida HTTP, HTML, navegación y estilo.
Autenticación y sesión	includes/auth.php; includes/sesion.php; login.php; logout.php.	Identidad, rol, expiración y protección de formulario.
Reglas y operaciones	Bloques PHP de pacientes.php, medicos.php, horarios.php, citas.php y seguimiento.	Validaciones y transacciones procedimentales; sin capa de servicios OO.

Responsabilidad lógica	Archivos reales	Criterio de separación
Consultas compartidas	includes/dashboard.php; includes/reportes.php.	Funciones que devuelven arreglos de lectura para vistas.
Acceso a datos	includes/conexion.php y uso de PDO en módulos.	Obtención diferida de conexión y ejecución parametrizada.
Persistencia	PostgreSQL y database/clinica_vida_salud.sql.	Tablas, claves, índices, CHECK y EXCLUDE.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

La separación de responsabilidades no es una afirmación de arquitectura MVC estricta. Por ejemplo, citas.php define endpoints JSON, procesa POST y contiene HTML y JavaScript. Representarlo en presentación y operaciones describe esa combinación real. La mejora de mantenibilidad puede consistir después en extraer reglas compartidas, pero no se introduce una refactorización ni se cambia la aplicación para acomodarla al diagrama.

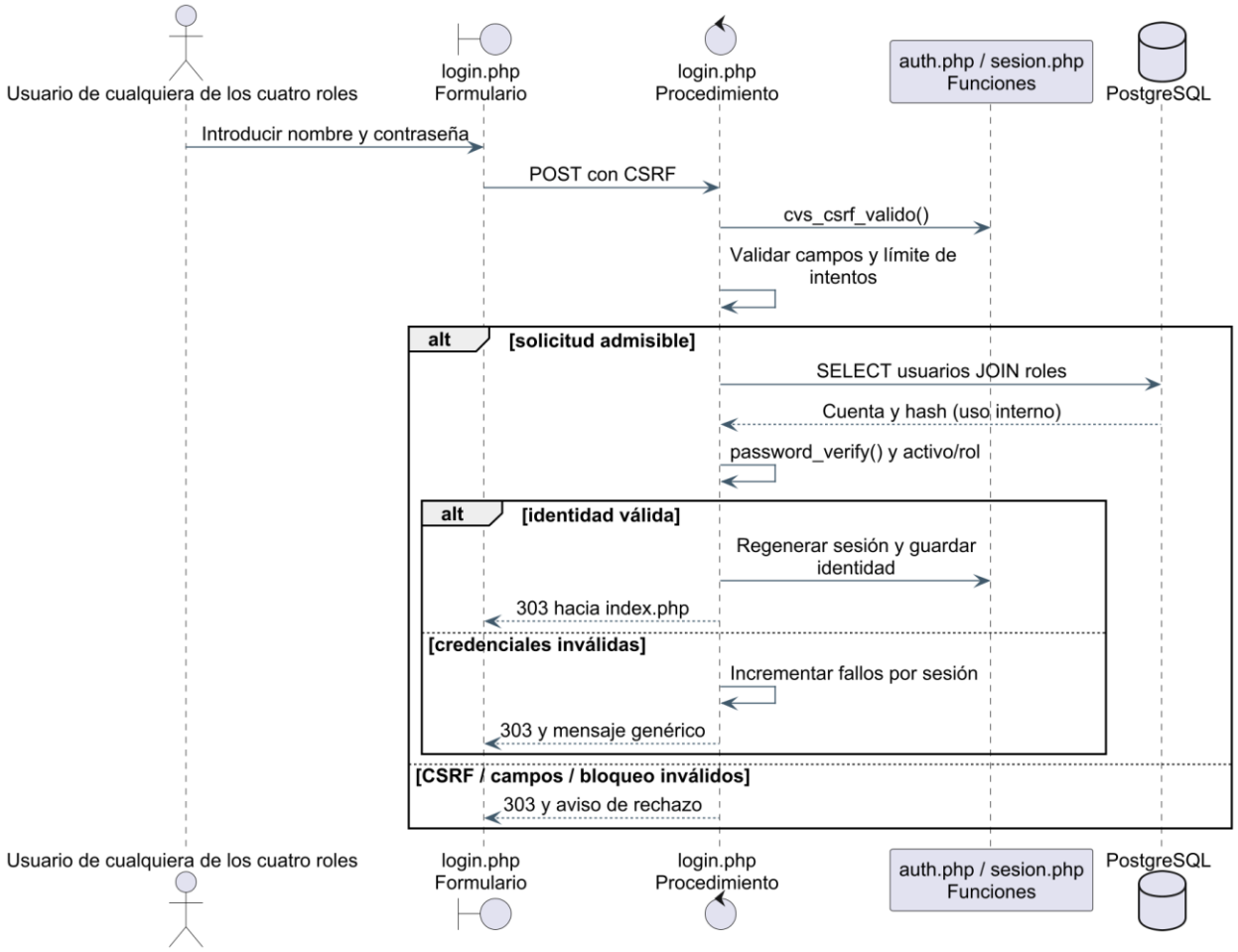
3.2. Diseñar casos de uso: diagramas de secuencia

Las secuencias materializan las realizaciones del análisis. Sus participantes representan actores, interfaces, procedimientos y mecanismos de acceso; una línea de vida con nombre de función o archivo no significa que exista una clase PHP homónima. Los mensajes se interpretan como llamadas, solicitudes o consultas verificadas. Las alternativas están condicionadas por validaciones o respuestas del motor, y sus resultados enlazan con los criterios de aceptación.

El caso principal se divide en selección/lectura y confirmación/escritura para preservar claridad. Esta división es relevante arquitectónicamente: el JSON de disponibilidad puede quedar desactualizado y la protección definitiva ocurre en el INSERT. Las secuencias adicionales incorporan reglas temporales y alcance de permisos que el diagrama general de casos de uso no puede expresar por sí solo.

Figura 12

D09 — Secuencia de inicio de sesión

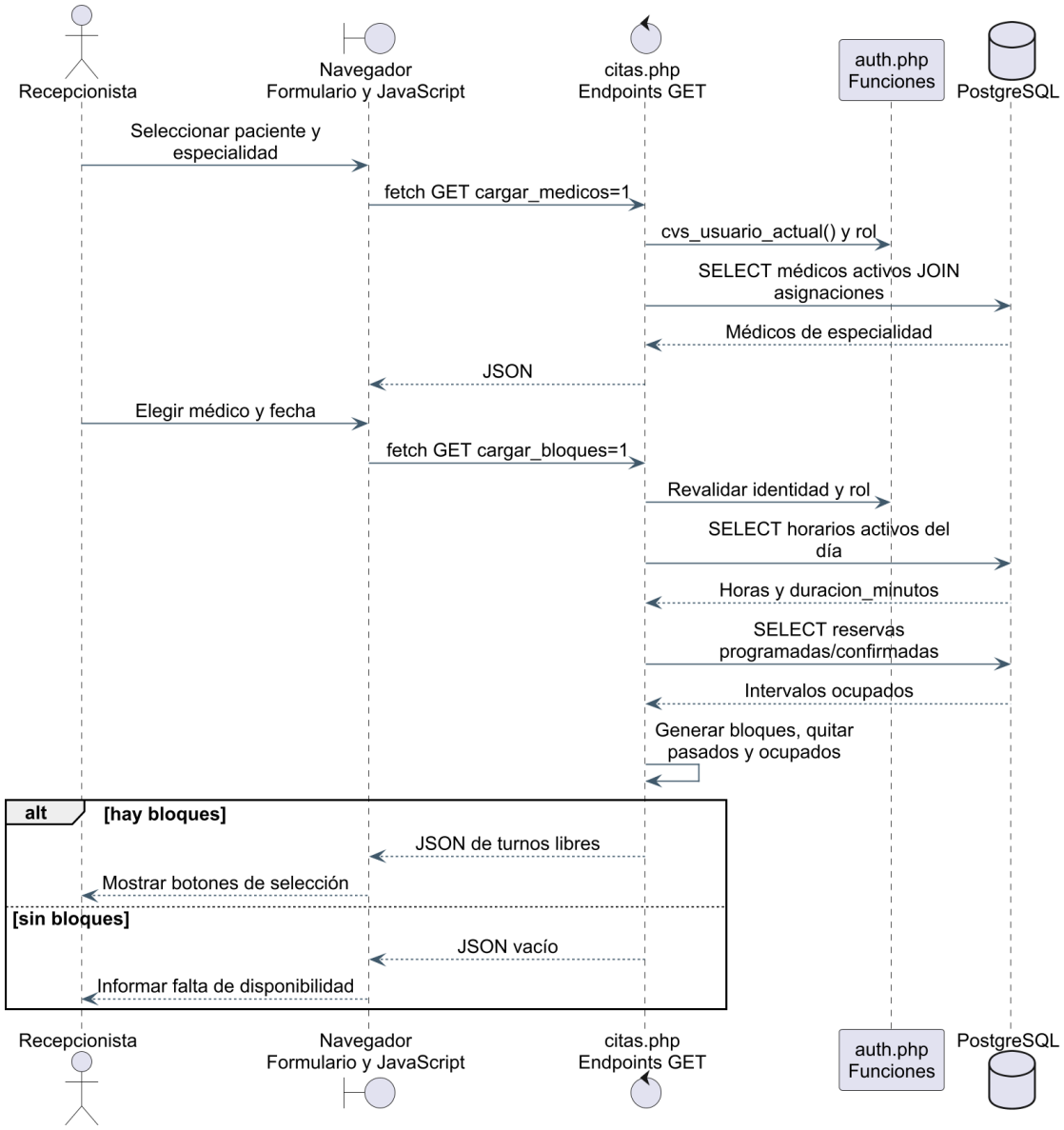


Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D09.puml.

El procedimiento valida el token y el límite por sesión antes de consultar usuarios y roles. password_verify no compara contraseñas en texto plano almacenadas. La identidad se conserva tras regenerar el identificador de sesión; las respuestas usan POST/Redirect/GET. Los fallos se limitan a cinco con bloqueo de quince minutos por sesión, sin constituir un bloqueo centralizado por cuenta. RF01.

Figura 13

D10A — Registrar cita: selección y consulta de disponibilidad

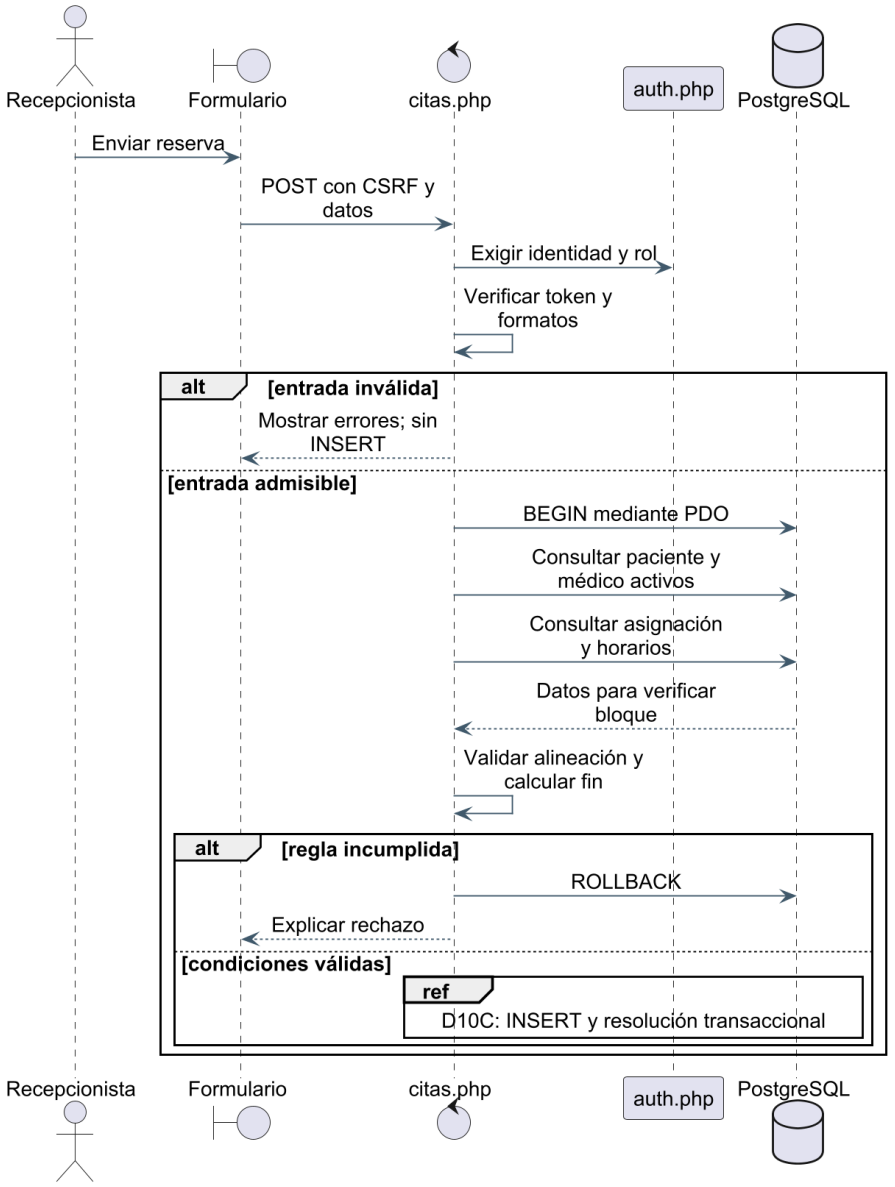


Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D10A.puml.

La inspección confirma llamadas fetch reales: los endpoints de médicos y bloques devuelven JSON. Los pacientes se cargan en la página inicial; no se inventa una búsqueda AJAX de pacientes. El cálculo divide cada horario en bloques completos, excluye horas pasadas de hoy e intervalos de reservas vigentes. Esta lectura informa la selección, pero no garantiza la reserva final. RF05, RF06 y RF07.

Figura 14

D10B — Registrar cita: validación del envío y del bloque

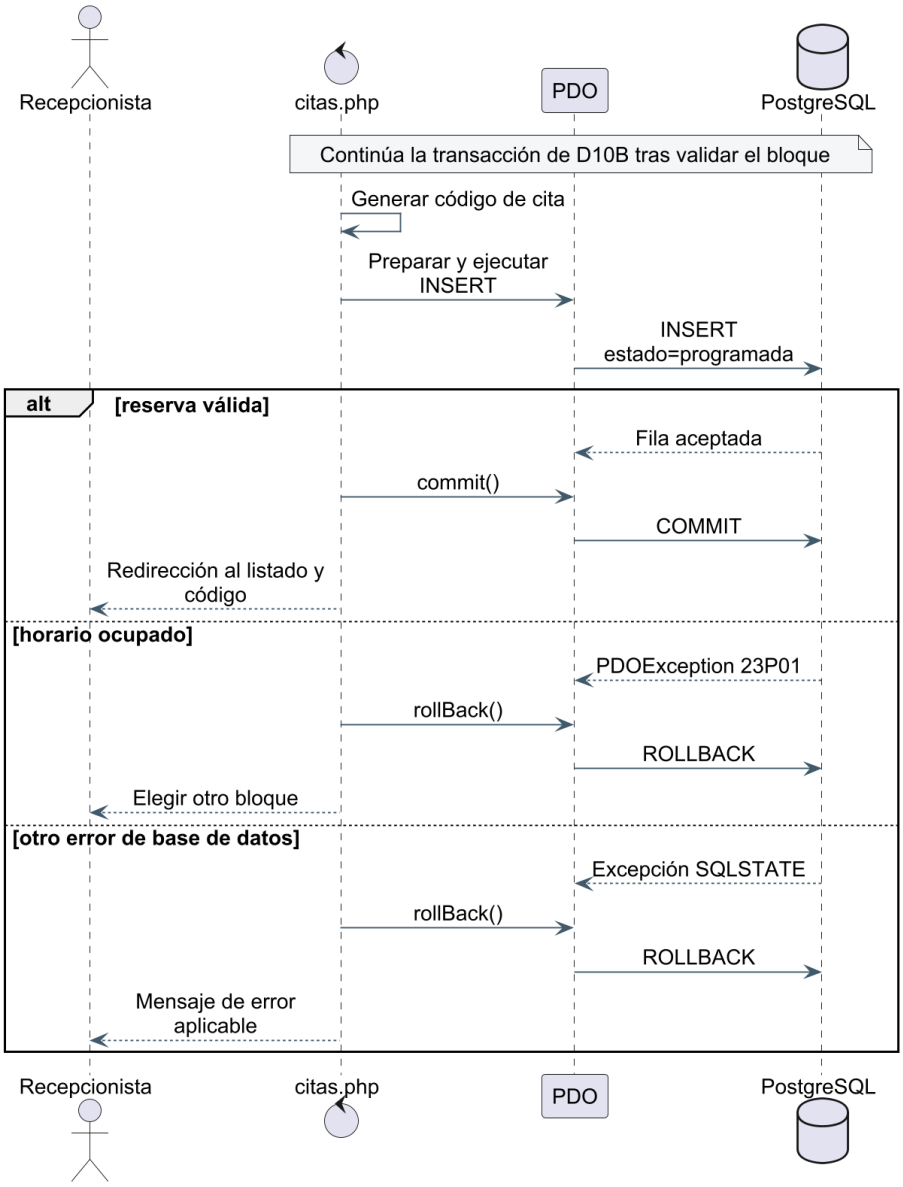


Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D10B.puml.

El POST exige identidad y rol y verifica el token antes de reservar. La transacción consulta paciente y médico activos, asignación y horarios; calcula hora_fin en servidor y comprueba la alineación del bloque. La rama válida continúa en D10C dentro de la misma transacción, no inicia una operación independiente. No consulta expresamente activo de especialidades ni repite una búsqueda de superposición previa al INSERT. RF06 y RF07.

Figura 15

D10C — Registrar cita: resolución de la persistencia y errores



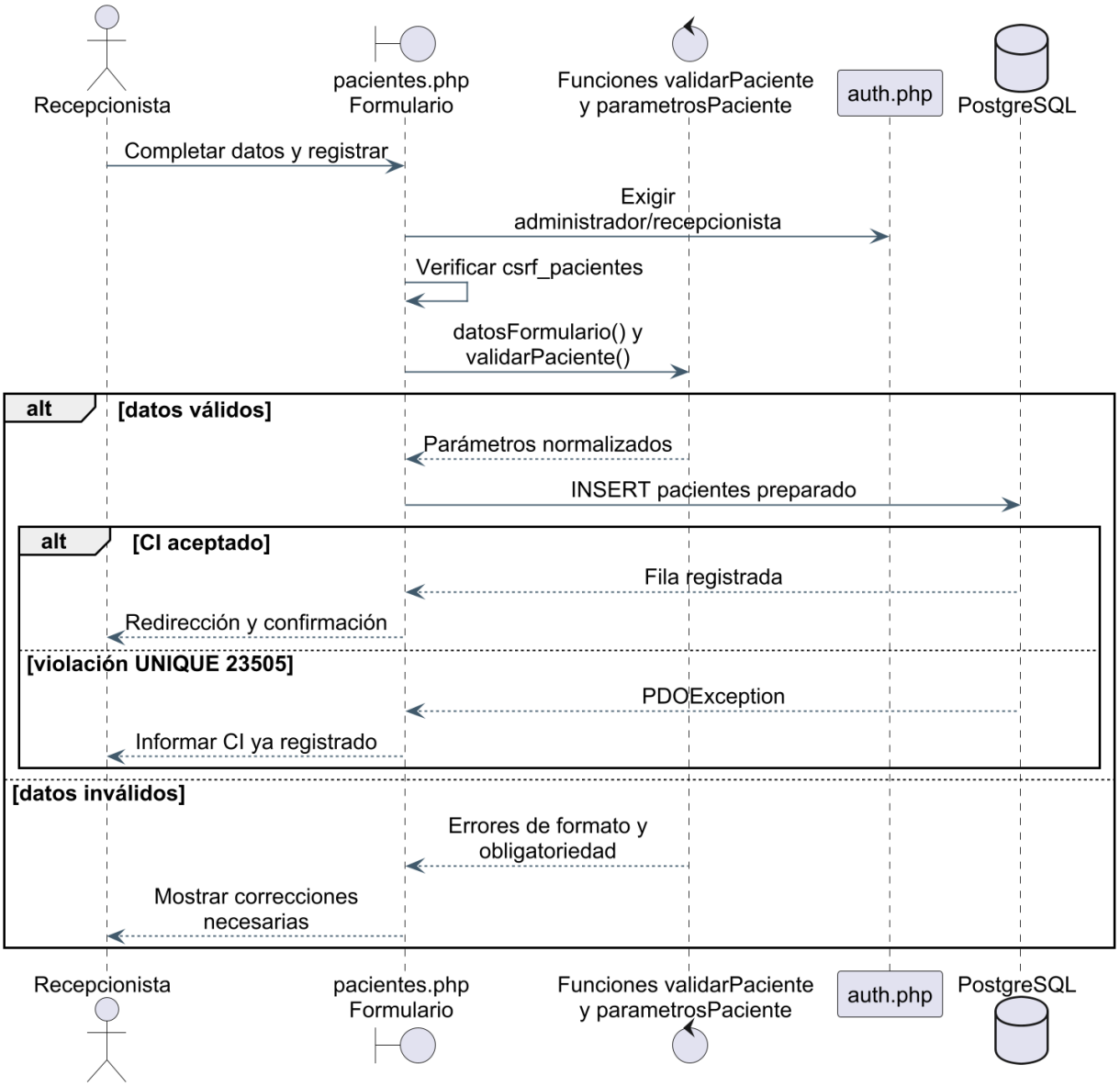
Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D10C.puml.

EXCLUDE constituye la decisión atómica frente a reservas concurrentes. Un 23P01 se comunica como horario reservado; 23505 como conflicto del código y 23503 como restricción referencial. Un error distinto recibe un mensaje genérico. El procedimiento confirma una reserva solo después de commit; las ramas de rechazo revierten. Los mensajes a la recepcionista

representan la respuesta de la misma página y su redirección, no notificaciones externas. RF06 y RF07.

Figura 16

D16 — Secuencia de registro de paciente



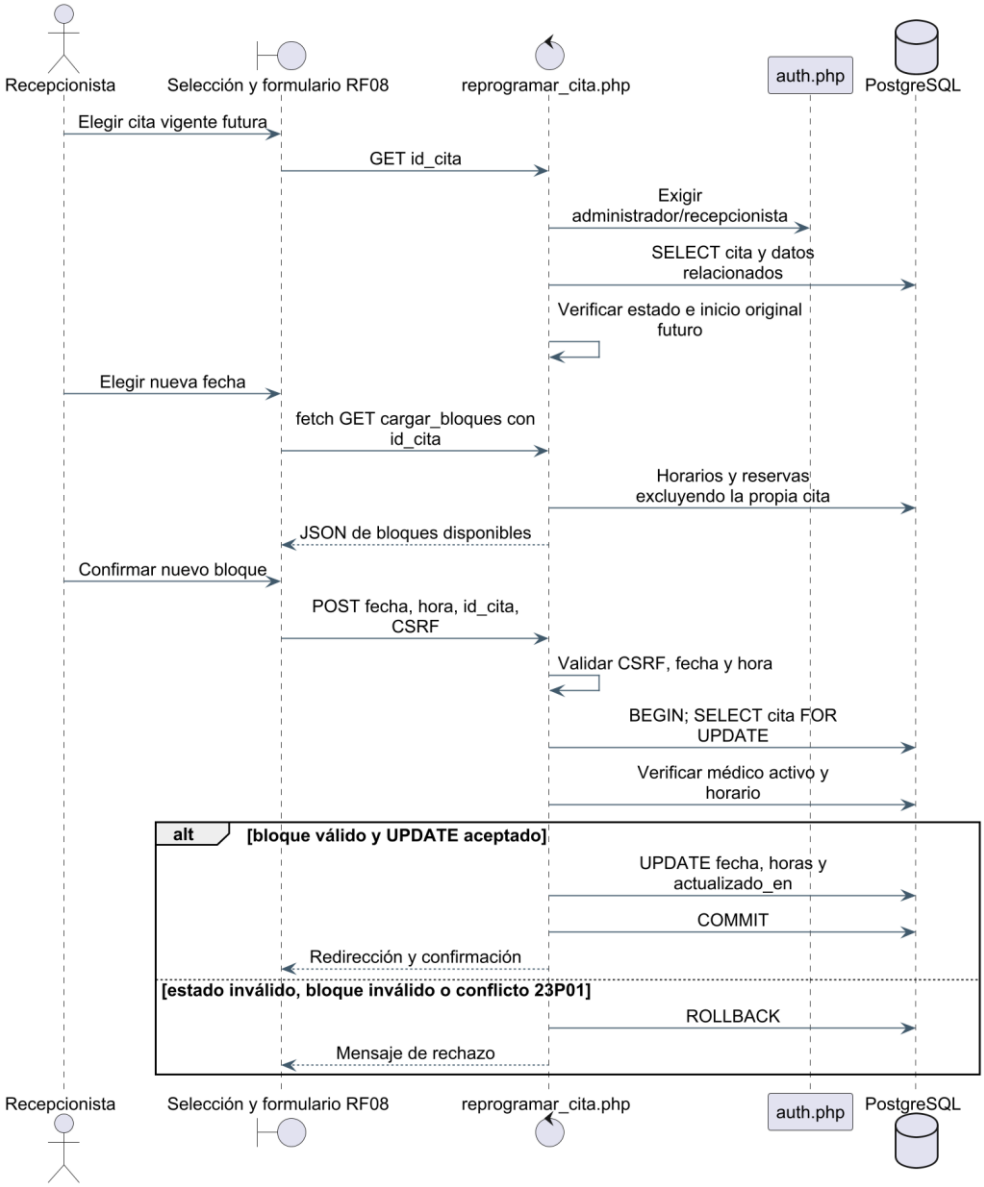
Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D16.puml.

El CI es opcional y las cadenas opcionales vacías se convierten en NULL. nombres, apellidos, fecha_nacimiento y telefono son obligatorios. La fecha de nacimiento debe existir y no ser futura. La unicidad de CI se garantiza por PostgreSQL y el procedimiento traduce 23505 en

un mensaje; su existencia en código no demuestra por sí sola la ejecución de la prueba de duplicado. RF02.

Figura 17

D11 — Secuencia de reprogramación de cita



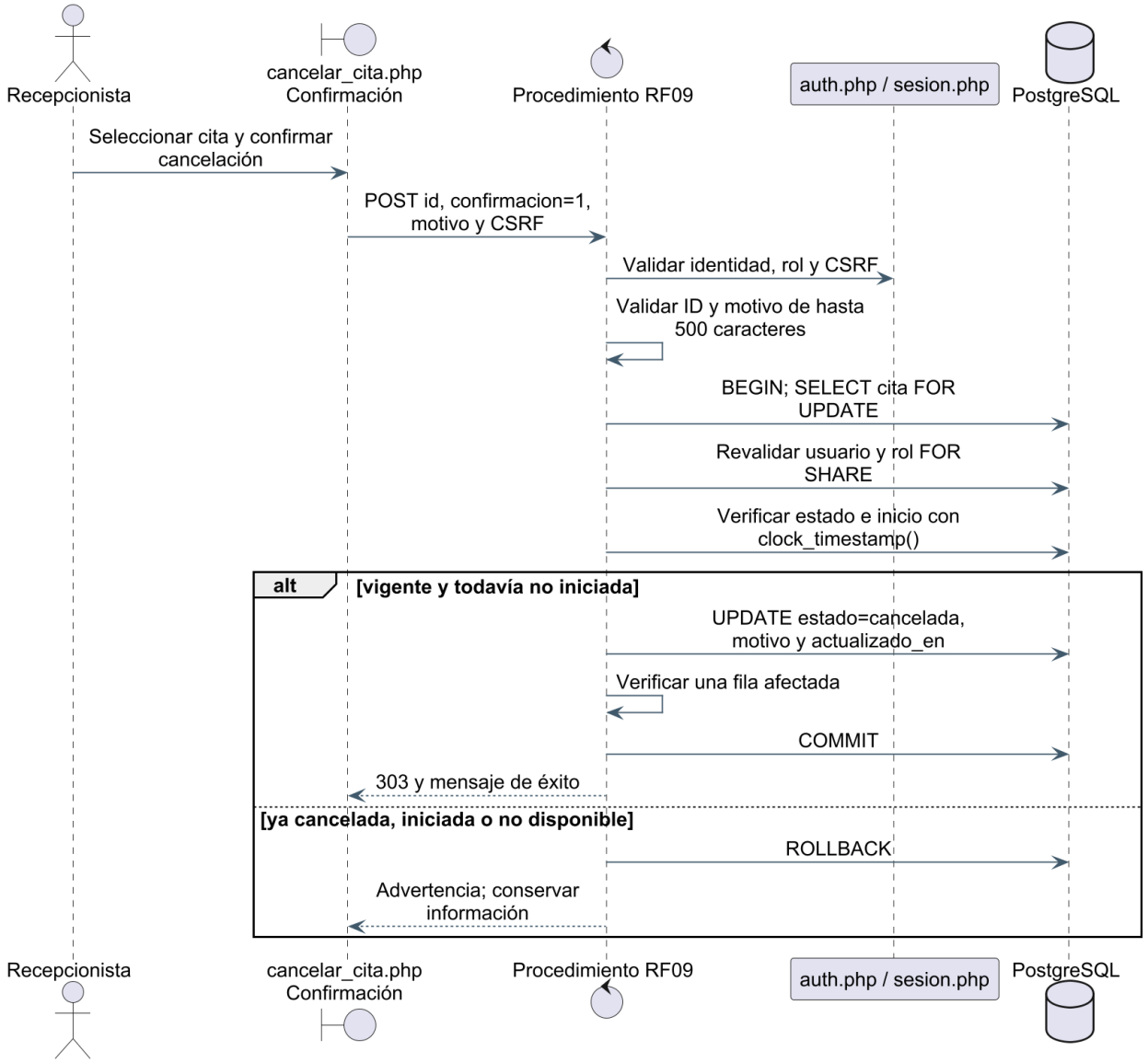
Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D11.puml.

La selección previa es de solo lectura y presenta quince citas por página. La reprogramación conserva id_cita, código, paciente, médico y especialidad; cambia fecha y horas. El bloqueo de fila protege la actualización de esa cita y EXCLUDE protege el intervalo. El inicio

original se controla antes de la transacción, sin una nueva comparación temporal equivalente después de adquirir el bloqueo: constituye un punto de prueba de concurrencia pendiente. RF08 y RF07.

Figura 18

D12 — Secuencia de cancelación de cita



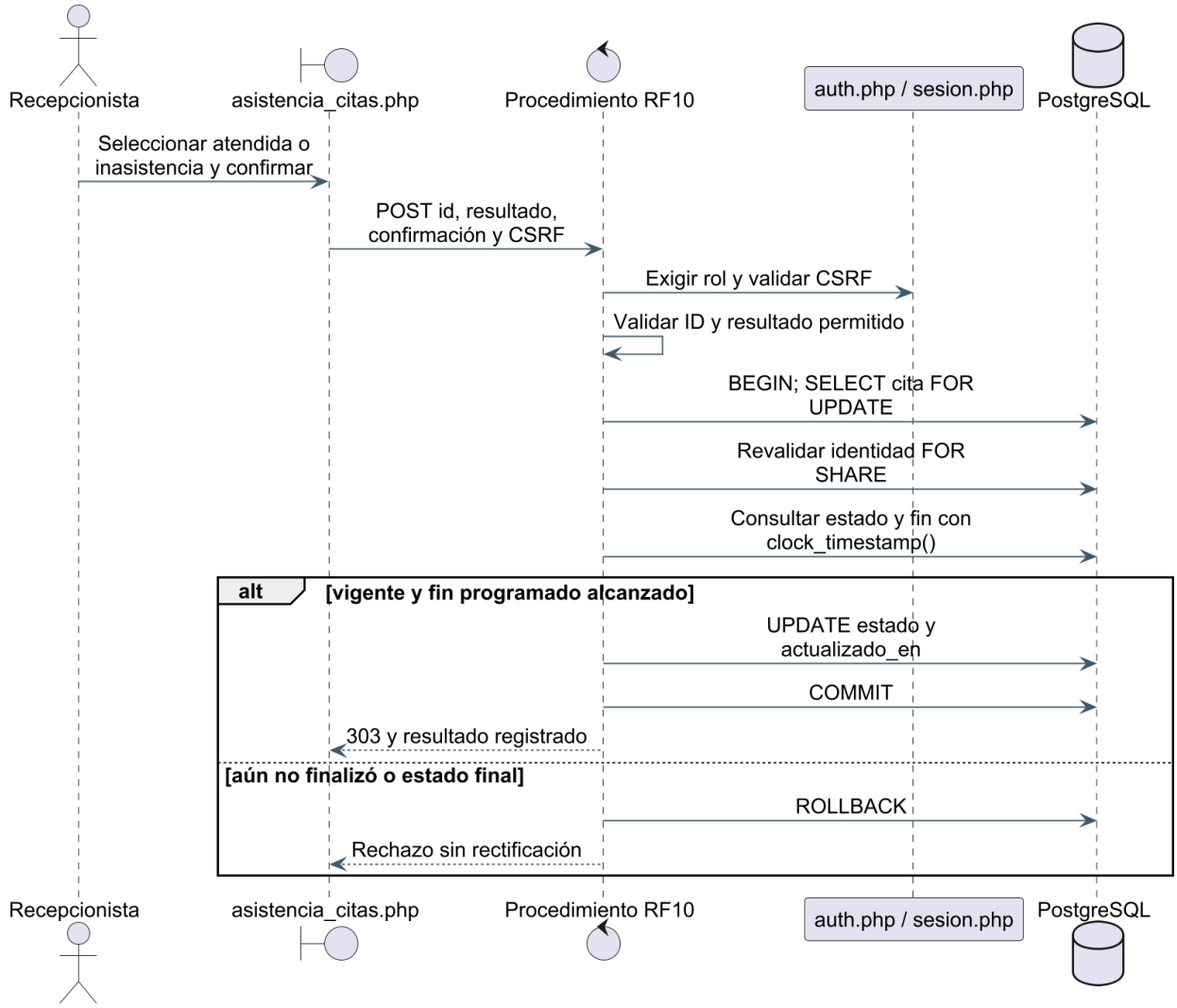
Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D12.puml.

La confirmación expresa y el POST impiden que una consulta GET cancele. La operación revalida el tiempo después de obtener el bloqueo usando clock_timestamp. Al cambiar

a cancelada, el registro permanece, pero ya no participa en la exclusión de reservas vigentes. No se envían notificaciones ni se eliminan datos. RF09.

Figura 19

D13 — Secuencia de registro de asistencia e inasistencia

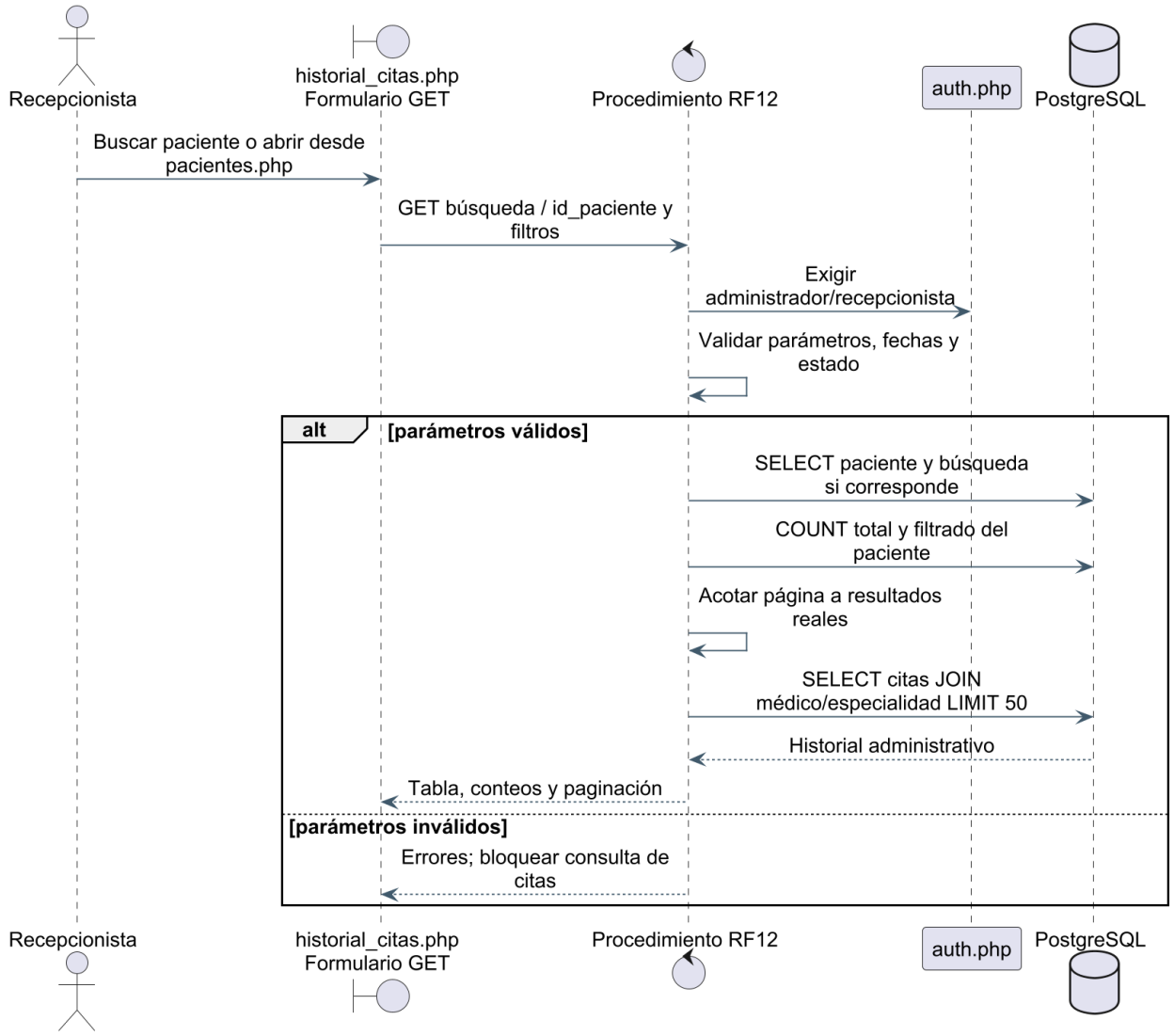


Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D13.puml.

RF10 registra el resultado administrativo después del fin programado, no la llegada ni una hora real de atención. Solo acepta atendida e inasistencia. Una cita cancelada o con resultado final no se rectifica desde este módulo. Las restricciones temporales son más precisas que la expresión informal marcar asistencia y orientan las pruebas de frontera.

Figura 20

D14 — Secuencia de consulta del historial de citas

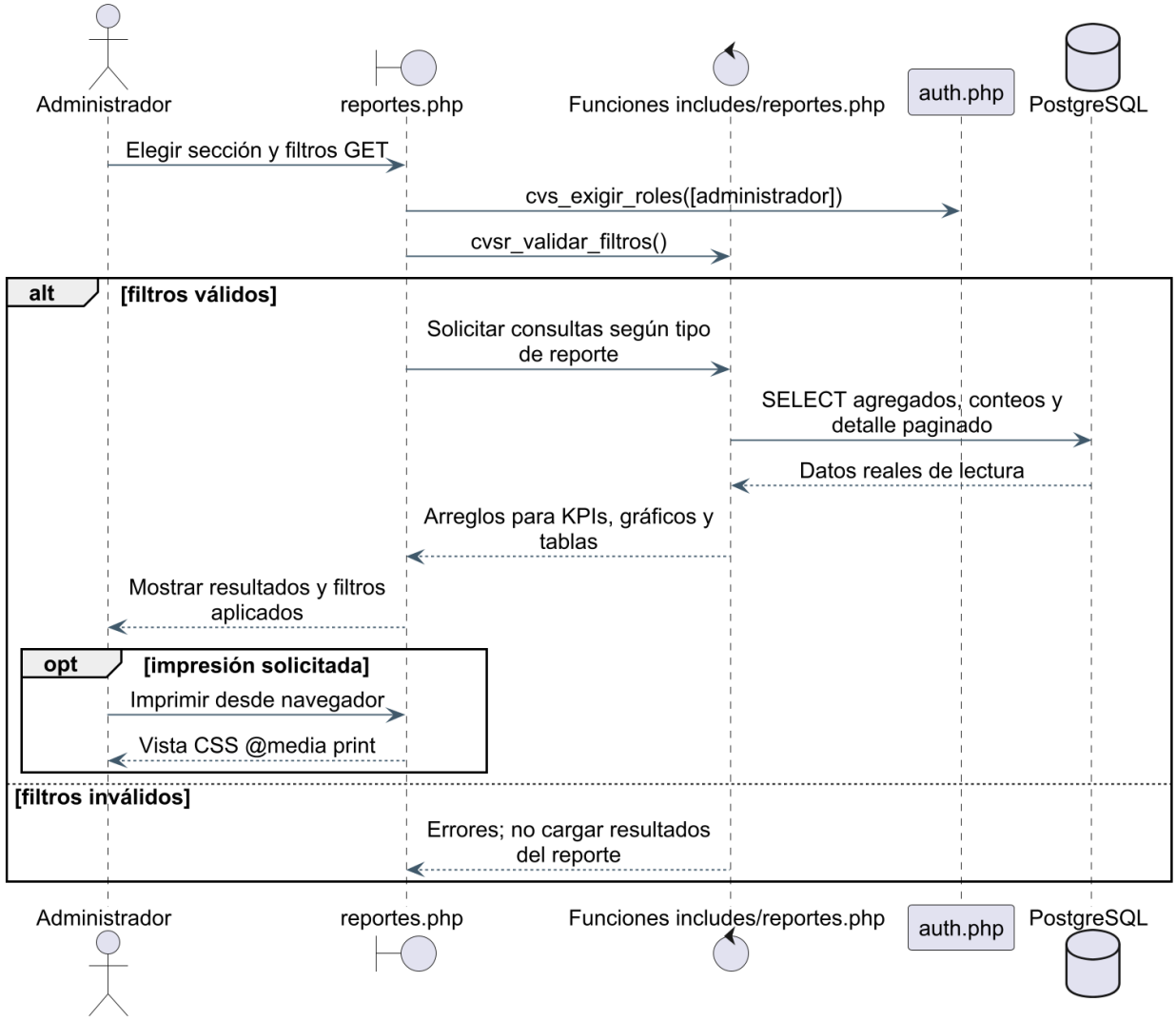


Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D14.puml.

El módulo recibe exclusivamente GET, preserva la identidad del paciente al aplicar filtros e incluye los cinco estados, citas futuras y asociaciones inactivas. No es una historia clínica ni una auditoría de las fechas anteriores de una reprogramación. Los conteos no quedan limitados a las cien filas de la vista abreviada que también existe en pacientes.php. RF12.

Figura 21

D15 — Secuencia de generación de reportes administrativos



Nota. Elaboración propia basada en código y esquema inspeccionados. UML. Fuente editable: D15.puml.

Solo el administrador inicia este caso. Las secciones por citas, médico, especialidad y estado comparten los filtros de citas; pacientes utiliza búsqueda y conteos sin período de alta porque pacientes no tiene creado_en. La agenda usa su propio médico y fecha. La impresión del navegador no equivale a una exportación PDF generada por el servidor. RF13.

Interpretación transversal de las secuencias

El diseño de seguridad y la persistencia se repiten como responsabilidades, no como diagramas inconexos. Iniciar sesión establece la identidad, pero cada módulo protegido la revalida. La cancelación y la asistencia agregan una revalidación dentro de su transacción y bloqueos compartidos sobre usuario y rol. La reprogramación bloquea la cita y comprueba nuevamente su estado. Esas diferencias se conservan en los modelos; no se uniforman artificialmente para aparentar un diseño más elaborado.

Los diagramas de consulta no incorporan INSERT ni UPDATE porque historial y reportes son de lectura. El registro de pacientes no crea usuarios ni una cita automáticamente. Cancelar no borra la fila, y asistencia no registra una hora clínica real. El diseño ayuda a delimitar resultados esperados y a derivar pruebas de rechazo; cuando una capacidad no aparece en código, se registra como pendiente y no se agrega un mensaje inventado para completar visualmente el flujo.

Capítulo complementario. Diseño de datos

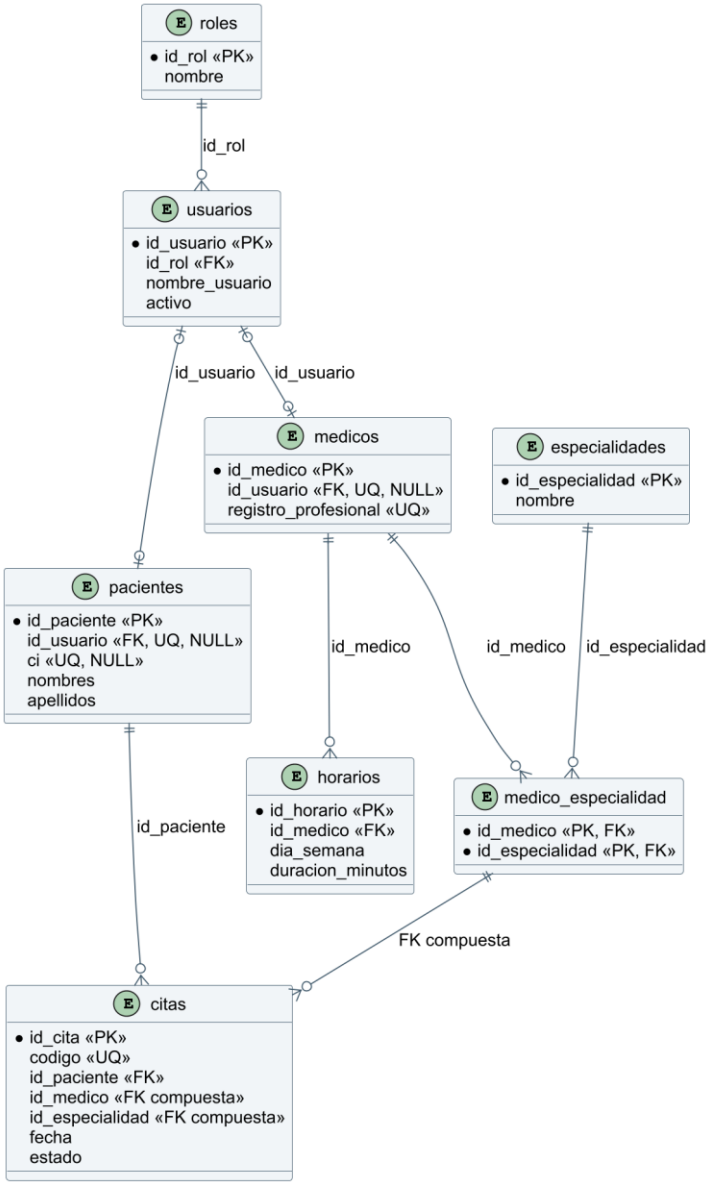
Modelo lógico de datos

El diseño de datos se basa en ocho tablas del esquema public: roles, usuarios, pacientes, medicos, especialidades, medico_especialidad, horarios y citas. La inspección de catálogo se contrastó con el SQL versionado. Los modelos lógicos y físicos se entregan como notación entidad-relación y se distinguen de los diagramas UML de clases: cumplen funciones complementarias en la documentación.

roles clasifica a usuarios. pacientes y medicos pueden relacionarse opcionalmente con una cuenta, pero su existencia administrativa no depende de ella. La oferta profesional se define mediante medicos y especialidades, relacionados por medico_especialidad. horarios describe días semanales e intervalos recurrentes, mientras citas describe una fecha concreta para un paciente y una asignación válida. Esta distinción evita tratar un horario semanal como una reserva ya creada.

Figura 22

D17 — Modelo lógico de datos



Nota. Elaboración propia basada en código y esquema inspeccionados. Entidad-relación. Fuente editable: D17.puml.

El modelo lógico utiliza notación entidad-relación de pata de cuervo, identificada expresamente como modelo de datos y no como diagrama de clases UML. La tabla intermedia resuelve la relación muchos a muchos de médicos y especialidades. Cada cita referencia una asignación válida mediante dos columnas; no hay FK directa a horarios. Las cardinalidades reflejan posibilidad estructural, no cantidad de registros de demostración.

La clave primaria compuesta de medico_especialidad impide repetir una asignación. citas referencia ese par mediante fk_cita_medico_especialidad; de esta manera un médico y una especialidad existentes no bastan si no están asociados. No se declara borrado en cascada en esas relaciones: quitar una asignación utilizada por citas puede producir una violación de referencia y el procedimiento de médicos la comunica con rollback.

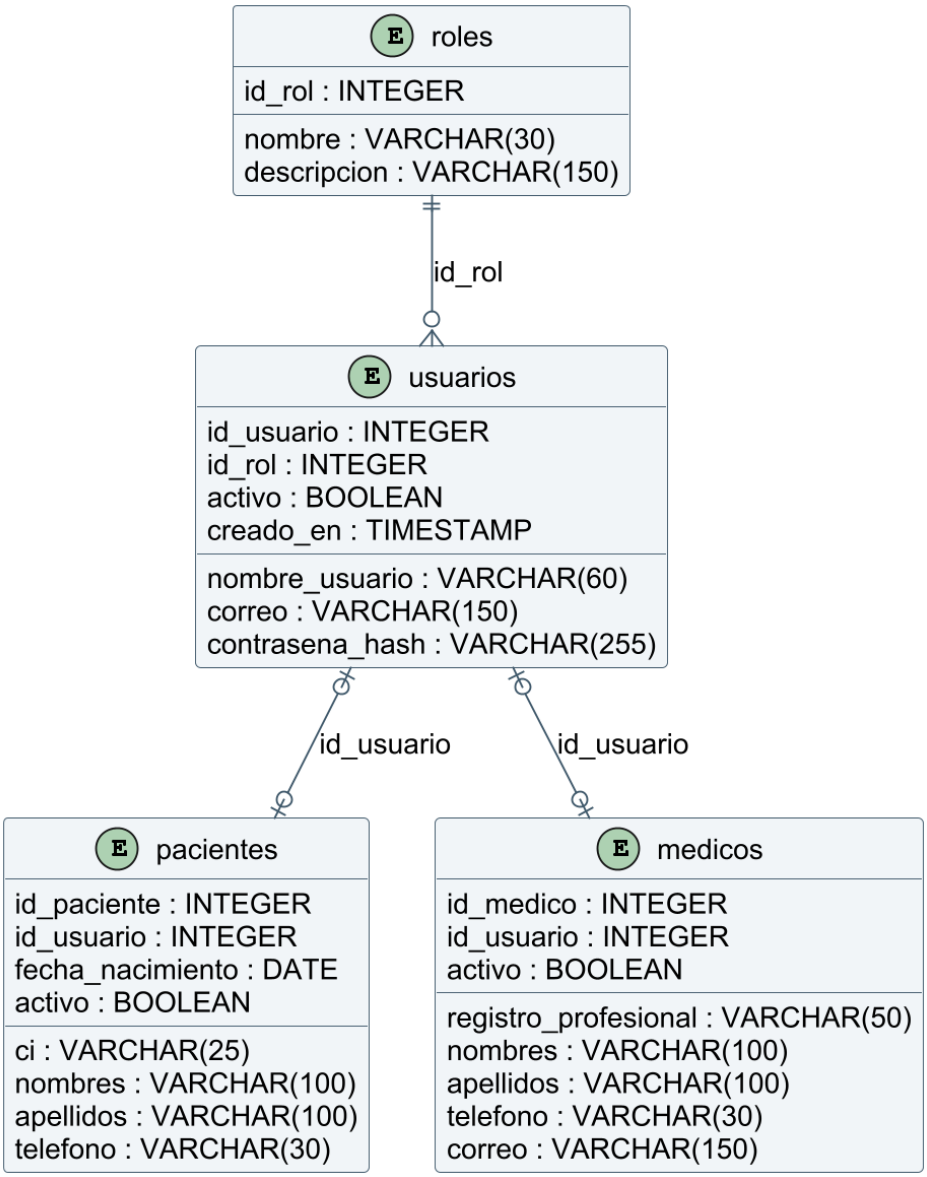
Los índices únicos anulables de id_usuario en los perfiles permiten múltiples filas sin cuenta y, cuando existe vínculo, una fila por usuario dentro de cada tabla. El CI del paciente también admite NULL: su unicidad se aplica al valor no nulo, y el formulario lo considera opcional. La base no contiene un campo creado_en en pacientes; por ello no es correcto describir un reporte de altas de pacientes por período usando ese dato inexistente.

Modelo físico y diccionario de datos

Las claves primarias simples son INTEGER GENERATED ALWAYS AS IDENTITY. Los campos de fecha y hora se almacenan como DATE y TIME sin zona; los timestamps tampoco incorporan zona de forma explícita. Los módulos de reservas y seguimiento interpretan su temporalidad en America/La_Paz. El diccionario siguiente conserva los tipos declarados en el SQL y señala anulabilidad, unicidad y referencias. Las cadenas de ejemplo de demostración y cualquier información privada quedan fuera del diccionario.

Figura 23

D18A — Modelo físico: seguridad y perfiles

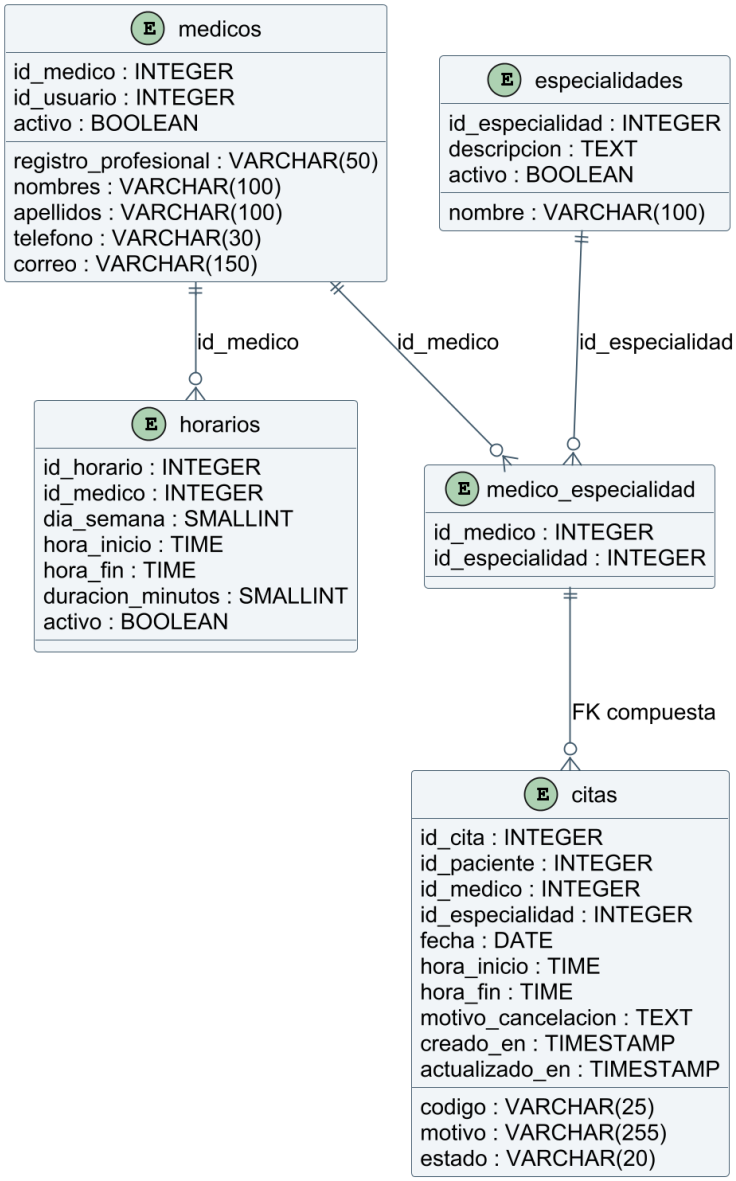


Nota. Elaboración propia basada en código y esquema inspeccionados. Entidad-relación. Fuente editable: D18A.puml.

Vista parcial del modelo físico D18. El diccionario de datos conserva todos los campos y sus restricciones; esta proyección facilita la lectura impresa sin cambiar entidades ni introducir atributos.

Figura 24

D18B — Modelo físico: oferta y reservas



Nota. Elaboración propia basada en código y esquema inspeccionados. Entidad-relación. Fuente editable: D18B.puml.

Vista parcial del modelo físico D18. El diccionario de datos conserva todos los campos y sus restricciones; esta proyección facilita la lectura impresa sin cambiar entidades ni introducir atributos.

La Tabla 30 describe la entidad roles, que identifica los cuatro perfiles del sistema y exige un nombre único para cada rol.

Tabla 30

Diccionario físico de roles

Campo	Tipo SQL	Claves, nulabilidad y condiciones
id_rol	INTEGER	PK; identidad; No nulo
nombre	VARCHAR(30)	No nulo; UNIQUE
descripcion	VARCHAR(150)	Admite NULL

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Cuentas de acceso; contraseña representada por hash y actividad revalidada.

Tabla 31

Diccionario físico de usuarios

Campo	Tipo SQL	Claves, nulabilidad y condiciones
id_usuario	INTEGER	PK; identidad; No nulo
id_rol	INTEGER	No nulo; REFERENCES roles(id_rol)
nombre_usuario	VARCHAR(60)	No nulo; UNIQUE
correo	VARCHAR(150)	Admite NULL; UNIQUE
contrasena_hash	VARCHAR(255)	No nulo
activo	BOOLEAN	No nulo; DEFAULT TRUE
creado_en	TIMESTAMP	No nulo; DEFAULT CURRENT_TIMESTAMP

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Ficha administrativa con identificación, contacto y datos opcionales; desactivación conserva citas.

Tabla 32

Diccionario físico de pacientes

Campo	Tipo SQL	Claves, nulabilidad y condiciones
id_paciente	INTEGER	PK; identidad; No nulo
id_usuario	INTEGER	Admite NULL; UNIQUE; REFERENCES usuarios(id_usuario)
ci	VARCHAR(25)	Admite NULL; UNIQUE
nombres	VARCHAR(100)	No nulo
apellidos	VARCHAR(100)	No nulo
fecha_nacimiento	DATE	No nulo

Campo	Tipo SQL	Claves, nulabilidad y condiciones
sexo	VARCHAR(25)	Admite NULL
telefono	VARCHAR(30)	No nulo
correo	VARCHAR(150)	Admite NULL
direccion	VARCHAR(255)	Admite NULL
tipo_sangre	VARCHAR(5)	Admite NULL
alergias	TEXT	Admite NULL
antecedentes	TEXT	Admite NULL
contacto_emergencia	VARCHAR(150)	Admite NULL
telefono_emergencia	VARCHAR(30)	Admite NULL
activo	BOOLEAN	No nulo; DEFAULT TRUE

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Profesionales con registro único; asociación opcional a cuenta y oferta por tabla intermedia.

Tabla 33

Diccionario físico de medicos

Campo	Tipo SQL	Claves, nulabilidad y condiciones
id_medico	INTEGER	PK; identidad; No nulo
id_usuario	INTEGER	Admite NULL; UNIQUE; REFERENCES usuarios(id_usuario)
registro_profesional	VARCHAR(50)	No nulo; UNIQUE
nombres	VARCHAR(100)	No nulo
apellidos	VARCHAR(100)	No nulo
telefono	VARCHAR(30)	Admite NULL
correo	VARCHAR(150)	Admite NULL
activo	BOOLEAN	No nulo; DEFAULT TRUE

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Catálogo con actividad y unicidad normalizada mediante índice funcional.

Tabla 34

Diccionario físico de especialidades

Campo	Tipo SQL	Claves, nulabilidad y condiciones
id_especialidad	INTEGER	PK; identidad; No nulo
nombre	VARCHAR(100)	No nulo
descripcion	TEXT	Admite NULL
activo	BOOLEAN	No nulo; DEFAULT TRUE

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Asignación de oferta; clave compuesta utilizada por citas.

Tabla 35

Diccionario físico de medico_especialidad

Campo	Tipo SQL	Claves, nulabilidad y condiciones
id_medico	INTEGER	PK; No nulo; REFERENCES medicos(id_medico)
id_especialidad	INTEGER	PK; No nulo; REFERENCES especialidades(id_especialidad)

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Oferta semanal con duración de bloque y exclusión de intervalos activos del mismo médico/día.

Tabla 36

Diccionario físico de horarios

Campo	Tipo SQL	Claves, nulabilidad y condiciones
id_horario	INTEGER	PK; identidad; No nulo
id_medico	INTEGER	No nulo; REFERENCES medicos(id_medico)
dia_semana	SMALLINT	No nulo; CHECK (dia_semana BETWEEN 1 AND 7)
hora_inicio	TIME	No nulo
hora_fin	TIME	No nulo
duracion_minutos	SMALLINT	No nulo; DEFAULT 15 CHECK (duracion_minutos > 0)
activo	BOOLEAN	No nulo; DEFAULT TRUE

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Reserva fechada con estado actual, motivos y marcas de creación/actualización.

Tabla 37

Diccionario físico de citas

Campo	Tipo SQL	Claves, nulabilidad y condiciones
id_cita	INTEGER	PK; identidad; No nulo
codigo	VARCHAR(25)	No nulo; UNIQUE
id_paciente	INTEGER	No nulo; REFERENCES pacientes(id_paciente)
id_medico	INTEGER	No nulo; FK compuesta a medico_especialidad
id_especialidad	INTEGER	No nulo; FK compuesta a medico_especialidad
fecha	DATE	No nulo
hora_inicio	TIME	No nulo
hora_fin	TIME	No nulo
motivo	VARCHAR(255)	Admite NULL
estado	VARCHAR(20)	No nulo; DEFAULT 'programada'
motivo_cancelacion	TEXT	Admite NULL
creado_en	TIMESTAMP	No nulo; DEFAULT CURRENT_TIMESTAMP
actualizado_en	TIMESTAMP	No nulo; DEFAULT CURRENT_TIMESTAMP

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Integridad referencial, unicidad e índices

Tabla 38

Restricciones e índices relevantes

Elemento real	Tipo	Garantía y límite
roles.nombre; usuarios.nombre_usuario/correo	UNIQUE	Identidad de rol y cuenta; correo de usuario admite NULL.
pacientes.ci; medicos.registro_profesional	UNIQUE	Evita repetición escalar; CI es opcional, registro profesional no.
uq_especialidades_nombre	Índice único funcional	LOWER(TRIM(nombre)); no duplica variando caso o espacios laterales.
fk_cita_medico_especialidad	FK compuesta	Exige asignación válida; no comprueba por sí misma activo del catálogo.
ck_horarios_horas; día y duración	CHECK	Fin mayor que inicio; día 1–7; duración positiva.

Elemento real	Tipo	Garantía y límite
ck_cita_horas; ck_cita_estado	CHECK	Fin mayor que inicio y estado dentro de cinco valores.
ex_horarios_medico	EXCLUDE GiST parcial	Médico y día iguales no comparten rango semanal activo superpuesto.
ex_citas_medico_sin_superposicion	EXCLUDE GiST parcial	Mismo médico sin intersección temporal entre programadas/confirmadas.
ix_citas_fecha	Índice B-tree	Apoya consultas por fecha; no es una regla de exclusión.
ix_citas_paciente_fecha; ix_citas_medico_fecha	Índices compuestos	Apoyan historial y agenda por sus identificadores y fecha.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

La integridad referencial garantiza referencias existentes, no la totalidad de la política operativa. No existen en citas una FK a horarios ni un CHECK que consulte la oferta semanal; la pertenencia del bloque y la duración se deciden en PHP. No se observaron disparadores que conserven un historial de reprogramaciones. El estado confirmada está permitido, pero el esquema no define una máquina de transiciones: son los módulos quienes restringen los cambios.

El README menciona PERIOD/EXCLUDE; el SQL real utiliza EXCLUDE con tsrange y no contiene una declaración PERIOD. El modelo físico documenta la construcción existente, evitando trasladar una formulación imprecisa de la descripción a la base. La exclusión por médico no se extiende a paciente. También existe un texto de error de horarios que alude a citas asociadas, pero el esquema no tiene FK cita-horario; se registra como discrepancia de mensaje y no como relación de datos.

Consistencia de intervalos y límites de concurrencia

Para horarios se transforma cada TIME a un rango anclado en la fecha constante 2000-01-03 y se combina con dia_semana. Ese anclaje sirve para comparar horas, no para afirmar que

el médico atiende en esa fecha histórica. Para citas se utiliza la fecha real más las horas. En ambos casos [) permite adyacencia: 09:00–09:15 y 09:15–09:30 no comparten tiempo interior.

En el registro, la lectura de turnos elimina intervalos ocupados para orientar al usuario. El POST revisa actividad y oferta temporal y el INSERT se somete a EXCLUDE. En reprogramación se excluye la propia fila al calcular disponibilidad y se bloquea la cita durante el cambio. En cancelación y asistencia, el cambio de estado hace que esa fila deje de participar en la exclusión vigente. Estas garantías existen en el esquema inspeccionado; la eficacia funcional concurrente se conserva como prueba pendiente porque no se ejecutaron escrituras simultáneas.

La lectura de pacientes o médicos activos durante la reserva no utiliza el mismo bloqueo compartido que RF09/RF10 aplican a la identidad. Por ello, no se afirma una protección completa contra cualquier cambio concurrente de los catálogos. También se debe revisar la homogeneidad temporal de módulos que utilizan la zona clínica frente a los que dependen de la configuración PHP. La recomendación es probar esas fronteras antes de elevar el incremento a una instalación pública.

Implementación física de la base de datos: script SQL

A continuación se reproduce íntegramente el archivo database/clinica_vida_salud.sql publicado en la rama main de GitHub. El script contiene las ocho tablas, sus relaciones, las restricciones de integridad, los índices y los registros iniciales de roles y especialidades. Se presenta como código fuente del sistema, no como una consulta ejecutada durante la preparación de este informe.

Listado SQL 1. Script de creación de la base de datos Clínica Vida Salud

```
-- CLINICA VIDA SALUD - Estructura inicial de base de datos
-- PostgreSQL 18 (compatible con versiones modernas de PostgreSQL)
-- Ejecutar una sola vez dentro de la base de datos clinica_vida_salud.
-- Los datos que se registren posteriormente NO se comparten automaticamente entre
instalaciones locales.

BEGIN;

-- Permite impedir reservas de intervalos superpuestos por medico.
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE roles (
    id_rol INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(30) NOT NULL UNIQUE,
    descripcion VARCHAR(150)
);

CREATE TABLE usuarios (
    id_usuario INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_rol INTEGER NOT NULL REFERENCES roles(id_rol),
    nombre_usuario VARCHAR(60) NOT NULL UNIQUE,
    correo VARCHAR(150) UNIQUE,
    contrasena_hash VARCHAR(255) NOT NULL,
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    creado_en TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE pacientes (
    id_paciente INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_usuario INTEGER UNIQUE REFERENCES usuarios(id_usuario),
    ci VARCHAR(25) UNIQUE,
    nombres VARCHAR(100) NOT NULL,
    apellidos VARCHAR(100) NOT NULL,
    fecha_nacimiento DATE NOT NULL,
    sexo VARCHAR(25),
    telefono VARCHAR(30) NOT NULL,
    correo VARCHAR(150),
    direccion VARCHAR(255),
    tipo_sangre VARCHAR(5),
    alergias TEXT,
    antecedentes TEXT,
    contacto_emergencia VARCHAR(150),
    telefono_emergencia VARCHAR(30),
    activo BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE medicos (
    id_medico INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_usuario INTEGER UNIQUE REFERENCES usuarios(id_usuario),
    registro_profesional VARCHAR(50) NOT NULL UNIQUE,
    nombres VARCHAR(100) NOT NULL,
    apellidos VARCHAR(100) NOT NULL,
    telefono VARCHAR(30),
```

```
    correo VARCHAR(150),
    activo BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE especialidades (
    id_especialidad INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT,
    activo BOOLEAN NOT NULL DEFAULT TRUE
);

-- No permite nombres duplicados sin importar mayusculas/minusculas o espacios laterales.
CREATE UNIQUE INDEX uq_especialidades_nombre ON especialidades (LOWER(TRIM(nombre)));

CREATE TABLE medico_especialidad (
    id_medico INTEGER NOT NULL REFERENCES medicos(id_medico),
    id_especialidad INTEGER NOT NULL REFERENCES especialidades(id_especialidad),
    PRIMARY KEY (id_medico, id_especialidad)
);

CREATE TABLE horarios (
    id_horario INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_medico INTEGER NOT NULL REFERENCES medicos(id_medico),
    dia_semana SMALLINT NOT NULL CHECK (dia_semana BETWEEN 1 AND 7),
    hora_inicio TIME NOT NULL,
    hora_fin TIME NOT NULL,
    duracion_minutos SMALLINT NOT NULL DEFAULT 15 CHECK (duracion_minutos > 0),
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT ck_horarios_horas CHECK (hora_fin > hora_inicio)
);

-- Evita que un mismo medico tenga horarios semanales activos que se solapen.
ALTER TABLE horarios ADD CONSTRAINT ex_horarios_medico
    EXCLUDE USING gist (
        id_medico WITH =,
        dia_semana WITH =,
        tsrange(DATE '2000-01-03' + hora_inicio, DATE '2000-01-03' + hora_fin, '[)') WITH &&
    ) WHERE (activo);

CREATE TABLE citas (
    id_cita INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    codigo VARCHAR(25) NOT NULL UNIQUE,
    id_paciente INTEGER NOT NULL REFERENCES pacientes(id_paciente),
    id_medico INTEGER NOT NULL,
    id_especialidad INTEGER NOT NULL,
    fecha DATE NOT NULL,
    hora_inicio TIME NOT NULL,
    hora_fin TIME NOT NULL,
    motivo VARCHAR(255),
    estado VARCHAR(20) NOT NULL DEFAULT 'programada',
    motivo_cancelacion TEXT,
    creado_en TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    actualizado_en TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_cita_medico_especialidad FOREIGN KEY (id_medico, id_especialidad)
        REFERENCES medico_especialidad(id_medico, id_especialidad),
    CONSTRAINT ck_cita_horas CHECK (hora_fin > hora_inicio),
    CONSTRAINT ck_cita_estado CHECK (estado IN ('programada', 'confirmada', 'atendida',
'cancelada', 'inasistencia'))
);

-- Exclusion atomica: dos reservas vigentes del mismo medico no pueden solaparse,
-- incluso si se guardan desde dos ventanas al mismo tiempo.
ALTER TABLE citas ADD CONSTRAINT ex_citas_medico_sin_superposicion
    EXCLUDE USING gist (
        id_medico WITH =,
        tsrange(fecha + hora_inicio, fecha + hora_fin, '[)') WITH &&
    ) WHERE (estado IN ('programada', 'confirmada'));

CREATE INDEX ix_citas_fecha ON citas(fecha);
CREATE INDEX ix_citas_paciente_fecha ON citas(id_paciente, fecha);
CREATE INDEX ix_citas_medico_fecha ON citas(id_medico, fecha);

INSERT INTO roles (nombre, descripcion) VALUES
    ('administrador', 'Administra usuarios, medicos, especialidades y reportes'),
    ('recepcionista', 'Registra pacientes y gestiona citas'),
```

```
    ('medico', 'Consulta su agenda y gestiona la atencion autorizada'),
    ('paciente', 'Consulta su perfil y sus citas');

INSERT INTO especialidades (nombre, descripcion) VALUES
    ('Pediatria', 'Atencion medica de ninos y adolescentes'),
    ('Medicina general', 'Atencion primaria y consultas generales'),
    ('Geriatria', 'Atencion de adultos mayores');

COMMIT;

-- COMPROBACION (ejecutar despues de crear las tablas):
-- SELECT table_name FROM information_schema.tables
-- WHERE table_schema = 'public' AND table_type = 'BASE TABLE'
-- ORDER BY table_name;
```

Nota. El código se transcribe del archivo SQL de la rama principal (main). Su ejecución requiere una base de datos preparada y los permisos apropiados; el script incorpora sentencias INSERT de datos iniciales de catálogo. El archivo ejecutable original permanece disponible en el repositorio de GitHub.

Capítulo complementario. Implementación del Ciclo 1

Herramientas de desarrollo y evidencia disponible

La tecnología de implementación se identifica en los archivos y el entorno: PHP procedimental, HTML, CSS y JavaScript nativo; PostgreSQL como motor; Apache mediante XAMPP en la instalación indicada; y Git/GitHub para versionado y publicación del repositorio. Se observó PHP 8.2.12 por CLI y PostgreSQL 18.6 mediante SHOW server_version. No se atribuyen versiones de Apache ni del navegador que no fueron registradas.

Visual Studio Code figura entre las herramientas solicitadas para describir el proyecto, pero su uso histórico no se acredita mediante los archivos revisados. Se lo reconoce como editor indicado para el contexto de desarrollo, sin afirmarlo como evidencia comprobada de autoría. Los modelos de esta entrega se elaboraron en PlantUML 1.2025.2 y se exportaron realmente a SVG y PNG; no se afirma que hayan sido creados en StarUML ni se entrega un archivo mdj de validez no comprobada. El informe se construyó con estilos de Word, tablas editables y fuentes de generación conservadas en control_documental.

Arquitectura implementada y organización del repositorio

Los puntos de entrada de la raíz representan objetivos funcionales. includes aporta funciones compartidas, assets contiene la presentación y database conserva el script de creación de esquema. scripts incluye herramientas de consola para el alta inicial del administrador y cuentas de prueba; no se ejecutaron en esta revisión y no sustituyen administración completa de usuarios desde la web. Los diagnósticos probar_conexion.php y verificar.php están restringidos a consola y no se describen como pantallas de negocio.

La conexión se obtiene de forma diferida mediante cvs_conexion, con una instancia estática PDO y configuración privada externa. El documento registra la responsabilidad y el resguardo de esa configuración, sin reproducirla. Las páginas anónimas no necesitan abrir conexión solo por cargar funciones; una identidad autenticada sí se revalida con la base. Este comportamiento explica tanto la portada pública como la protección posterior de los módulos.

Tabla 39

Mapa de implementación por subsistema

Subsistema	**RF**	**Archivos principales**	**Responsabilidad real**	**Estado**
Seguridad	RF01	login.php; logout.php; includes/auth.php; includes/sesion.php	Hash, identidad, expiración, roles, CSRF y cierre.	Inspeccionado; funciones personales y usuarios web pendientes.
Pacientes	RF02	pacientes.php	Alta, edición, búsqueda, activación y desactivación; enlace a historial.	Código presente; alta reportada.
Oferta profesional	RF03; RF04	medicos.php; especialidades.php	Catálogos y asignaciones; unicidad y conservación referencial.	Código inspeccionado; batería pendiente.
Oferta temporal	RF05	horarios.php	Día, intervalo, duración y actividad; exclusión semanal.	Código y restricción inspeccionados.
Reservas	RF06; RF07	citas.php; SQL de citas	GET JSON de oferta; POST validado e INSERT transaccional.	Registro y disponibilidad reportados; concurrencia pendiente.
Reprogramación	RF08	seleccionar_reprogramacion.php; reprogramar_cita.php	Selección paginada y cambio de programación con bloqueo.	Recorrido integral pendiente.
Seguimiento	RF09; RF10	cancelar_cita.php; asistencia_citas.php	Transiciones temporales con confirmación; conservación de datos.	Resultados reportados y agregados corroborados.

Subsistema	RF	Archivos principales	Responsabilidad real	Estado
Consultas	RF11; RF12	agenda_medica.php; historial_citas.php; pacientes.php	Agenda administrativa e historial filtrado.	Agenda personal y pruebas completas pendientes.
Reportes	RF13	reportes.php; includes/reportes.php	Seis secciones; conteos, filtros, gráficos y paginación.	Consulta reportada; batería completa pendiente.
Presentación común	Transversal	includes/ui.php; assets/estilos.css	Marco visual y menú por rol; adaptabilidad CSS.	Diseño inspeccionado; móvil integral pendiente.
Panel administrativo	RF02; RF05; RF06; RF13 como apoyo	index.php; includes/dashboard.php	KPIs, calendario, turnos y actividad de solo lectura.	Integración inspeccionada; lectura de actividad por creado_en.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Integración de los módulos RF01–RF13

RF01 proporciona el contrato común de identidad y permisos. RF02 aporta pacientes utilizables por RF06; RF03 y RF04 construyen las asignaciones que una cita debe referenciar; RF05 aporta los intervalos y la duración. RF06 registra una reserva y RF07 impide el resultado incompatible frente a concurrencia. Estos vínculos se materializan mediante consultas y restricciones reales, no mediante un servicio ficticio de objetos.

RF08 conserva la identidad de la cita al cambiar su programación. RF09 conserva la misma fila al cancelarla y RF10 registra un resultado final después de terminar el intervalo. RF11 consulta por médico y fecha para personal administrativo. RF12 recibe el identificador desde pacientes.php y mantiene el ámbito del paciente durante filtros y paginación. RF13 agrega información de esas mismas tablas y restringe su entrada al administrador. El dashboard reutiliza datos sin escribir y no se confunde con un reporte clínico.

La sesión conserva cvs_auth con id_usuario, id_rol, nombre_usuario y rol; cvs_ultima_actividad controla treinta minutos de inactividad. También puede contener tokens,

avisos y conteos de fallos por sesión. El identificador se regenera al olvidar autenticación y al establecer un acceso válido. La cookie usa HttpOnly, SameSite=Lax y Secure cuando la solicitud es HTTPS; no se afirma que una instalación HTTP local ya garantice transporte cifrado.

Los formularios de pacientes, especialidades, médicos, horarios, citas y reprogramación utilizan tokens específicos de módulo; login, logout y seguimiento aprovechan el token común. Las consultas preparadas y el escape de salida se observan en las entradas y vistas. La protección de roles se ejecuta en cada módulo; un usuario médico no obtiene reportes por conocer la URL. El límite de intentos del login es por sesión y puede eludirse creando otra, como reconoce el propio código; no se presenta como control centralizado de producción.

Capítulo complementario. Pruebas del Ciclo 1

Plan de pruebas

El propósito del plan de pruebas es verificar la correspondencia entre requisitos, modelos y comportamiento del sistema, especialmente en la reserva consistente, la autorización y la conservación de información. El enfoque metodológico distingue la planificación, el diseño de casos y procedimientos, la ejecución y la evaluación de resultados (Torossi, s. f., pp. 51–54). Las revisiones estáticas y las consultas de solo lectura no sustituyen una prueba funcional de extremo a extremo, como también distingue SWEBOK (s. f.-b).

El alcance funcional comprende los diez escenarios mínimos transcritos en la solicitud y doce complementarios. Se incluyen clases de entrada válidas e inválidas, transiciones de estado y concurrencia. La prioridad es alta para acceso, CI duplicado, reserva válida, superposición y pertenencia al horario. La existencia de una restricción se evalúa técnicamente, pero su respuesta desde la interfaz requiere un recorrido funcional que se registra por separado.

Los recursos previstos son las integrantes del grupo, un navegador de escritorio, perfiles de prueba de los cuatro roles, datos ficticios identificables, servidor local PHP/Apache y PostgreSQL, además de dispositivos para la verificación móvil. Esa enumeración es un plan de recursos y no una declaración de que todas las cuentas o dispositivos fueron utilizados. Las operaciones de escritura deberán ejecutarse posteriormente en un conjunto de pruebas controlado; durante la presente revisión documental no se registraron ni modificaron pacientes, usuarios, horarios o citas.

Tabla 40
Componentes operativos del plan

Componente	**Definición para el Ciclo 1**
Entorno observado	Windows; PHP CLI 8.2.12; PostgreSQL 18.6; instalación local XAMPP indicada.

Componente	Definición para el Ciclo 1
Entrada	Línea base del proyecto actualmente publicada en main; requisitos RF01–RF13; modelos y casos de prueba.
Procedimiento	Preparar datos de ensayo; registrar estado inicial; ejecutar pasos; contrastar esperado/observado; conservar evidencia sin secretos.
Técnicas previstas	Particiones de equivalencia, valores de frontera, escenarios, estados y dos sesiones para concurrencia.
Criterio de aceptación de un caso	Resultado observado coincide con el esperado y se conserva evidencia reproducible suficiente.
Criterio de evaluación del ciclo	Riesgos críticos revisados, pendientes identificados y trazabilidad completa; no aprobación global por sintaxis.
Resultado de esta revisión	Sintaxis PHP, metadatos y agregados de solo lectura registrados; operaciones funcionales no repetidas.
Salida	Matriz PF01–PF22, inventario de evidencia y riesgos para la próxima validación.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Matriz de pruebas funcionales

La matriz se presenta en fichas tabulares para mantener legibles las once dimensiones solicitadas en papel carta vertical. La hoja Pruebas del archivo Excel conserva el conjunto completo con filtros. Reportada significa que la comprobación fue comunicada en el avance suministrado, sin bitácora reproducida aquí. Un dato corroborado por SELECT acredita persistencia observada, pero no reconstruye automáticamente los pasos de la interfaz ni una ejecución nueva del caso. El estado «Aprobada» califica únicamente la especificación del caso de prueba, no su ejecución. La ejecución, los resultados observados y las evidencias se documentan por separado; los ensayos sin evidencia conservan su condición pendiente.

Tabla 41
PF01 — Registrar paciente

Dimensión	Registro de prueba
Código	PF01
Requisito relacionado	RF02
Objetivo	Registrar paciente

Dimensión	Registro de prueba
Precondición	Administrador o recepcionista autenticado; ficha de prueba nueva
Datos de entrada previstos	Nombres Prueba, apellidos Académica, nacimiento válido, teléfono ficticio; CI reservado para ensayo
Procedimiento previsto	Abrir pacientes; completar obligatorios; enviar; buscar la ficha
Resultado esperado	Mensaje de alta y ficha persistida activa
Resultado observado o conocido	Alta comprobada durante desarrollo según información suministrada; no repetida en esta revisión
Estado del diseño	Aprobada
Evidencia	Declaración de avance; pacientes.php:106–150

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 42

PF02 — Rechazar CI duplicado

Dimensión	Registro de prueba
Código	PF02
Requisito relacionado	RF02
Objetivo	Rechazar CI duplicado
Precondición	Paciente de ensayo con CI no nulo ya existente
Datos de entrada previstos	Mismo CI y demás obligatorios válidos
Procedimiento previsto	Intentar segunda alta; comprobar que no aumenta el conjunto por ese CI
Resultado esperado	23505 traducido a CI ya registrado; segunda ficha no creada
Resultado observado o conocido	No ejecutado; UNIQUE y manejo de error inspeccionados
Estado del diseño	Aprobada
Evidencia	SQL:27–44; pacientes.php:141–149

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 43

PF03 — Registrar cita en turno libre

Dimensión	Registro de prueba
Código	PF03
Requisito relacionado	RF06; RF07
Objetivo	Registrar cita en turno libre

Dimensión	Registro de prueba
Precondición	Paciente y médico activos, asignación válida, horario futuro libre
Datos de entrada previstos	Paciente de ensayo; médico, especialidad, fecha futura y bloque alineado
Procedimiento previsto	Consultar bloques; elegir uno; enviar; localizar código en listado
Resultado esperado	Cita programada con inicio y fin calculados; confirmación
Resultado observado o conocido	Registro válido reportado durante desarrollo; sin repetición de escritura
Estado del diseño	Aprobada
Evidencia	Declaración de avance; citas.php:187–328

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 44

PF04 — Rechazar cita superpuesta

Dimensión	Registro de prueba
Código	PF04
Requisito relacionado	RF07
Objetivo	Rechazar cita superpuesta
Precondición	Una reserva vigente del mismo médico
Datos de entrada previstos	Segundo intervalo con solapamiento parcial, total y mismo inicio
Procedimiento previsto	Intentar reservar un bloque ya ocupado, incluida solicitud desactualizada; revisar persistencia
Resultado esperado	Segunda reserva rechazada; primera permanece; 23P01 si llega al motor
Resultado observado o conocido	No ejecutado; exclusión confirmada en catálogo
Estado del diseño	Aprobada
Evidencia	postgresql_solo_lectura.json; SQL:114–118

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 45

PF05 — Rechazar reserva fuera del horario

Dimensión	Registro de prueba
Código	PF05
Requisito relacionado	RF05; RF06; RF07
Objetivo	Rechazar reserva fuera del horario

Dimensión	Registro de prueba
Precondición	Médico con horario activo de ensayo
Datos de entrada previstos	Inicio fuera de rango, inicio desalineado o bloque que termina después del fin
Procedimiento previsto	Enviar solicitudes controladas a la reserva con cada condición
Resultado esperado	Mensaje de bloque inválido o fuera de horario; ninguna inserción
Resultado observado o conocido	No ejecutado; validaciones identificadas en POST
Estado del diseño	Aprobada
Evidencia	citas.php:246–281

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 46

PF06 — Cancelar cita

Dimensión	Registro de prueba
Código	PF06
Requisito relacionado	RF09
Objetivo	Cancelar cita
Precondición	Reserva vigente cuyo inicio aún no transcurrió
Datos de entrada previstos	ID válido; confirmacion=1; motivo de ensayo; CSRF vigente
Procedimiento previsto	Abrir confirmación; enviar cancelación; revisar estado e historial
Resultado esperado	Estado cancelada; motivo conservado; intervalo liberado; registro no eliminado
Resultado observado o conocido	Cancelación reportada; confirmado un registro cancelado para 2026-10-30 mediante agregado
Estado del diseño	Aprobada
Evidencia	postgresql_solo_lectura.json; cancelar_cita.php

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 47

PF07 — Consultar historial de paciente

Dimensión	Registro de prueba
Código	PF07
Requisito relacionado	RF12
Objetivo	Consultar historial de paciente

Dimensión	Registro de prueba
Precondición	Paciente de ensayo con varias citas
Datos de entrada previstos	id_paciente; filtros de código, estado y fechas
Procedimiento previsto	Abrir desde pacientes; revisar citas; filtrar; recorrer páginas si existen
Resultado esperado	Solo citas de ese paciente, estados conservados y conteos coherentes
Resultado observado o conocido	Consulta e integración reportadas; filtros exhaustivos no repetidos
Estado del diseño	Aprobada
Evidencia	Declaración de avance; historial_citas.php

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 48

PF08 — Consultar citas de médico por fecha

Dimensión	Registro de prueba
Código	PF08
Requisito relacionado	RF11
Objetivo	Consultar citas de médico por fecha
Precondición	Administrador o recepcionista autenticado
Datos de entrada previstos	Médico activo y fecha con/sin citas
Procedimiento previsto	Seleccionar médico y fecha; contrastar orden y conteos con datos de ensayo
Resultado esperado	Agenda del médico y fecha solicitados; resumen por estado
Resultado observado o conocido	No ejecutado de extremo a extremo en esta revisión
Estado del diseño	Aprobada
Evidencia	agenda_medica.php:19–89

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 49

PF09 — Denegar acceso no autorizado

Dimensión	Registro de prueba
Código	PF09
Requisito relacionado	RF01
Objetivo	Denegar acceso no autorizado

Dimensión	Registro de prueba
Precondición	Sesión ausente o rol sin permiso administrativo
Datos de entrada previstos	GET de módulo protegido y endpoint JSON; sin credenciales
Procedimiento previsto	Solicitar URL sin sesión; repetir con médico/paciente en ambiente de pruebas
Resultado esperado	HTML redirige al login; JSON anónimo 401; rol no admitido 403
Resultado observado o conocido	Restricciones reportadas; control estático confirmado; solicitud anónima se registra aparte si ejecutada
Estado del diseño	Aprobada
Evidencia	auth.php:52–70; citas.php:9–27

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 50

PF10 — Rechazar datos inválidos

Dimensión	Registro de prueba
Código	PF10
Requisito relacionado	RF02; RF06; RF08
Objetivo	Rechazar datos inválidos
Precondición	Acceso válido y formularios disponibles
Datos de entrada previstos	Fecha inexistente, ID inválido, obligatorio vacío, motivo largo, CSRF inválido
Procedimiento previsto	Enviar cada clase de entrada; comprobar mensaje y ausencia de cambios
Resultado esperado	Rechazo antes de guardar; preservación de integridad
Resultado observado o conocido	No ejecutado como batería de entradas
Estado del diseño	Aprobada
Evidencia	Validaciones PHP inspeccionadas

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 51

PF11 — Iniciar y cerrar sesión

Dimensión	Registro de prueba
Código	PF11
Requisito relacionado	RF01
Objetivo	Iniciar y cerrar sesión

Dimensión	Registro de prueba
Precondición	Cuenta activa de ensayo de cada rol
Datos de entrada previstos	Nombre y contraseña introducidos privadamente; token de formulario
Procedimiento previsto	Ingresar; verificar identidad; cerrar por POST; intentar volver al módulo
Resultado esperado	Identidad válida, sesión regenerada y posterior acceso protegido
Resultado observado o conocido	Autenticación reportada; no se usaron credenciales durante esta revisión
Estado del diseño	Aprobada
Evidencia	login.php; logout.php; sesion.php

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 52

PF12 — Controlar permisos por rol

Dimensión	Registro de prueba
Código	PF12
Requisito relacionado	RF01; RF13
Objetivo	Controlar permisos por rol
Precondición	Una cuenta de ensayo por cada uno de los cuatro roles
Datos de entrada previstos	Matriz de URL de pacientes, médicos, horarios y reportes
Procedimiento previsto	Acceder directamente a cada URL con cada cuenta, sin depender del menú
Resultado esperado	Solo administrador accede a reportes y catálogos; recepción a gestión permitida
Resultado observado o conocido	Protección administrativa reportada; matriz de código corroborada
Estado del diseño	Aprobada
Evidencia	auth.php; ui.php; encabezado de cada módulo

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

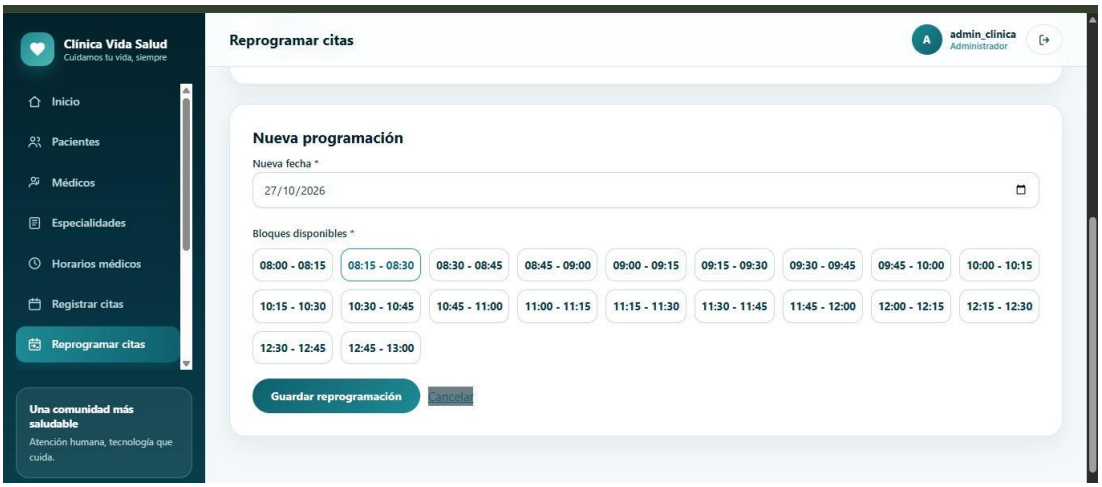
Mensaje de CI duplicado



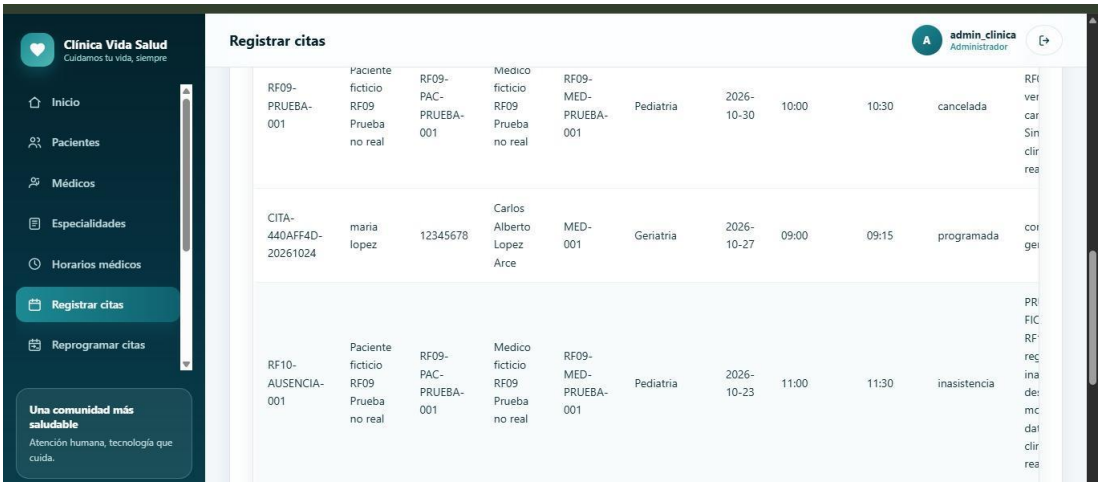
Disponibilidad de turnos



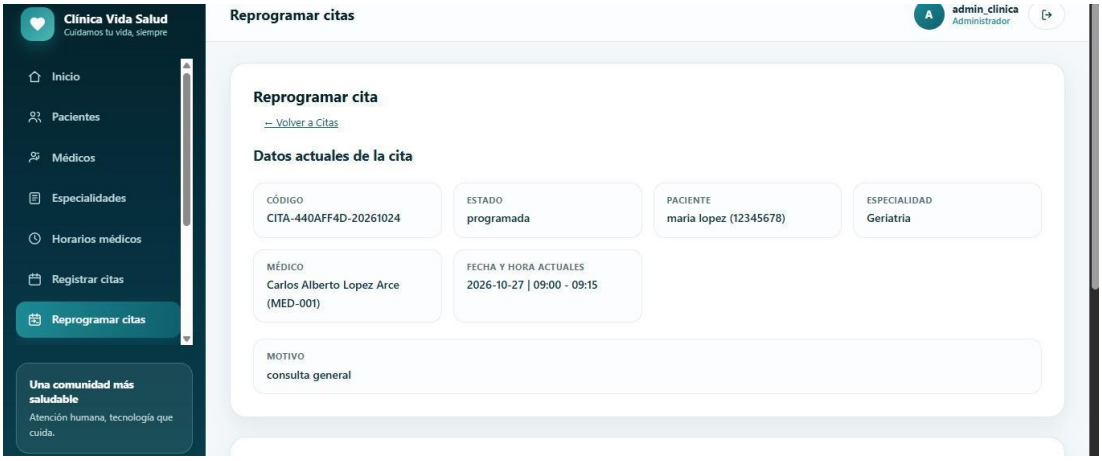
Nuevo horario de reprogramación



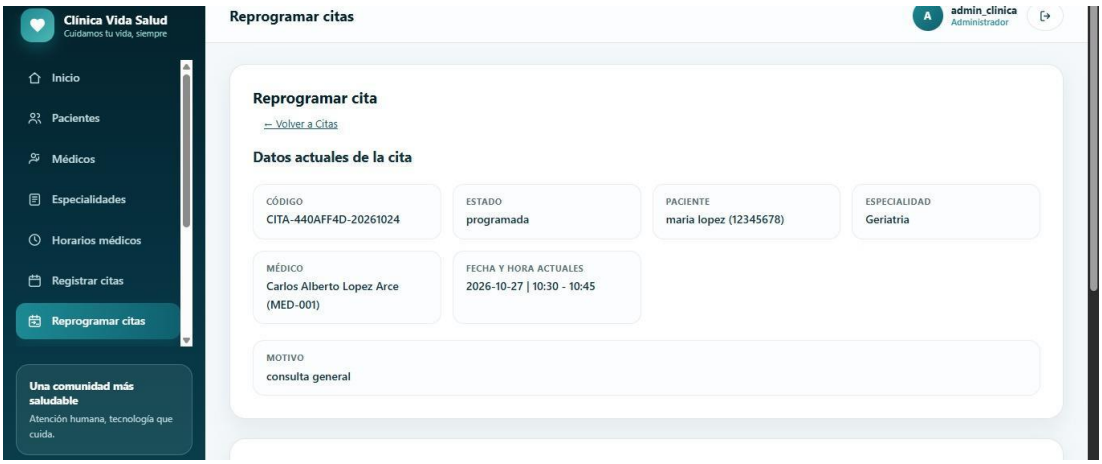
Listado de citas registradas



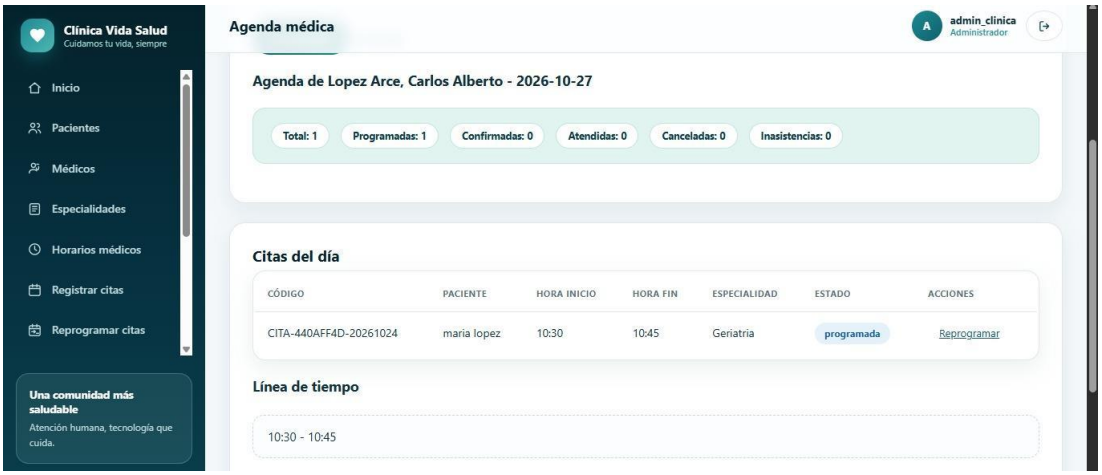
Reprogramación: selección y cambio



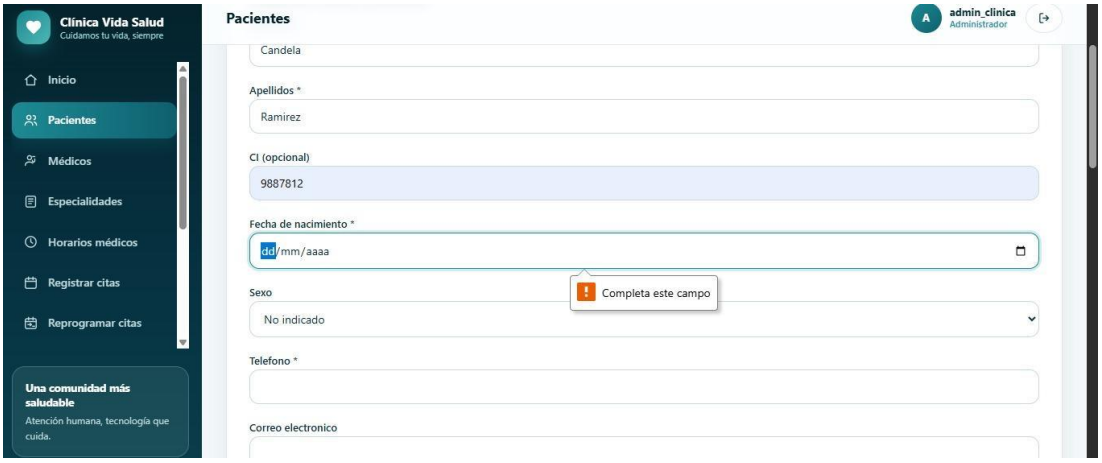
Reprogramación: confirmación



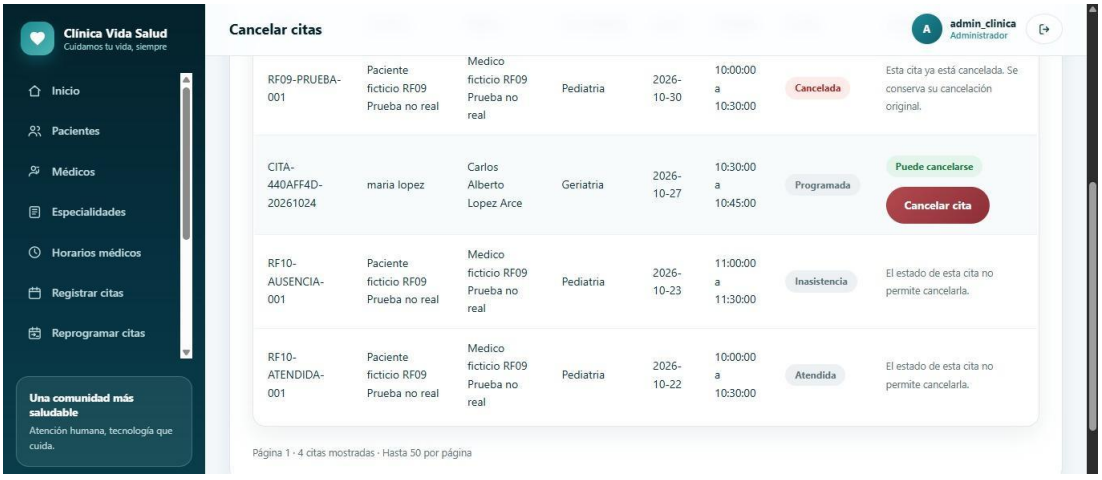
Agenda médica por fecha



Validación del formulario de pacientes



Pantalla de cancelación



Consulta de reportes

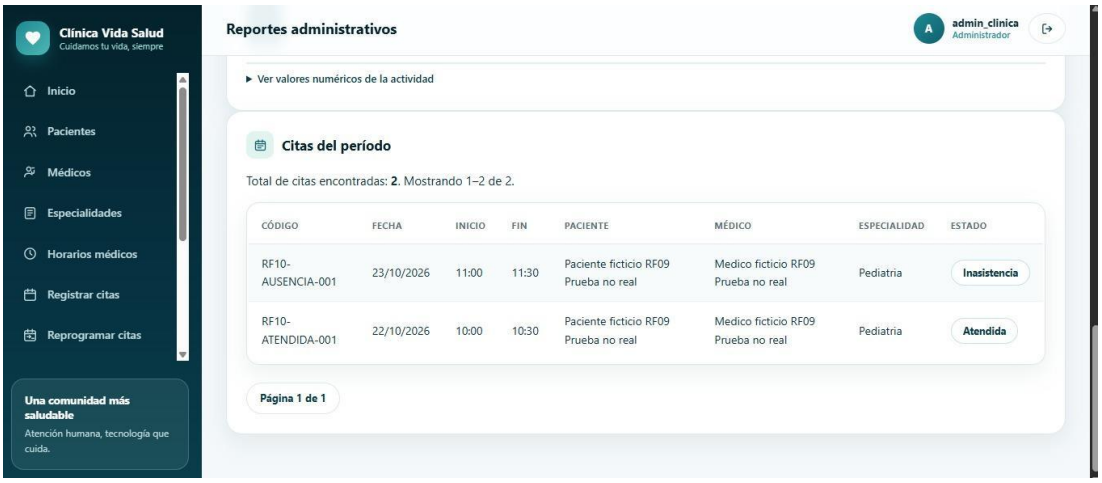


Tabla 53

PF13 — Registrar resultado atendida

Dimensión	**Registro de prueba**
Código	PF13
Requisito relacionado	RF10
Objetivo	Registrar resultado atendida
Precondición	Cita vigente cuyo fin programado ya pasó
Datos de entrada previstos	ID de ensayo; resultado=atendida; confirmación y CSRF
Procedimiento previsto	Abrir confirmación; registrar; revisar estado y repetición rechazada
Resultado esperado	Estado atendida; no modificar otros campos ni permitir rectificación
Resultado observado o conocido	Resultado reportado; agregado de octubre confirma una atendida
Estado del diseño	Aprobada
Evidencia	postgresql_solo_lectura.json; asistencia_citas.php

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 54

PF14 — Registrar inasistencia

Dimensión	**Registro de prueba**
Código	PF14
Requisito relacionado	RF10
Objetivo	Registrar inasistencia
Precondición	Cita vigente cuyo fin programado ya pasó

Dimensión	Registro de prueba
Datos de entrada previstos	ID distinto; resultado=inasistencia; confirmación y CSRF
Procedimiento previsto	Confirmar resultado y revisar historial
Resultado esperado	Estado inasistencia; historial conserva cita
Resultado observado o conocido	Resultado reportado; agregado de octubre confirma una inasistencia
Estado del diseño	Aprobada
Evidencia	postgresql_solo_lectura.json; asistencia_citas.php

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 55

PF15 — Reprogramar integralmente

Dimensión	Registro de prueba
Código	PF15
Requisito relacionado	RF08; RF07
Objetivo	Reprogramar integralmente
Precondición	Cita vigente futura y nuevo turno libre del mismo médico
Datos de entrada previstos	Nueva fecha y bloque alineado; ID y código iniciales registrados
Procedimiento previsto	Seleccionar cita; cargar bloques; cambiar; verificar identidad conservada; ensayar conflicto
Resultado esperado	Nueva fecha y horas; mismo ID, código, paciente, médico y especialidad; rollback en rechazo
Resultado observado o conocido	No ejecutado integralmente
Estado del diseño	Aprobada
Evidencia	reprogramar_cita.php:181–323

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 56

PF16 — Validar estadísticas y filtros completos

Dimensión	Registro de prueba
Código	PF16
Requisito relacionado	RF13
Objetivo	Validar estadísticas y filtros completos

Dimensión	Registro de prueba
Precondición	Administrador y conjunto de ensayo conocido
Datos de entrada previstos	Período 2026-10-01 a 2026-10-24; médico/especialidad/estado; secciones restantes
Procedimiento previsto	Contrastar KPIs y tablas; variar filtros; paginar e imprimir; revisar conjuntos vacíos
Resultado esperado	Totales coincidentes con SELECT; pacientes independiente del período; agenda con filtros propios
Resultado observado o conocido	Visualización reportada; SELECT agregado confirma dos citas: una por cada resultado final indicado
Estado del diseño	Aprobada
Evidencia	postgresql_solo_lectura.json; reportes.php; includes/reportes.php

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 57

PF17 — Navegación adaptable

Dimensión	Registro de prueba
Código	PF17
Requisito relacionado	RNF-UI
Objetivo	Navegación adaptable
Precondición	Navegador y dispositivos de prueba definidos
Datos de entrada previstos	Anchos 360, 640, 768 y escritorio; zoom y teclado
Procedimiento previsto	Recorrer login, menú, formularios, tablas y reportes sin pérdida funcional
Resultado esperado	Controles utilizables; navegación accesible; desbordamiento de tabla controlado
Resultado observado o conocido	No se realizó revisión móvil integral; existen media queries
Estado del diseño	Aprobada
Evidencia	assets/estilos.css:581–608 y 1027–1079

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 58

PF18 — Verificar despliegue público

Dimensión	Registro de prueba
Código	PF18
Requisito relacionado	RNF-DEP
Objetivo	Verificar despliegue público

Dimensión	Registro de prueba
Precondición	Servidor público elegido y configuración preparada
Datos de entrada previstos	URL HTTPS y base de pruebas separada, cuando existan
Procedimiento previsto	Verificar instalación, conexión, permisos, sesiones y operación básica remota
Resultado esperado	Servicio público seguro y reproducible sin exposición de configuración
Resultado observado o conocido	No existe evidencia de despliegue público
Estado del diseño	Aprobada
Evidencia	Solo entorno local acreditado

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 59

PF19 — Validar médico y asignaciones

Dimensión	Registro de prueba
Código	PF19
Requisito relacionado	RF03
Objetivo	Validar médico y asignaciones
Precondición	Administrador; especialidades activas
Datos de entrada previstos	Registro profesional de ensayo; especialidades válidas y asignación histórica
Procedimiento previsto	Crear/editar; intentar duplicado y quitar asignación con citas
Resultado esperado	Unicidad; sincronización transaccional; FK impide eliminar asignación referenciada
Resultado observado o conocido	No ejecutado; procedimiento inspeccionado
Estado del diseño	Aprobada
Evidencia	medicos.php:76–259

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 60

PF20 — Validar unicidad de especialidad

Dimensión	Registro de prueba
Código	PF20
Requisito relacionado	RF04
Objetivo	Validar unicidad de especialidad

Dimensión	Registro de prueba
Precondición	Administrador; especialidad de ensayo existente
Datos de entrada previstos	Nombre con distintas mayúsculas y espacios laterales
Procedimiento previsto	Intentar duplicado; desactivar; buscar; reactivar
Resultado esperado	No duplicar LOWER(TRIM(nombre)); conservar asociaciones al desactivar
Resultado observado o conocido	No ejecutado; índice y consultas confirmados
Estado del diseño	Aprobada
Evidencia	especialidades.php; SQL:64–65

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 61

PF21 — Rechazar horario semanal solapado

Dimensión	Registro de prueba
Código	PF21
Requisito relacionado	RF05
Objetivo	Rechazar horario semanal solapado
Precondición	Administrador; horario activo del mismo médico y día
Datos de entrada previstos	Intervalos parcialmente coincidentes; día y duración inválidos
Procedimiento previsto	Intentar alta y reactivación que solapen; ensayar bloques adyacentes
Resultado esperado	Rechazo por exclusión en solapamiento; adyacencia aceptable; CHECK para valores inválidos
Resultado observado o conocido	No ejecutado; restricciones confirmadas en catálogo
Estado del diseño	Aprobada
Evidencia	horarios.php; SQL:73–90

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 62

PF22 — Concurrencia de reservas y reprogramación

Dimensión	Registro de prueba
Código	PF22
Requisito relacionado	RF07; RF08
Objetivo	Concurrencia de reservas y reprogramación

Dimensión	Registro de prueba
Precondición	Dos sesiones y datos exclusivos de ensayo
Datos de entrada previstos	Mismo médico e intervalo libre inicial; cita reprogramable cerca del inicio
Procedimiento previsto	Enviar simultáneamente dos reservas; comprobar bloqueo y desenlace; repetir cambio de estado concurrente
Resultado esperado	A lo sumo una reserva vigente por intervalo; ninguna actualización inconsistente
Resultado observado o conocido	No ejecutado; no se simularon escrituras concurrentes
Estado del diseño	Aprobada
Evidencia	EXCLUDE; reprogramar_cita.php:209–297

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Resultados conocidos y verificaciones efectivamente realizadas

La información de avance suministrada reporta autenticación y restricciones por roles, protección de módulos administrativos, consulta de turnos, registro válido, cancelación, resultados de atendida e inasistencia, consulta de historial e integración desde pacientes, visualización de RF13 y verificaciones técnicas de PHP/Git. Se conserva esa procedencia y no se presentan esos recorridos como repetidos en la presente revisión.

La consulta agregada de la base local verificó que entre el 1 y el 24 de octubre de 2026 existen dos citas: una atendida y una con inasistencia. También se encontró una cita cancelada para el 30 de octubre de 2026. Son datos de demostración; no se exponen nombres, códigos de personas ni contenido clínico. Los conteos corroboran el estado actual almacenado, no la autoría ni el momento de cada operación.

Se ejecutó la comprobación php -l para los 25 archivos PHP versionados y todos devolvieron salida de sintaxis válida. Se registraron el estado inicial de Git y el commit de base, y se inspeccionaron columnas, restricciones e índices mediante consultas de solo lectura. No se

ejecutaron los scripts de alta de cuentas ni el script de creación de esquema. Una comprobación de sintaxis no evalúa permisos efectivos, diseño visual o cumplimiento del flujo de reserva.

Tabla 63
Registro de evidencia y nivel de afirmación

Evidencia	Qué verifica	Qué no demuestra por sí sola
sintaxis_php.json	25 fuentes PHP con salida 0 de php -l.	Éxito funcional ni ausencia de errores de lógica.
postgresql_solo_lectura.json	Versión, ocho tablas, columnas, restricciones, índices y agregados anónimos.	Ejecución nueva de alta, cambio o rechazo desde la interfaz.
git_inicial.json	Rama, commit y ausencia de cambios al iniciar.	Distribución histórica completa de trabajo por integrante.
manifiesto_inicial.json	Huellas de archivos versionados para contraste final.	Resultados de pruebas de usuarios finales.
Declaración de avance de la solicitud	Operaciones comunicadas como comprobadas durante desarrollo.	Bitácora paso a paso no aportada.
Fuentes .puml y exportaciones	Modelos compilados y editables como fuente.	Implementación física de clases conceptuales.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Pruebas pendientes y evaluación de riesgos

Se mantienen pendientes CI duplicado, superposición real y concurrente, reserva fuera del horario, reprogramación integral, filtros completos de agenda, batería completa de reportes, compatibilidad móvil integral y despliegue en la nube. La inspección técnica aporta indicios de protección, pero no cambia el estado de esas pruebas a aprobada. Para las operaciones reportadas también conviene registrar posteriormente un recorrido reproducible con datos de ensayo y contraste de persistencia.

Los valores de frontera prioritarios son el inicio exacto para cancelar y reprogramar, el fin exacto para registrar asistencia, un bloque que termina en el límite del horario y dos intervalos adyacentes. La prueba concurrente debe conservar tanto el resultado aceptado como el rechazo de la segunda solicitud y el estado final, porque dos pantallas con mensajes no

garantizan por sí solas consistencia persistida. La validación de catálogo desactivado y de cambio de rol durante una operación completa el análisis de riesgos identificado.

Capítulo complementario. Trazabilidad de modelos

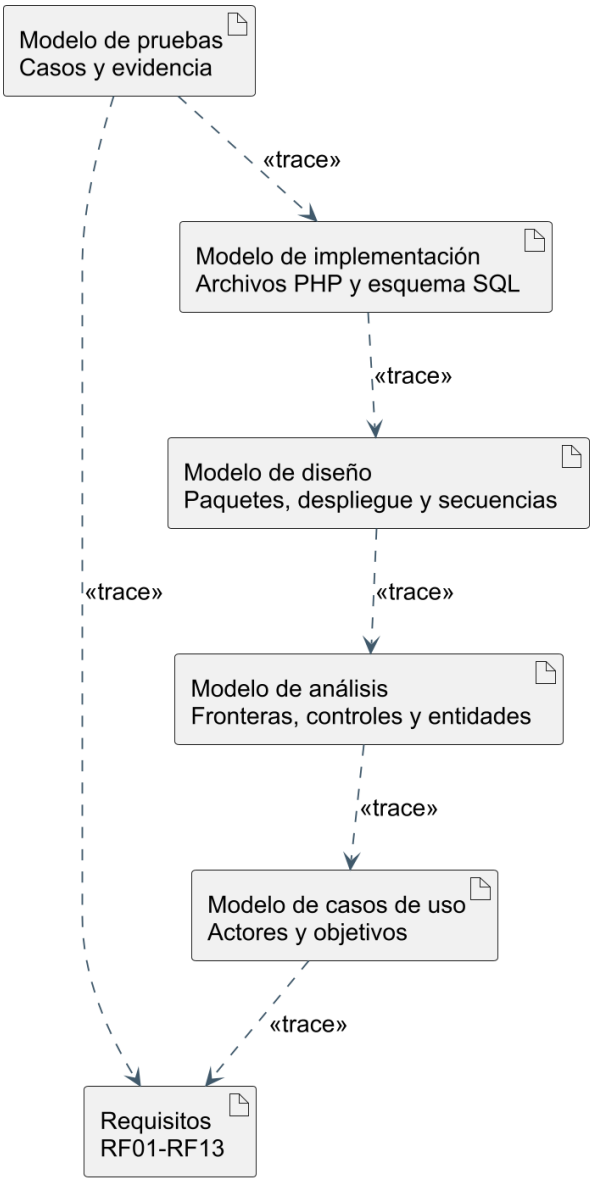
Propósito y criterios de vinculación

La trazabilidad permite reconstruir la justificación de cada elemento y detectar qué debe revisarse ante un cambio. SWEBOK (s. f.-a) distingue el seguimiento hacia el origen de un requisito y hacia los diseños, código y pruebas que lo realizan. En esta entrega el vínculo se organiza como Requisito → Caso de uso → Elemento de análisis → Elemento de diseño → Archivo → Prueba. No es una relación uno a uno ni una prueba de que todos los artefactos estén aprobados.

Los identificadores de requisitos, casos, diagramas y pruebas son estables en todo el documento. La matriz completa agrega paquete, clase conceptual, tabla PostgreSQL, estado y evidencia. Las clases de análisis se rotulan como conceptuales; el archivo PHP indica la construcción realmente implementada. Cuando un requisito incluye alcance futuro, su estado conserva esa diferencia, particularmente RF01, RF11 y RF12.

Figura 25

D19 — Trazabilidad entre los modelos del ciclo



Nota. Elaboración propia basada en código y esquema inspeccionados. UML de artefactos y trazas. Fuente editable: D19.puml.

Las dependencias de trazabilidad unen artefactos de distinto nivel de abstracción. No representan llamadas de ejecución ni una cadena de fases obligatoriamente lineal. Un requisito puede realizarse mediante varios casos y pruebas; RF07 es transversal a registro y reprogramación. La matriz de control permite recorrer también el vínculo inverso desde el archivo hasta la necesidad.

Tabla 64

Trazabilidad de requisitos a modelos y pruebas — RF01 a RF07

RF y objetivo	CU	Análisis conceptual	Diseño	Implementación	Prueba
RF01 Seguridad y autenticación	CU-AUT; CU-SAL	ControlAcceso; usuarios; roles	D09	login.php; logout.php; includes/auth.php; includes/sesion.php; includes/conexion.php	PF09; PF11; PF12
RF02 Registro y administración de pacientes	CU-PAC; CU-PAC-ALTA	FormularioPaciente; ControlPacientes; pacientes	D16	pacientes.php	PF01; PF02
RF03 Registro y administración de médicos	CU-MED	ControlOferta; medicos	D03; D08	medicos.php	PF19
RF04 Administración de especialidades	CU-ESP	ControlOferta; especialidades	D03; D08	especialidades.php	PF20
RF05 Administración de horarios médicos	CU-HOR; CU-DIS	ControlHorarios; horarios	D10A; D18B	horarios.php; citas.php; includes/dashboard.php	PF05; PF21
RF06 Registro de citas médicas	CU-REGISTRAR-CITA; CU-DIS	FormularioCita; ControlReservas; citas	D04; D10A; D10B; D21; D10C	citas.php	PF03; PF10
RF07 Prevención de citas duplicadas	CU-REGISTRAR-CITA; CU-REP; CU-DIS	ControlReservas; citas; horarios	D10A; D10B; D11; D10C	citas.php; reprogramar_cita.php; database/clinica_vida_salud.sql	PF04; PF05; PF22

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Tabla 65

Trazabilidad de requisitos a modelos y pruebas — RF08 a RF13

RF y objetivo	CU	Análisis conceptual	Diseño	Implementación	Prueba
RF08 Reprogramación de citas	CU-REP; CU-DIS	ControlReservas; citas	D11	seleccionar_reprogramacion.php; reprogramar_cita.php	PF15
RF09 Cancelación de citas	CU-CAN	ControlSeguimiento; citas	D12	cancelar_cita.php	PF06
RF10 Control de asistencia	CU-ASI	ControlSeguimiento; citas	D13	asistencia_citas.php	PF13; PF14

RF y objetivo	CU	Análisis conceptual	Diseño	Implementación	Prueba
RF11 Consulta de agenda médica	CU-AGE; CU-MIA	ControlConsultas; citas; medicos	D01; D08	agenda_medica.php	PF08
RF12 Historial de citas por paciente	CU-HIS; CU-MIP	ControlConsultas; pacientes; citas	D14	historial_citas.php; pacientes.php	PF07
RF13 Reportes administrativos	CU-INF	ControlReportes; citas; pacientes	D15	reportes.php; includes/reportes.php; includes/dashboard.php	PF16

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Control interno de consistencia

El archivo matriz_trazabilidad.xlsx contiene once columnas de control interno: requisito, caso de uso, paquete de análisis, clase conceptual, elemento de diseño, diagrama, archivo PHP, tabla PostgreSQL, caso de prueba, estado y evidencia. Se agregan hojas de casos de uso, pruebas, campos y fuentes para facilitar la revisión. El mismo inventario alimenta las tablas del Word, evitando mantener dos conjuntos de identificadores distintos.

La revisión verifica que cada RF01–RF13 tenga al menos un caso, un elemento de análisis, un diseño, un archivo y una prueba planificada; comprueba la existencia de los archivos y tablas referidos y la inclusión de los diagramas citados. Para CU-MIA, CU-MIP y CU-USU la traza hacia implementación se conserva como pendiente, no se inventa una pantalla. Un caso de uso agrupado mantiene sus actores en el catálogo aunque la vista de paquetes omita asociaciones para reducir densidad gráfica.

Como ejemplo, RF06 enlaza CU-REGISTRAR-CITA con FormularioCita, ControlReservas y citas; D10A/D10B describe selección y confirmación; citas.php implementa GET JSON y POST; citas y las tablas de oferta participan en la persistencia; PF03, PF05, PF10 y PF22 verifican resultado, límites y concurrencia junto con RF07. Si cambia la duración de los

turnos, se debe revisar horarios, cálculo de bloques, ambas secuencias y las pruebas de alineación, y no únicamente editar una figura.

Conclusiones

El primer ciclo dispone de un núcleo administrativo de gestión de citas integrado en PHP y PostgreSQL. Se identificaron módulos de acceso, pacientes, oferta médica, horarios, reserva, reprogramación, cancelación, resultado de asistencia, agenda, historial y reportes. Su documentación establece correspondencia entre objetivos, responsabilidades, interacciones y persistencia, sin convertir las abstracciones UML en clases PHP inexistentes ni declarar finalizado el alcance completo.

La captura de requisitos permitió distinguir los cuatro actores y separar servicios administrativos de servicios personales pendientes. La relación entre RF07 y varios casos de uso muestra que la consistencia de reservas no pertenece a una sola pantalla. El análisis explicó la colaboración entre identidad, pacientes, oferta y cita, y permitió señalar que historial administrativo, historia clínica y auditoría de reprogramación no son conceptos intercambiables.

El diseño arquitectónico aclaró la ejecución cliente-servidor local y la distribución real de responsabilidades procedimentales. Los modelos de datos y el catálogo observado confirmaron la asignación médico-especialidad mediante clave compuesta y el uso de exclusiones GiST para intervalos. La prevención de superposición depende de reglas PHP y del motor, con límites concretos: se protege al médico para reservas vigentes y no se acredita una exclusión por paciente.

Las verificaciones ejecutadas aportaron sintaxis válida de 25 fuentes PHP y evidencia de metadatos y agregados de demostración. Se corroboraron dos resultados finales dentro del período indicado y una cancelación futura, sin exponer datos personales. Las pruebas funcionales reportadas se distinguieron de las pendientes, por lo que estos resultados no constituyen aprobación total ni despliegue productivo.

Permanecen como riesgos las pruebas no ejecutadas de concurrencia y de fronteras temporales; la validación integral de filtros y dispositivos; la funcionalidad de acceso por correo electrónico prevista en RF01; y la implementación de pantallas personales y administración web de usuarios. El contraste con el enunciado y el material metodológico permite precisar estas brechas sin confundir implementación con pruebas aprobadas. La evaluación del porcentaje de avance corresponde a la rúbrica docente.

Recomendaciones

Mantener el enunciado oficial, el documento PUDS-UML y la fotografía de la pizarra entre las referencias de control; actualizar los modelos cuando se modifique un requisito. Revisar especialmente el acceso por correo electrónico exigido en RF01 y la administración completa de usuarios, distinguiendo funcionalidad solicitada, implementada y probada. Conservar la matriz de trazabilidad y los procedimientos de prueba como evidencia del avance incremental.

Completar las pantallas personales de médico y paciente con autorización basada en propiedad de datos, y finalizar la administración de cuentas desde la web. La existencia de cvs_exigir_propietario ofrece una función preparada para uso futuro, pero no equivale a una integración terminada. Las nuevas pantallas deben verificar identidad y ámbito en servidor y generar sus propios casos de prueba.

Ejecutar primero las pruebas de duplicidad, horario, concurrencia y transiciones temporales en un conjunto separado de datos ficticios. Conservar entradas, estado inicial, resultado observado y evidencia de persistencia. Después ampliar el contraste de reportes, agenda, historial, navegación adaptable y accesibilidad. Las capturas deberán ser auténticas, identificar el escenario y evitar credenciales o datos privados.

Revisar las diferencias de generación de disponibilidad y tratamiento temporal para reducir el riesgo de divergencia entre registro, reprogramación y dashboard. Verificar la política de especialidades inactivas y la comprobación del inicio original después de esperas concurrentes al reprogramar. Revisar mensajes y codificación visual identificados, sin confundir estas mejoras con funciones ya corregidas.

Planificar el despliegue público únicamente después de verificar configuración, HTTPS, permisos y respaldo. Mantener la configuración privada fuera del repositorio y una estrategia de

recuperación comprobable. Continuar usando control de versiones para relacionar cambios con requisitos y pruebas, y conservar esta entrega como línea base sin sustituir documentos previos de las integrantes.

Referencias bibliográficas

Bourgeois, D. (2019). *Chapter 1: What is an information system?* En Information systems for business and beyond (2019). https://opentextbook.site/informationsystems2019/chapter/chapter-1-what-is-an-information-system-information-systems-introduction/

Grupo 17. (2026). *Clínica Vida Salud [Código fuente, rama principal main].* GitHub. https://github.com/yllbbtjqmss/clinica-vida-salud

MDN contributors. (s. f.). *Descripción general de cliente-servidor.* MDN Web Docs. Recuperado el 24 de octubre de 2026, de https://developer.mozilla.org/es/docs/Learn_web_development/Extensions/Server-side/First_steps/Client-Server_overview

Object Management Group. (2017). *Unified modeling language (version 2.5.1).* https://www.omg.org/spec/UML/2.5.1

PostgreSQL Global Development Group. (s. f.-a). *5.5. Constraints.* PostgreSQL 18 documentation. Recuperado el 24 de octubre de 2026, de https://www.postgresql.org/docs/18/ddl-constraints.html

PostgreSQL Global Development Group. (s. f.-b). *8.17. Range types.* PostgreSQL 18 documentation. Recuperado el 24 de octubre de 2026, de https://www.postgresql.org/docs/18/rangetypes.html

SWEBOK. (s. f.-a). *Chapter 1: Software requirements.* Recuperado el 24 de octubre de 2026, de https://swebokwiki.org/Chapter_1:_Software_Requirements

SWEBOK. (s. f.-b). *Chapter 4: Software testing.* Recuperado el 24 de octubre de 2026, de https://swebokwiki.org/Chapter_4:_Software_Testing

SWEBOK. (s. f.-c). *Chapter 8: Software engineering process.* Recuperado el 24 de octubre de 2026, de https://swebokwiki.org/Chapter_8:_Software_Engineering_Process

The PHP Documentation Group. (s. f.-a). *Consultas preparadas y procedimientos almacenados.* Manual de PHP. Recuperado el 24 de octubre de 2026, de https://www.php.net/manual/es/pdo.prepared-statements.php

The PHP Documentation Group. (s. f.-b). *password_verify.* Manual de PHP. Recuperado el 24 de octubre de 2026, de https://www.php.net/manual/es/function.password-verify.php

Torossi, G. (s. f.). El Proceso Unificado de Desarrollo de Software [Material de apoyo de Diseño de Sistemas, 54 páginas].

El enunciado oficial y el material metodológico orientan la estructura y la justificación de los modelos. Las observaciones técnicas sobre el estado de la aplicación se basan en los archivos de implementación y en los resultados registrados de comprobación. Las operaciones no ejecutadas se documentan como pendientes y no como éxitos de prueba.

Anexos

Anexo A. Inventario de evidencias y fuentes consultadas

Tabla 66
Materiales de referencia solicitados y disponibilidad

Material	Disponibilidad	Uso o limitación
PDF PUDS-UML atribuido a Gustavo Torossi	Consultado	54 páginas; metodología y actividades en pp. 26–54.
Fotografía de pizarra	Consultada	Secuencia de captura 1.1–1.5, análisis 2.1–2.4 y diseño 3.1–3.2.
Enunciado oficial del segundo parcial	Consultado	RF01–RF13, diez pruebas obligatorias, estructura y fechas del ciclo.
Capturas del sistema	No incorporadas al proyecto	No se fabrican evidencias; inventario de interfaz basado en fuentes.
Código y esquema	Disponibles e inspeccionados	Base principal de correspondencia de esta entrega.
Catálogo PostgreSQL y agregados	Consultados en solo lectura	Evidencia JSON anonimizada del entorno local.

Nota. Elaboración propia a partir del repositorio y del esquema SQL inspeccionado.

Anexo B. Registro técnico de revisión

La versión del proyecto se encuentra publicada en la rama principal main. El estado inicial de la documentación y el commit de referencia se conservan en evidencias/git_inicial.json. El inventario SHA-256 permite comprobar al cierre que la documentación no alteró archivos versionados de la aplicación. Las verificaciones finales de Git

y la validación visual/editorial se registran en control_documental/informe_validacion.md para que el estado técnico no dependa solo de una afirmación narrativa.

Las consultas de inspección solo solicitaron versión, catálogo público, nombres de roles y agregados de citas. No se reproducen configuraciones, contraseñas, hashes de cuentas, tokens de sesión ni filas de pacientes. El código de inspección está restringido a consola y su transacción se configuró explícitamente como de solo lectura. La conservación de fuentes intermedias permite auditar los procedimientos de documentación sin copiar todo el proyecto.

Anexo C. Modelo físico completo y fuentes editables

El modelo físico completo D18 se conserva en SVG y PNG; las proyecciones D18A y D18B incorporadas en el capítulo de datos facilitan lectura impresa. Todos los diagramas de la entrega tienen fuente PlantUML .puml en diagramas/fuentes y exportaciones en diagramas/exportados. Un SVG es una exportación vectorial editable gráficamente; la fuente .puml conserva la estructura de modelado y es la referencia para cambiar relaciones. No se entrega un .mdj ni se presenta un PNG como modelo UML editable.

Relación entre la vista completa y las vistas impresas

El archivo D18.svg conserva la vista integral de las ocho tablas para consulta vectorial y edición mediante D18.puml. Para la presentación impresa se incorporaron D18A y D18B en el capítulo de datos, evitando reducir la vista integral a un tamaño de letra poco legible. Las entidades compartidas entre ambas vistas permiten reconstruir el conjunto; el diccionario físico enumera los sesenta campos contrastados con el catálogo local.

Anexo D. Repositorio y código QR

Repositorio oficial del proyecto: https://github.com/yllbbtjqmss/clinica-vida-salud. La rama principal main contiene la versión integrada del sistema Clínica Vida Salud. El siguiente código QR dirige al repositorio oficial, cuyo acceso predeterminado muestra main. La publicación del código en GitHub no debe confundirse con un despliegue público y operativo de la aplicación.

Figura 26
Código QR del repositorio oficial de Clínica Vida Salud



Nota. Elaboración propia. Destino: https://github.com/yllbbtjqmss/clinica-vida-salud
https://github.com/yllbbtjqmss/clinica-vida-salud